
FluencyOS

Repository structure, and how the Conversation feature works

Written for: engineers joining or maintaining this codebase

Scope: repository layout · conversation architecture · complete source of every
file that implements the feature

Stack: Electron 33 · React 18 · FastAPI · SQLite (WAL) · llama.cpp · faster-whisper · Pocket TTS

1. What this is

FluencyOS is a local-first language-learning desktop application. Everything that matters — the speech recogniser, the language model, the voice, the database — runs on the user's own machine. The conversation feature is the part of the app where that promise is most visible and most demanding: it holds a spoken conversation with the learner, in real time, using three separate AI models at once, on consumer hardware. This document explains the repository layout, then walks through how that feature actually works, then reproduces every file that implements it in full.

2. Repository structure

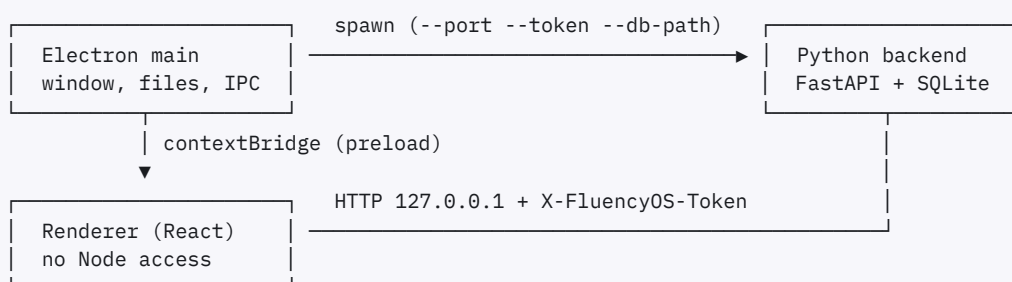
Three processes, three top-level directories, plus a shared test suite.

```
fluency-os/
├── electron/                                Desktop shell – the only process with OS access
│   ├── main/
│   │   ├── index.ts                        Creates the window, owns the app lifecycle
│   │   ├── backend-process.ts             Spawns the Python backend, picks a free port,
│   │   │                                   mints the per-launch handshake token
│   │   ├── handshake.ts                   Token generation
│   │   ├── ipc-handlers.ts                File pickers, window controls, UI scale
│   │   ├── scene-embeds.ts                Referer header for the YouTube scene embeds
│   │   └── logger.ts
│   └── preload/index.ts                    contextBridge – the ONLY surface the renderer
│                                           has to Node/Electron APIs
├── renderer/                               React 18 + Vite + Tailwind (Chromium)
│   ├── index.html                          CSP lives here
│   └── src/
│       ├── App.tsx                         Screen switch (no router – a switch statement)
│       ├── lib/apiClient.ts                fetch wrapper; attaches the handshake token
│       ├── store/                          Zustand stores, one per feature
│       │   ├── conversationStore.ts        ← conversation state
│       │   ├── appStore.ts, shellStore.ts, engineStore.ts
│       │   └── mediaStore.ts, vocabularyStore.ts, reviewStore.ts ...
│       ├── features/
│       │   ├── conversation/               ← the conversation screens (see §3)
│       │   ├── player/                     Learn by watching
│       │   ├── reader/ bookshelf/          vocabulary/ review/ challenge/ ...
│       │   └── shell/                       Nav, header, window chrome
│       └── types/api.ts                     Hand-written mirrors of the backend models
├── backend/                                FastAPI + SQLite, spawned by Electron
│   └── app/
│       ├── main.py                          App factory, router registration, startup tasks
│       ├── db.py                            Per-request SQLite connection (WAL)
│       ├── config.py                        CLI flags from Electron (--port --token --db-path)
│       ├── security.py                      require_token – every route depends on it
│       ├── migrations/                     Numbered .sql, applied in order at startup
│       │   ├── 0008_conversation.sql        ← conversation schema
│       │   └── 0012_turn_speech_seconds.sql ← speech duration
│       ├── models/                         Pydantic request/response shapes
│       │   ├── conversation.py              ← conversation API contract
│       │   └── routers/                     HTTP surface, one module per feature
│       └── conversation.py                  ← conversation endpoints
```



3. How the three processes fit together

The application is three operating-system processes, not one. **1. The Electron main process** is the only part with real OS access. On launch it picks a free TCP port, mints a random token for this launch only, and spawns the Python backend with both as command-line flags. It then creates the window. Nothing else in the app may read a file or spawn a process. **2. The renderer** is ordinary Chromium running React. It has no Node access at all; a `contextBridge` in the preload script exposes a short, explicit list of functions and nothing more. It reaches the backend over plain HTTP on 127.0.0.1, attaching the handshake token to every request. **3. The Python backend** is FastAPI over SQLite. It holds the models, the database and all the logic. Every route depends on `require_token`, so no other process on the machine can call it even though the port is open locally.



The consequence worth remembering: **the renderer never touches a model, a file or the database**. Every capability it appears to have is a request to the backend.

4. The conversation feature

A conversation session is a guided spoken (or typed) practice run. The learner picks a scenario — free chat, a job interview, a doctor's visit — and the system injects a handful of words from their own vocabulary that are due for review. The AI plays the other side of the conversation and is told to work those words into what it says. At the end, the session is analysed and the results feed back into the spaced-repetition scheduler. Three models are involved, and on the hardware this app targets they cannot all be resident at once without care:

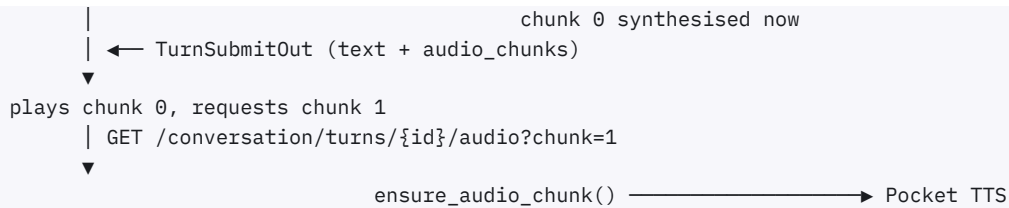
Role	Engine	Where
Speech → text	faster-whisper <code>tiny.en</code>	<code>voice/stt_engine.py</code>
The reply	llama.cpp (local) · OpenRouter · Gemini	<code>voice/llm_chat_engine.py</code> and friends
Text → speech	Pocket TTS (Kyutai)	<code>voice/pocket_tts_engine.py</code>

Because of that, models are **not** loaded on demand. The learner presses “Launch AI” explicitly, and until they do, the conversation endpoints refuse with a message that says what to do. That gate is `_ensure_launched()` in the service.

5. The journey of one spoken turn

This is the path of one spoken turn, and it is the single most useful thing to understand about the feature. Every numbered step is a real function.





Why generation happens before persistence (step 5). If the user turn were written first and the model then failed, the session would be left with a user turn and no reply — and the next request would rebuild a history ending on the user, which the chat template rejects. Generating first means a failure leaves the database exactly as it was. **Why the history is normalised (step 3).** Gemma's chat template requires strict `user / assistant / user / assistant` alternation starting with `user`. This was measured, not assumed: a history of `system + assistant + user` fails outright, while `system + user + assistant + user` works. Before `normalise_for_chat` existed, every conversation died immediately after the greeting, because the greeting is an assistant turn and it came first. The function collapses consecutive same-role turns and drops a leading assistant message. **Why the reply is split into two chunks (step 6).** Pocket TTS is autoregressive, so the first audio arrives sooner for a shorter piece of text. Splitting the reply in two and synthesising only the first chunk before responding cuts the silence the learner sits through. The split point is chosen at a sentence boundary by `reply_chunking.split_for_streaming`. Two was measured as better than one, and more than two gave no further benefit while making the prosody audibly restart.

6. The engines

Three providers, one interface. The learner may run the model locally or supply an API key. `llm_target()` resolves which, and `_generate_reply()` dispatches on it. The local path is llama.cpp through `llama-cpp-python`; the cloud paths are OpenRouter and Gemini. All three expose the same two functions — `generate_chat` and `generate_json` — so the service never branches beyond the dispatcher. **Model JSON is not trusted.** Small local models routinely emit JSON wrapped in prose, fenced in markdown, or with an unclosed bracket. Rather than failing the analysis, `voice/json_utils.py` strips fences, finds the outermost object, and repairs unbalanced brackets before parsing. This is why a session still produces a report when a 1B model rambles. **Loading and unloading.** `launch_engines()` loads exactly the engines the chosen channel needs — text needs the LLM only, voice needs all three — and `unload_engines()` frees them. The app also unloads on quit, so closing the window returns the memory.

7. The post-session report

When a session ends, `end_session()` produces a report. Its design has one rule: **every number is either computed or judged, and the two are never mixed.**

Computed from the transcript	Judged by the model
words per minute (from real speech seconds)	grammatical precision
filler rate per 100 words	the three most instructive errors
average response delay	self-corrections
longest uninterrupted run	per-word usage classification
type-token ratio, above-level words	summary

The judged half is described by a single Pydantic model, `LlmReportAnalysis`, and the prompt that asks for it is rendered *from that model's own fields*. The prompt therefore cannot drift from what the reader parses — a class of bug that had previously left four report fields permanently zero, because three copies of the prompt asked for different keys than the reader consumed. **Speech time is real.** `words_per_minute` divides learner words by the summed duration of recognised speech segments, not by wall clock — wall clock includes the model thinking and the AI talking, and is not a speech rate. A text-channel session reports `None` rather than a number that means nothing. **Results feed the scheduler.** Each target word's usage is rated on the same scale a flashcard answer uses — spontaneous → Easy, prompted → Good, incorrect → Again, avoided → Hard — and applied through the same FSRS code path. Using a word in conversation moves its schedule exactly as recognising it on a card would. That is the central argument the project makes, and `review.apply_conversation_outcomes()` is where it happens.

8. The renderer side

The store owns the session; the screen owns the microphone. `conversationStore.ts` is a Zustand store holding the session, its turns, the engine status and the report. It never touches audio hardware — every method is a request. **Voice capture is voice-activated, not push-to-talk.** `useVadRecorder.ts` holds the microphone open, measures frame energy, and decides when an utterance starts and ends. Two details matter: it keeps a rolling pre-roll buffer so the first syllable is not clipped, and it raises its threshold while the AI is speaking so the reply coming out of the speakers does not trip detection and interrupt itself.

Word-by-word highlighting. `spokenTiming.ts` distributes the reply's words across the known duration of each audio chunk, weighted by word length, so `SpokenText.tsx` can light up each word as it is spoken. It is an estimate — Pocket TTS does not return word timings — and it is deliberately an even, length-weighted estimate rather than a guess dressed up as data.

9. The complete source

Every file that implements the conversation feature, in full, grouped by role. Each is preceded by a note on what it is for and what to look at first.

Database schema

1. backend/app/migrations/0008_conversation.sql

The four tables the feature lives in. Sessions, turns, the target words injected into a session, and the report. Note that target words reference vocab_words, so a session is always about the learner's own vocabulary.

backend/app/migrations/0008_conversation.sql

46 lines

```
1  -- Conversation increment – real sessions/turns backed by a local voice
2  -- pipeline (see app/services/voice/), plus review_logs: the honest half of
3  -- "Dynamic SRS Routing" that's actually buildable today.
4  --
5  -- No mastery/due/interval/rating columns anywhere here, on purpose: there is
6  -- no FSRS scheduler in this app yet (vocab_words has none either, see
7  -- 0006_vocabulary.sql's own comment), so review_logs records what really
8  -- happened (a target word's real usage outcome in a real conversation)
9  -- without inventing a fake schedule to route it into.
10
11 CREATE TABLE IF NOT EXISTS conversation_sessions (
12   id                TEXT PRIMARY KEY,
13   user_id           TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
14   scenario          TEXT NOT NULL,           -- 'free' | 'coffee' | 'job' | 'debate'
15   channel           TEXT NOT NULL CHECK (channel IN ('voice', 'text')),
16   target_word_ids   TEXT NOT NULL DEFAULT '[]', -- JSON array of vocab_words.id
17   model_id          TEXT,                   -- e.g. "qwen2.5-1.5b-instruct-q4_k_m", frozen at session start
18   report_json       TEXT,                   -- populated at end_session; NULL until then
19   started_at        TEXT NOT NULL,
20   ended_at          TEXT
21 );
22 CREATE INDEX IF NOT EXISTS idx_conversation_sessions_user ON conversation_sessions(user_id, started_at);
23
24 CREATE TABLE IF NOT EXISTS conversation_turns (
25   id                TEXT PRIMARY KEY,
26   session_id        TEXT NOT NULL REFERENCES conversation_sessions(id) ON DELETE CASCADE,
27   turn_index        INTEGER NOT NULL,
28   speaker           TEXT NOT NULL CHECK (speaker IN ('user', 'ai')),
29   text              TEXT NOT NULL,
30   audio_path        TEXT,                   -- local wav file path, voice channel only
31   stt_confidence    REAL,                   -- 0-1 proxy from faster-whisper avg_logprob; user turns from audio only
32   created_at        TEXT NOT NULL
33 );
34 CREATE INDEX IF NOT EXISTS idx_conversation_turns_session ON conversation_turns(session_id, turn_index);
35
36 CREATE TABLE IF NOT EXISTS review_logs (
37   id                TEXT PRIMARY KEY,
38   user_id           TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
39   vocab_word_id     TEXT NOT NULL REFERENCES vocab_words(id) ON DELETE CASCADE,
40   session_id        TEXT REFERENCES conversation_sessions(id) ON DELETE SET NULL,
41   source            TEXT NOT NULL CHECK (source IN ('conversation')),
42   outcome           TEXT NOT NULL CHECK (outcome IN ('spontaneous', 'prompted', 'incorrect', 'avoided')),
43   created_at        TEXT NOT NULL
44 );
45 CREATE INDEX IF NOT EXISTS idx_review_logs_word ON review_logs(vocab_word_id, created_at);
```

2. backend/app/migrations/0012_turn_speech_seconds.sql

Adds speech_seconds to a turn. Without it, words-per-minute could only be computed against wall clock, which includes the model thinking and is not a speech rate.

backend/app/migrations/0012_turn_speech_seconds.sql

13 lines

```
1 -- How long the learner actually spoke for, per turn.
2 --
3 -- Without this, words-per-minute divided the learner's word count by whole
4 -- session wall-clock time – which includes the AI thinking and talking – so it
5 -- measured the model's latency rather than the learner's speech rate.
6 --
7 -- stt_engine already decodes each clip and computes its duration (for the
8 -- silent-clip guard), so this costs nothing extra to collect. NULL on text
9 -- turns and on every turn recorded before this migration, which the report
10 -- reads as "no speech rate measurable" rather than as zero.
11
12 ALTER TABLE conversation_turns ADD COLUMN speech_seconds REAL;
```

API contract

3. backend/app/models/conversation.py

Pydantic shapes for every request and response. ConversationReportOut carries defaults on every field so an older stored report still deserialises instead of raising a 500, and report_version lets the client offer to regenerate it.

backend/app/models/conversation.py

118 lines

```
1 from typing import Literal
2
3 from pydantic import BaseModel
4
5 ScenarioKey = Literal["free", "coffee", "job", "debate"]
6 Channel = Literal["voice", "text"]
7 Speaker = Literal["user", "ai"]
8 UsageOutcome = Literal["spontaneous", "prompted", "incorrect", "avoided"]
9
10
11 class ConversationSessionCreate(BaseModel):
12     user_id: str
13     scenario: ScenarioKey
14     channel: Channel = "text"
15     # "Practise this again": reuse a previous session's target words instead of
16     # selecting a fresh set. Omitted for a normal new conversation.
17     seed_word_ids: list[str] | None = None
18
19
20 class ConversationTurnOut(BaseModel):
21     id: str
22     turn_index: int
23     speaker: Speaker
24     text: str
25     audio_url: str | None
26     # How many sentence-sized audio pieces this turn can produce. Each is
27     # fetched (and synthesized) separately so the first can start playing
28     # while the rest are still being made.
29     audio_chunk_count: int = 0
30     # The exact text of each of those pieces, in order. The client needs this
31     # to highlight words in time with the voice: it has to know which words
32     # belong to the clip currently playing, and the split is decided here (by
33     # the selected TTS engine) rather than by any rule the client could
```

```

34     # reproduce. Empty for turns that are never spoken aloud.
35     audio_chunks: list[str] = []
36     stt_confidence: float | None
37     created_at: str
38
39
40     class TargetWordOut(BaseModel):
41         id: str
42         word: str
43         used_outcome: UsageOutcome | None # None until end_session classifies it
44
45
46     class ConversationSessionOut(BaseModel):
47         id: str
48         user_id: str
49         scenario: ScenarioKey
50         channel: Channel
51         target_words: list[TargetWordOut]
52         started_at: str
53         ended_at: str | None
54         has_report: bool
55         # The engine this session is pinned to, which is not necessarily the one
56         # Settings currently points at - 'local' | 'openrouter' | 'gemini'.
57         engine_provider: str
58         engine_label: str
59
60
61     class ConversationSessionDetailOut(ConversationSessionOut):
62         turns: list[ConversationTurnOut]
63
64
65     class TurnSubmitOut(BaseModel):
66         user_turn: ConversationTurnOut
67         ai_turn: ConversationTurnOut
68
69
70     class ReportErrorOut(BaseModel):
71         bad: str
72         good: str
73         why: str
74
75
76     class ReportRoutingRowOut(BaseModel):
77         word: str
78         outcome: UsageOutcome
79         evidence_turn: int | None # first turn_index the word appears in, if used
80
81
82     class ConversationReportOut(BaseModel):
83         """Every field added after v1 carries a default, so a report written by an
84         older version still opens instead of 500-ing on validation. `None` means
85         "not measured", which is deliberately distinct from a measured zero - the
86         v1 schema could not express that difference, which is how four permanently
87         -zero fields went unnoticed for so long."""
88
89         session_id: str
90         # 1 = the original shape. Lets the UI say "not measured in this report"
91         # rather than drawing a zero dial for something never computed.
92         report_version: int = 1
93         summary: str
94         turn_count: int
95         routing: list[ReportRoutingRowOut] = []
96         errors: list[ReportErrorOut] = []
97
98         # Dials, all 0-100. None where the session gave nothing to measure.
99         contextual_accuracy_pct: int | None = None # None when the session had no target words
100        grammatical_precision: int | None = None
101        lexical_range: int | None = None # type-token ratio as a percentage
102        pronunciation_score: int | None = None # stt-confidence proxy; None for text sessions
103
104        # Fluency proxies.
105        words_per_minute: int | None = None # None without measured speech time (text sessions, pre-v2 turns)
106        filler_rate_per_100w: float | None = None
107        avg_response_delay_seconds: float | None = None # how long before the learner started answering
108        longest_run_words: int | None = None
109        type_token_ratio: float | None = None

```

```

110     above_level_words: list[str] = []
111     self_corrections: int | None = None
112
113
114 class EngineStatusOut(BaseModel):
115     llm: str
116     stt: str
117     tts: str

```

The service — where the feature lives

4. backend/app/services/conversation.py

The core. Engine readiness and the explicit launch gate, provider dispatch, target-word selection, the chat-history normaliser, session lifecycle, turn submission, TTS chunking and the end-of-session report. Read `submit_user_turn()` and `end_session()` first — everything else supports those two.

backend/app/services/conversation.py

824 lines

```

1  """Session/turn orchestration for the Conversation feature: real target-word
2  selection, real local LLM chat generation, real STT/TTS on the voice
3  channel, and real post-chat analytics + review_logs writes.
4
5  Target-word selection is an honest substitute for real SRS "due" scheduling
6  (which doesn't exist anywhere in this app yet, see 0006_vocabulary.sql):
7  words with no conversation usage yet are preferred, tie-broken by most
8  recently saved.
9  """
10
11 import json
12 import os
13 import sqlite3
14 from pathlib import Path
15
16 from pydantic import ValidationError
17
18 from app.config import settings
19 from app.services import conversation_report, review
20 from app.services.voice import (
21     cloud_llm_engine,
22     engine_health,
23     engine_status,
24     gemini_llm_engine,
25     llm_chat_engine,
26     model_catalog,
27     model_manager,
28     stt_engine,
29     tts,
30 )
31 from app.services.voice.errors import EngineUnavailable
32 from app.utils.ids import uuid7
33 from app.utils.time import iso8601_utc_now
34
35 TARGET_WORD_COUNT = 8
36
37 SCENARIOS: dict[str, dict[str, str]] = {
38     "free": {
39         "label": "Free talk",
40         "brief": "An open, casual conversation about whatever comes up – the learner's week, opinions, small
updates.",
41     },
42     "coffee": {
43         "label": "Order coffee",
44         "brief": "You are a barista at a coffee shop. Stay in character; take the learner's order, ask natural
follow-ups (size, milk, name for the cup).",
45     },
46     "job": {

```

```

47         "label": "Job interview",
48         "brief": "You are interviewing the learner for a job they're applying to. Ask realistic interview questions
and react to their answers.",
49     },
50     "debate": {
51         "label": "Debate a topic",
52         "brief": "Pick a everyday debatable topic and take a clear side, pushing back on the learner's points
respectfully.",
53     },
54 }
55
56
57 def _audio_dir() -> Path:
58     d = Path(settings.db_path).resolve().parent / "conversation_audio"
59     d.mkdir(parents=True, exist_ok=True)
60     return d
61
62
63 def selected_llm_option(conn: sqlite3.Connection, user_id: str) -> model_catalog.LlmOption:
64     row = conn.execute("SELECT llm_model_id FROM user_settings WHERE user_id = ?", (user_id,)).fetchone()
65     explicit_key = row["llm_model_id"] if row else None
66     if explicit_key:
67         return model_catalog.llm_option(explicit_key)
68
69     # No explicit choice made yet in Settings – prefer whichever option is
70     # actually downloaded rather than blindly falling back to the catalog's
71     # fixed recommended default, which may not be the one the user has. This
72     # keeps "download a model" alone sufficient to unblock Conversation, with
73     # no separate "now go select it" step required.
74     for option in model_catalog.LLM_OPTIONS:
75         if model_catalog.llm_is_downloaded(option):
76             return option
77     return model_catalog.llm_option(None)
78
79
80 def llm_target(conn: sqlite3.Connection, user_id: str) -> dict:
81     """Resolves what a real LLM call should actually hit right now: either a
82     local catalog option (llama.cpp, loaded via Launch AI) or a cloud model
83     over OpenRouter or Gemini (no local load step – a live API call every
84     time either way).
85
86     Shape: {"provider": "local", "option": LlmOption} or
87           {"provider": "openrouter" | "gemini", "model": str, "api_key": str | None}.
88     `llm_model_id` is intentionally NOT reused for a cloud model string – it
89     stays local-catalog-key-only, so a cloud model id can never be misread as
90     a local one (see 0010_openrouter_llm.sql). `api_provider` picks which
91     cloud service 'api' mode actually means (see 0011_gemini_llm.sql)."""
92     row = conn.execute(
93         "SELECT llm_mode, api_provider, openrouter_api_key, openrouter_model, gemini_api_key, gemini_model "
94         "FROM user_settings WHERE user_id = ?",
95         (user_id,),
96     ).fetchone()
97     if row is not None and row["llm_mode"] == "api":
98         if row["api_provider"] == "gemini":
99             return {
100                 "provider": "gemini",
101                 "model": row["gemini_model"] or gemini_llm_engine.DEFAULT_MODEL,
102                 "api_key": row["gemini_api_key"],
103             }
104         return {
105             "provider": "openrouter",
106             "model": row["openrouter_model"] or cloud_llm_engine.DEFAULT_MODEL,
107             "api_key": row["openrouter_api_key"],
108         }
109     return {"provider": "local", "option": selected_llm_option(conn, user_id)}
110
111
112 def llm_target_label(target: dict) -> str:
113     if target["provider"] == "openrouter":
114         return f"{target['model']} (OpenRouter)"
115     if target["provider"] == "gemini":
116         return f"{target['model']} (Gemini)"
117     return target["option"].label
118
119
120 def _session_llm_target(conn: sqlite3.Connection, session: sqlite3.Row) -> dict:

```

```

121     """A conversation follows whatever engine Settings currently points at, so
122     switching model or provider takes effect on the chat already open instead
123     of only on the next one.
124
125     Sessions used to be pinned to the engine they started on, for consistent
126     voice across a transcript. In practice that traded a cosmetic consistency
127     for a real dead end: when the pinned engine became unusable (quota spent,
128     key revoked, model deleted) the conversation could never continue, and no
129     Settings change could rescue it. 'model_id' now records what most recently
130     answered rather than dictating what must answer next."""
131     return llm_target(conn, session["user_id"])
132
133
134 def session_engine(conn: sqlite3.Connection, session: sqlite3.Row) -> dict:
135     """The engine this session's next turn will really call, named for the UI
136     so it can show the truth instead of inferring it from global state (which
137     is how it ended up claiming one engine while the turns called another)."""
138     target = _session_llm_target(conn, session)
139     return {"provider": target["provider"], "label": llm_target_label(target)}
140
141
142 def _session_model_id(target: dict) -> str:
143     if target["provider"] == "gemini":
144         return f"cloud:gemini:{target['model']}"
145     if target["provider"] == "openrouter":
146         return f"cloud:{target['model']}"
147     return target["option"].key
148
149
150 def _cloud_engine(provider: str):
151     return gemini_llm_engine if provider == "gemini" else cloud_llm_engine
152
153
154 def _generate_reply(target: dict, system_prompt: str, history: list[tuple[str, str]], max_tokens: int = 120) -> str:
155     if target["provider"] != "local":
156         return _cloud_engine(target["provider"]).generate_reply(
157             system_prompt, history, api_key=target["api_key"], model=target["model"], max_tokens=max_tokens
158         )
159     option = target["option"]
160     return llm_chat_engine.generate_reply(
161         system_prompt, history, repo_id=option.repo_id, filename=option.filename, max_tokens=max_tokens
162     )
163
164
165 def _generate_analysis(
166     target: dict, transcript: list[tuple[str, str]], target_words: list[str]
167 ) -> conversation_report.LlmReportAnalysis:
168     """The judged half of the report. Prompt and parsing both come from
169     conversation_report, so the keys asked for are by construction the keys
170     read back – see that module's docstring for what went wrong when they
171     were maintained separately."""
172     system_prompt = conversation_report.report_system_prompt(target_words)
173     user_prompt = conversation_report.report_user_prompt(transcript)
174
175     if target["provider"] != "local":
176         raw = _cloud_engine(target["provider"]).generate_json(
177             system_prompt, user_prompt, api_key=target["api_key"], model=target["model"], max_tokens=700
178         )
179     else:
180         option = target["option"]
181         raw = llm_chat_engine.generate_json(
182             system_prompt, user_prompt, repo_id=option.repo_id, filename=option.filename, max_tokens=700
183         )
184     # A small model will sometimes omit a key or mistype a value; a partial
185     # analysis is still worth showing, so tolerate that rather than failing the
186     # whole report. What it cannot do is invent a field nobody asked for.
187     try:
188         return conversation_report.LlmReportAnalysis.model_validate(raw)
189     except ValidationError:
190         return conversation_report.LlmReportAnalysis()
191
192
193 def _readiness_for_target(target: dict, channel: str, tts_name: str = tts.DEFAULT_ENGINE) -> dict:
194     if target["provider"] == "local":
195         llm_ready = model_catalog.llm_is_downloaded(target["option"])
196     else:

```

```

197     llm_ready = bool(target["api_key"])
198 if channel == "voice":
199     stt_ready = model_catalog.stt_is_downloaded()
200     # Only the *selected* voice engine has to be downloaded. Having
201     # Kokoro on disk says nothing about whether Pocket TTS is, and a
202     # session that would fail on the engine it will actually use should
203     # say so up front rather than at the first attempt to speak.
204     tts_ready = tts.is_downloaded(tts_name)
205 else:
206     stt_ready = tts_ready = True
207 return {
208     "ready": llm_ready and stt_ready and tts_ready,
209     "llm": llm_ready,
210     "stt": stt_ready,
211     "tts": tts_ready,
212     "llm_model_label": llm_target_label(target),
213 }
214
215
216 def readiness(conn: sqlite3.Connection, user_id: str, channel: str) -> dict:
217     """What a session needs before it can actually run – checked up front so
218     Conversation can refuse to start with a clear message instead of the old
219     behavior of silently kicking off a multi-minute implicit download (or
220     now, failing deep inside an HTTP request). For the cloud provider,
221     "ready" means a real API key is configured – there's no download step.
222     Reflects the user's *current* Settings preference – see
223     _ensure_ready_for_target for the session-pinned equivalent."""
224     return _readiness_for_target(llm_target(conn, user_id), channel, tts.selected_name(conn, user_id))
225
226
227 def _ensure_ready_for_target(target: dict, channel: str, tts_name: str = tts.DEFAULT_ENGINE) -> None:
228     r = _readiness_for_target(target, channel, tts_name)
229     if r["ready"]:
230         return
231     missing = [name for name, ok in (("LLM", r["llm"]), ("STT", r["stt"]), ("TTS", r["tts"])) if not ok]
232     raise EngineUnavailable(
233         f'{"/".join(missing)} model{s} if len(missing) > 1 else {} not downloaded yet – "
234         "download it in Settings before starting a conversation."
235     )
236
237
238 def _ensure_ready(conn: sqlite3.Connection, user_id: str, channel: str) -> None:
239     """Checks the user's *current* Settings preference – right for
240     start_session (no session exists yet). submit_user_turn/end_session use
241     _ensure_ready_for_target with the session's own pinned target instead,
242     so switching Settings mid-conversation (e.g. back to local with nothing
243     downloaded) can't block turns in a session that's still perfectly able
244     to run on what it was actually started with."""
245     _ensure_ready_for_target(llm_target(conn, user_id), channel, tts.selected_name(conn, user_id))
246
247
248 def full_engine_status(conn: sqlite3.Connection, user_id: str) -> dict:
249     """Like engine_status.status(), but provider-aware: the cloud LLM has no
250     local "loaded into memory" concept, so it reads as ready the instant an
251     API key is configured, rather than through the local model-residency
252     check (which would otherwise always show not_loaded for it)."""
253     target = llm_target(conn, user_id)
254     if target["provider"] != "local":
255         st = engine_status.status(None)
256         # A saved key is not a working key. "ready" here means a real request
257         # has actually succeeded – established by Launch AI, and revoked again
258         # the moment one genuinely fails. Read from a record rather than probed
259         # here, so polling this endpoint never spends quota.
260         st["llm"] = (
261             "ready"
262             if target["api_key"] and engine_health.is_verified(target["provider"], target["model"])
263             else "not_loaded"
264         )
265     return st
266     option = target["option"]
267     return engine_status.status(str(model_manager.llm_model_path(option.repo_id, option.filename)))
268
269
270 def _ensure_launched(needs: set[str], *, llm_target: dict, tts_name: str | None = None) -> None:
271     """Distinct from _ensure_ready: that checks a model is *downloaded*/
272     *configured*, this checks it's actually *loaded into memory* – a check

```

```

273 that only applies to the local provider, since a cloud API call has no
274 load step at all (once _ensure_ready confirmed a key exists, it's
275 already "launched"). For local, this is pinned to whichever model
276 'llm_target' says is actually selected right now – without that,
277 switching to a different (downloaded) model in Settings would still
278 read as "ready" off whatever was previously loaded, and the next turn
279 would silently pay a reload mid-conversation instead of the learner
280 choosing when via Launch AI.
281 effective_needs = set(needs)
282 llm_path = None
283 if llm_target["provider"] != "local":
284     effective_needs.discard("llm")
285 else:
286     option = llm_target["option"]
287     llm_path = str(model_manager.llm_model_path(option.repo_id, option.filename))
288 st = engine_status.status(llm_path, tts_name)
289 missing = sorted(n.upper() for n in effective_needs if st.get(n) != "ready")
290 if missing:
291     raise EngineUnavailable(
292         f"AI isn't launched yet ({'/'}.join(missing)} not loaded) – "
293         "use the Launch AI button at the top of the app, then try again."
294     )
295
296
297 def _select_target_words(conn: sqlite3.Connection, user_id: str) -> list[sqlite3.Row]:
298     """An 'avoided' log means the word was offered as a target but the learner
299     never actually said it – counting that as practice would push exactly the
300     words that still need work to the back of the queue, so only outcomes
301     where the word was really produced count as usage here."""
302     return conn.execute(
303         """
304         SELECT vw.id, vw.word, vw.definition,
305             (SELECT COUNT(*) FROM review_logs rl
306              WHERE rl.vocab_word_id = vw.id AND rl.outcome <> 'avoided') AS usage_count
307         FROM vocab_words vw
308         WHERE vw.user_id = ?
309         ORDER BY usage_count ASC, vw.created_at DESC
310         LIMIT ?
311         """,
312         (user_id, TARGET_WORD_COUNT),
313     ).fetchall()
314
315
316 def _words_by_ids(conn: sqlite3.Connection, user_id: str, word_ids: list[str]) -> list[sqlite3.Row]:
317     """Scoped to the owner, so a seeded id can't pull in someone else's word.
318     Silently drops ids that no longer exist – a word deleted since the original
319     session shouldn't block repeating it."""
320     if not word_ids:
321         return []
322     placeholders = ",".join("?" for _ in word_ids)
323     return conn.execute(
324         f"SELECT id, word, definition FROM vocab_words WHERE user_id = ? AND id IN ({placeholders})",
325         [user_id, *word_ids],
326     ).fetchall()
327
328
329 def normalise_for_chat(pairs: list[tuple[str, str]]) -> tuple[list[str], list[tuple[str, str]]]:
330     """Reshape a transcript into something every provider will accept.
331
332     Returns (assistant_opening_lines, history), where history begins with a
333     'user' message and strictly alternates user/assistant.
334
335     Two real failures make this necessary, both observed in production:
336
337     1. **A session opens with the AI speaking.** Gemma's chat template rejects
338     that outright – measured against gemma-3-1b, ``system + assistant +
339     user`` raises "Conversation roles must alternate
340     user/assistant/user/assistant/...", while ``system + user + assistant +
341     user`` is fine. So every reply after the AI greeting failed on any
342     Gemma model. Leading assistant turns are lifted out here and handed to
343     the caller to fold into the system prompt, which keeps their content
344     without breaking the alternation rule. Gemini requires the same shape.
345
346     2. **Consecutive turns from one speaker.** A failed generation used to
347     leave the learner's turn committed with no reply, so the next attempt
348     saw two user turns in a row and hit the same template error – the

```

```

349         error then masked whatever had actually gone wrong. Orphans are no
350         longer created (see submit_user_turn), but sessions already in that
351         state still have to work, so same-role runs are merged into one
352         message rather than dropped.
353     """
354     merged: list[tuple[str, str]] = []
355     for role, text in pairs:
356         clean = (text or "").strip()
357         if not clean:
358             continue
359         if merged and merged[-1][0] == role:
360             merged[-1] = (role, f"{merged[-1][1]} {clean}")
361         else:
362             merged.append((role, clean))
363
364     opening: list[str] = []
365     while merged and merged[0][0] == "assistant":
366         opening.append(merged.pop(0)[1])
367     return opening, merged
368
369 def _system_prompt(
370     scenario: str, target_words: list[sqlite3.Row], opening: list[str] | None = None
371 ) -> str:
372     scenario_cfg = SCENARIOS[scenario]
373     words_block = (
374         "\n".join(f"- {w['word']}: {w['definition']} or 'no definition on file'" for w in target_words)
375         or "(the learner has no saved vocabulary yet – just have a natural conversation)"
376     )
377     return (
378         f"You are Juno, a friendly AI conversation partner helping someone practice English. "
379         f"Scenario: {scenario_cfg['brief']}\n\n"
380         f"The learner is trying to use these target words naturally in this conversation:\n{words_block}\n\n"
381         "Rules: never say a target word yourself before the learner does. Keep replies to ONE or TWO "
382         "short sentences – real spoken conversation, not an essay. Speaking them aloud takes longer "
383         "than generating them, so length is what the learner waits on. Ask a follow-up question to "
384         "keep the conversation going."
385         # Whatever the AI already said cannot stay in the message list without
386         # breaking alternation (see normalise_for_chat), so it is given back
387         # here instead – the model still knows how it opened.
388         + (f"\n\nYou have already said this to the learner: {' '.join(opening)}" if opening else "")
389     )
390
391
392 def launch_engines(conn: sqlite3.Connection, user_id: str) -> dict:
393     """The "Launch AI" action: explicitly loads whatever's downloaded into
394     memory right now, instead of leaving that to happen (or be refused, per
395     _ensure_launched) at first real use. Cloud providers have nothing to load
396     for the LLM itself – just need a configured key – but voice's local
397     STT/TTS still get warmed up either way."""
398     target = llm_target(conn, user_id)
399     if target["provider"] == "local":
400         if not model_catalog.llm_is_downloaded(target["option"]):
401             raise EngineUnavailable("No AI model downloaded yet – download one in Settings before launching AI.")
402         llm_chat_engine.warm_up(target["option"].repo_id, target["option"].filename)
403     elif not target["api_key"]:
404         label = "Gemini" if target["provider"] == "gemini" else "OpenRouter"
405         raise EngineUnavailable(f"No {label} API key configured – add one in Settings before launching AI.")
406     else:
407         # The cloud equivalent of loading a model: spend one small request to
408         # establish that the key really works, instead of assuming it from the
409         # fact that something is saved.
410         _cloud_engine(target["provider"]).verify(api_key=target["api_key"], model=target["model"])
411     if model_catalog.stt_is_downloaded():
412         stt_engine.warm_up()
413     tts_name = tts.selected_name(conn, user_id)
414     if tts.is_downloaded(tts_name):
415         # Whichever engine this user has picked – warming the other one would
416         # hold a few hundred MB for a voice that is never going to be called.
417         tts.engine_for(tts_name).warm_up()
418     return full_engine_status(conn, user_id)
419
420
421 def unload_engines(conn: sqlite3.Connection, user_id: str) -> dict:
422     """The opposite of launch_engines: hands the memory back.
423
424

```

```

425     Each engine's own unload() takes the same lock generation holds, so this
426     waits for any in-flight turn rather than pulling a model out from under
427     it. A cloud LLM has nothing resident to free – its key stays verified, so
428     the conversation can carry on while the local speech models are released.""
429     target = llm_target(conn, user_id)
430     if target["provider"] == "local":
431         llm_chat_engine.unload()
432     stt_engine.unload()
433     tts.unload_all()
434     return full_engine_status(conn, user_id)
435
436
437 def start_session(
438     conn: sqlite3.Connection,
439     *,
440     user_id: str,
441     scenario: str,
442     channel: str,
443     seed_word_ids: list[str] | None = None,
444 ) -> str:
445     """`seed_word_ids` backs "practise this again": rather than picking a fresh
446     set, the new session reuses the words from the one being repeated, so the
447     words that just went badly are the ones that come straight back."""
448     _ensure_ready(conn, user_id, channel)
449     target = llm_target(conn, user_id)
450     _ensure_launched(
451         {"llm"} | ({"tts"} if channel == "voice" else set()),
452         llm_target=target,
453         tts_name=tts.selected_name(conn, user_id),
454     )
455
456     target_words = _words_by_ids(conn, user_id, seed_word_ids) if seed_word_ids else []
457     if not target_words:
458         target_words = _select_target_words(conn, user_id)
459     # Generate the opening BEFORE writing anything, for two reasons. It keeps
460     # a failure from leaving a session row with no turns behind – one that
461     # would then sit in the learner's history forever as an empty
462     # conversation, and which "resume" could only ever reopen empty. And it
463     # means no write transaction is held open across a call that can take
464     # minutes on a cold model load, which would block every other writer.
465     opening = _generate_reply(
466         target,
467         _system_prompt(scenario, target_words),
468         [("user", "(The learner has just joined. Greet them and open the conversation.)")],
469     )
470
471     session_id = uuid7()
472     conn.execute(
473         """
474         INSERT INTO conversation_sessions (id, user_id, scenario, channel, target_word_ids, model_id, started_at)
475         VALUES (?, ?, ?, ?, ?, ?, ?)
476         """,
477         (
478             session_id,
479             user_id,
480             scenario,
481             channel,
482             json.dumps([w["id"] for w in target_words]),
483             _session_model_id(target),
484             iso8601_utc_now(),
485         ),
486     )
487     conn.commit()
488     _insert_turn(conn, session_id, 0, "ai", opening, channel)
489     return session_id
490
491
492 def _insert_turn(
493     conn: sqlite3.Connection,
494     session_id: str,
495     turn_index: int,
496     speaker: str,
497     text: str,
498     channel: str,
499     stt_confidence: float | None = None,
500     speech_seconds: float | None = None,

```

```

501 ) -> sqlite3.Row:
502     turn_id = uuid7()
503     # Deliberately no synthesis here. Kokoro runs slower than real time on this
504     # hardware, so making the whole reply's audio before returning meant the
505     # learner waited ~30s to see any reply at all. Audio is produced per
506     # sentence, on demand, once the text is already on screen.
507     audio_path = None
508
509     conn.execute(
510         """
511         INSERT INTO conversation_turns
512         (id, session_id, turn_index, speaker, text, audio_path, stt_confidence, speech_seconds, created_at)
513         VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
514         """,
515         (
516             turn_id,
517             session_id,
518             turn_index,
519             speaker,
520             text,
521             audio_path,
522             stt_confidence,
523             speech_seconds,
524             iso8601_utc_now(),
525         ),
526     )
527     conn.commit()
528     return conn.execute("SELECT * FROM conversation_turns WHERE id = ?", (turn_id,)).fetchone()
529
530
531 def audio_chunk_path(turn_id: str, index: int, engine: str = tts.DEFAULT_ENGINE) -> Path:
532     """The engine is part of the filename because the audio is that engine's
533     voice. Without it, switching engines would keep serving whatever the
534     previous one had already cached for a turn, so the change would appear to
535     do nothing on every reply already on screen. It also leaves the two free
536     to split replies differently without one's chunk 0 being served as the
537     other's."""
538     return _audio_dir() / f"{turn_id}.{engine}.{index}.wav"
539
540
541 def ensure_audio_chunk(turn_id: str, text: str, index: int, engine: str = tts.DEFAULT_ENGINE) -> Path:
542     """Synthesizes one sentence of a reply, cached on disk after the first
543     request. Called when that sentence is about to be played rather than while
544     the learner is still waiting to read the reply."""
545     name = tts.normalise(engine)
546     chunks = tts.engine_for(name).split_for_streaming(text)
547     if index < 0 or index >= len(chunks):
548         raise IndexError(f"chunk {index} out of range for {len(chunks)} chunks")
549     path = audio_chunk_path(turn_id, index, name)
550     if not path.exists():
551         audio = tts.engine_for(name).synthesize(chunks[index])
552         # Written to a private temp file and moved into place, never straight
553         # to 'path'. The player fetches chunk N+1 while N is still audible, so
554         # two requests for the same chunk really can overlap (a replay, a
555         # retry, a second window) – and a FileResponse reading a path that
556         # another thread is still writing serves a truncated WAV, which plays
557         # as a click or as silence. os.replace is atomic, so a reader sees
558         # either the whole file or no file.
559         tmp = path.with_suffix(f".{uuid7()}.part")
560         try:
561             tmp.write_bytes(audio)
562             os.replace(tmp, path)
563         finally:
564             tmp.unlink(missing_ok=True)
565     return path
566
567
568 def audio_chunk_texts(turn: sqlite3.Row, channel: str, engine: str = tts.DEFAULT_ENGINE) -> list[str]:
569     """The text of each audio piece this turn produces, in order. Only the AI
570     speaks aloud, and only on the voice channel.
571
572     The client highlights words in time with the voice, which means it needs
573     the same split the synthesizer used – not a guess at it. Returning the
574     text rather than only a count is what makes the two agree."""
575     if channel != "voice" or turn["speaker"] != "ai":
576         return []

```

```

577     return tts.engine_for(engine).split_for_streaming(turn["text"])
578
579
580 def audio_chunk_count(turn: sqlite3.Row, channel: str, engine: str = tts.DEFAULT_ENGINE) -> int:
581     """How many audio pieces this turn can produce."""
582     return len(audio_chunk_texts(turn, channel, engine))
583
584
585 def get_session_row(conn: sqlite3.Connection, session_id: str, user_id: str) -> sqlite3.Row | None:
586     return conn.execute(
587         "SELECT * FROM conversation_sessions WHERE id = ? AND user_id = ?", (session_id, user_id)
588     ).fetchone()
589
590
591 def get_turns(conn: sqlite3.Connection, session_id: str) -> list[sqlite3.Row]:
592     return conn.execute(
593         "SELECT * FROM conversation_turns WHERE session_id = ? ORDER BY turn_index", (session_id,)
594     ).fetchall()
595
596
597 def submit_user_turn(
598     conn: sqlite3.Connection,
599     session: sqlite3.Row,
600     *,
601     text: str | None,
602     audio_bytes: bytes | None,
603 ) -> tuple[sqlite3.Row, sqlite3.Row]:
604     channel = session["channel"]
605     target = _session_llm_target(conn, session)
606     _ensure_ready_for_target(target, channel, tts.selected_name(conn, session["user_id"]))
607     # Record what is actually answering now, so the session's engine label
608     # reflects reality after a mid-conversation switch.
609     model_id = _session_model_id(target)
610     if session["model_id"] != model_id:
611         conn.execute("UPDATE conversation_sessions SET model_id = ? WHERE id = ?", (model_id, session["id"]))
612         conn.commit()
613     _ensure_launched(
614         {"llm"} | ({"stt"} if audio_bytes is not None else set()) | ({"tts"} if channel == "voice" else set()),
615         llm_target=target,
616         tts_name=tts.selected_name(conn, session["user_id"]),
617     )
618     stt_confidence = None
619     speech_seconds = None
620     if audio_bytes is not None:
621         text, stt_confidence, speech_seconds = stt_engine.transcribe(audio_bytes)
622
623     text = (text or "").strip()
624     if not text:
625         # Audio was provided (the router already rejects "neither text nor
626         # audio") but STT came back empty – silence, too short a clip, or
627         # unclear speech. Sending an empty turn to the LLM isn't meaningful
628         # for any provider, and Gemini specifically rejects it outright
629         # (its "ends with a model turn" error is really this in disguise:
630         # an empty user part gets dropped from validation, leaving the
631         # prior AI turn looking like the last one). Fail honestly instead.
632         raise ValueError("Didn't catch any speech there – try speaking again, a bit louder or closer to the mic.")
633     turns = get_turns(conn, session["id"])
634     next_index = turns[-1]["turn_index"] + 1 if turns else 0
635
636     target_words = conn.execute(
637         f"SELECT id, word, definition FROM vocab_words WHERE id IN "
638         f"({','.join('?' for _ in json.loads(session['target_word_ids']))})",
639         json.loads(session["target_word_ids"]),
640     ).fetchall() if json.loads(session["target_word_ids"]) else []
641
642     pairs = [("assistant" if t["speaker"] == "ai" else "user", t["text"]) for t in turns]
643     pairs.append(("user", text))
644     opening, history = normalise_for_chat(pairs)
645     system_prompt = _system_prompt(session["scenario"], target_words, opening)
646
647     # Generate BEFORE writing anything. A failure here used to leave the
648     # learner's turn committed with no reply, and the next attempt then saw
649     # two user turns in a row – which Gemma's template rejects, so the session
650     # was permanently stuck behind an alternation error that said nothing
651     # about the original failure. Nothing is persisted unless there is a
652     # reply to persist alongside it.

```

```

653     reply = _generate_reply(target, system_prompt, history)
654
655     user_turn = _insert_turn(
656         conn, session["id"], next_index, "user", text, channel, stt_confidence, speech_seconds
657     )
658     ai_turn = _insert_turn(conn, session["id"], next_index + 1, "ai", reply, channel)
659
660     return user_turn, ai_turn
661
662
663 def end_session(conn: sqlite3.Connection, session: sqlite3.Row) -> dict:
664     target = _session_llm_target(conn, session)
665     _ensure_launched({"llm"}, llm_target=target)
666     turns = get_turns(conn, session["id"])
667     target_word_ids = json.loads(session["target_word_ids"])
668     target_words = (
669         conn.execute(
670             f"SELECT id, word FROM vocab_words WHERE id IN ({','.join('?' for _ in target_word_ids)}",
671             target_word_ids,
672         ).fetchall()
673     ) if target_word_ids
674     else []
675 )
676
677 transcript = [(t["speaker"], t["text"]) for t in turns]
678 analysis = _generate_analysis(target, transcript, [w["word"] for w in target_words])
679
680 cefr_row = conn.execute("SELECT cefr_level FROM users WHERE id = ?", (session["user_id"],)).fetchone()
681 target_cefr = conversation_report.resolve_cefr(cefr_row["cefr_level"] if cefr_row else None)
682 metrics = conversation_report.compute_metrics(turns, target_cefr)
683
684 user_turns = [t for t in turns if t["speaker"] == "user"]
685 ai_turns = [t for t in turns if t["speaker"] == "ai"]
686 routing = []
687 outcomes_for_logs: list[tuple[str, str]] = []
688 for w in target_words:
689     # Whether the learner said the word at all is a fact about the
690     # transcript, not a judgement – so it is established here, not asked.
691     evidence_turn = next(
692         (t["turn_index"] for t in user_turns if conversation_report.says_word(t["text"], w["word"])),
693         None,
694     )
695     outcome = conversation_report.match_word_usage(analysis.word_usage, w["word"]) or "avoided"
696
697     if evidence_turn is not None and outcome == "avoided":
698         # "avoided" means "never used at all", and the transcript shows
699         # otherwise – a claim the evidence disproves, so it does not
700         # stand. Observed with gemma-3-1b marking every word avoided in a
701         # session where the learner had plainly used all three; the report
702         # then read 0/3 and the words were logged as unpractised, pushing
703         # them back to the front of the queue they had just earned their
704         # way out of.
705         #
706         # Only this one outcome is corrected. "incorrect" is a judgement
707         # about *how* the word was used and nothing here can second-guess
708         # it; "avoided" is the only verdict that is checkable.
709         said_first = any(
710             t["turn_index"] < evidence_turn and conversation_report.says_word(t["text"], w["word"])
711             for t in ai_turns
712         )
713         # The spec's own distinction: prompted means used only after the
714         # AI said it. That much is visible in the transcript.
715         outcome = "prompted" if said_first else "spontaneous"
716
717     routing.append({"word": w["word"], "outcome": outcome, "evidence_turn": evidence_turn})
718     outcomes_for_logs.append((w["id"], outcome))
719
720 correct_count = sum(1 for r in routing if r["outcome"] in ("spontaneous", "prompted"))
721 # None, not 0: a session with no target words didn't score zero accuracy,
722 # it had nothing to be accurate about.
723 contextual_accuracy_pct = round(100 * correct_count / len(routing)) if routing else None
724
725 report = {
726     "report_version": conversation_report.REPORT_VERSION,
727     "session_id": session["id"],
728     "contextual_accuracy_pct": contextual_accuracy_pct,

```

```

729     "grammatical_precision": analysis.grammatical_precision,
730     # Type-token ratio is 0-1; the dial needs 0-100.
731     "lexical_range": round(100 * metrics.type_token_ratio),
732     "pronunciation_score": metrics.pronunciation_score,
733     "words_per_minute": metrics.words_per_minute,
734     "filler_rate_per_100w": metrics.filler_rate_per_100w,
735     "avg_response_delay_seconds": metrics.avg_response_delay_seconds,
736     "longest_run_words": metrics.longest_run_words,
737     "type_token_ratio": metrics.type_token_ratio,
738     "above_level_words": metrics.above_level_words,
739     "self_corrections": analysis.self_corrections,
740     "turn_count": len(turns),
741     "routing": routing,
742     "errors": [e.model_dump() for e in analysis.errors[:3]],
743     "summary": analysis.summary or "No summary was generated.",
744 }
745
746 now = iso8601_utc_now()
747 conn.execute(
748     "UPDATE conversation_sessions SET report_json = ?, ended_at = ? WHERE id = ?",
749     (json.dumps(report), now, session["id"]),
750 )
751 # Ending recomputes over the WHOLE transcript so far, so a re-end
752 # supersedes this session's previous logs rather than adding a second set
753 # - appending would make one conversation count twice, both in a word's
754 # usage history and in _select_target_words' "not practised yet" ordering.
755 conn.execute("DELETE FROM review_logs WHERE session_id = ?", (session["id"],))
756 # Spec §6.3: this is what makes conversation count as study. Each outcome
757 # is rated on the same scale a flashcard would be and applied to the same
758 # FSRS card, so producing a word out loud reschedules it exactly as
759 # answering a flashcard would - the project's actual thesis, rather than
760 # a log nobody acts on.
761 review.apply_conversation_outcomes(
762     conn, session["user_id"], session["id"], outcomes_for_logs
763 )
764
765 return report
766
767
768 def list_sessions(conn: sqlite3.Connection, user_id: str) -> list[sqlite3.Row]:
769     return conn.execute(
770         "SELECT * FROM conversation_sessions WHERE user_id = ? ORDER BY started_at DESC", (user_id,)
771     ).fetchall()
772
773
774 def delete_session(conn: sqlite3.Connection, user_id: str, session_id: str) -> bool:
775     """Deletes a session and its turns (DB cascade handles conversation_turns
776     and nulls out review_logs.session_id - the real usage history those rows
777     represent outlives the session record itself). Audio files aren't
778     covered by any DB cascade, so they're unlinked from disk explicitly
779     first."""
780     row = conn.execute(
781         "SELECT id FROM conversation_sessions WHERE id = ? AND user_id = ?", (session_id, user_id)
782     ).fetchone()
783     if row is None:
784         return False
785
786     for turn in conn.execute("SELECT id, audio_path FROM conversation_turns WHERE session_id = ?", (session_id,)):
787         if turn["audio_path"]:
788             Path(turn["audio_path"]).unlink(missing_ok=True)
789             # Per-sentence audio is written lazily and named off the turn id, so it
790             # isn't reachable through audio_path.
791             for chunk in _audio_dir().glob(f"{turn['id']}.*.wav"):
792                 chunk.unlink(missing_ok=True)
793
794     conn.execute("DELETE FROM conversation_sessions WHERE id = ?", (session_id,))
795     return True
796
797
798 def word_usage_counts(conn: sqlite3.Connection, vocab_word_id: str) -> dict[str, int]:
799     """Per-outcome CONVERSATION usage for one word - the visible half of
800     Dynamic SRS Routing, surfaced on the Vocabulary entry screen.
801
802     The source filter is load-bearing. review_logs carried only conversation
803     rows until 0015 gave flashcard reviews the same table (spec §5.5's
804     dual-source review), and without it this counted both - so the Vocabulary

```

```

805     page reported "conversation usage: again 1x" for a word that had only
806     ever been answered on a flashcard. "again" is not something anyone can do
807     in a conversation.""
808     rows = conn.execute(
809         "SELECT outcome, COUNT(*) AS n FROM review_logs "
810         "WHERE vocab_word_id = ? AND source = 'conversation' GROUP BY outcome",
811         (vocab_word_id,),
812     ).fetchall()
813     return {row["outcome"]: row["n"] for row in rows}
814
815 def flashcard_review_counts(conn: sqlite3.Connection, vocab_word_id: str) -> dict[str, int]:
816     """The other half: how this word has been answered on flashcards."""
817     rows = conn.execute(
818         "SELECT outcome, COUNT(*) AS n FROM review_logs "
819         "WHERE vocab_word_id = ? AND source = 'flashcard' GROUP BY outcome",
820         (vocab_word_id,),
821     ).fetchall()
822     return {row["outcome"]: row["n"] for row in rows}
823

```

5. backend/app/services/conversation_report.py

The post-chat analysis. `LlmReportAnalysis` defines the judged fields; `report_system_prompt()` is rendered from that model's own fields so the prompt and the reader cannot drift apart; `compute_metrics()` is the deterministic half and is a pure function over the turn list.

backend/app/services/conversation_report.py

220 lines

```

1  """Post-chat analysis: what the report measures, and who measures it.
2
3  Every field is either DETERMINISTIC (computed here from the transcript, so it
4  is reproducible, free, and unit-testable) or JUDGED (asked of the LLM, because
5  it genuinely needs language understanding). Keeping that line sharp is what
6  makes the report defensible: a model asked for a "fluency score" will invent
7  one, whereas a model asked whether a learner used a word correctly is doing
8  work only a language model can do.
9
10 This module is also the single source of truth for the judged half. The prompt
11 is rendered FROM 'LlmReportAnalysis', and 'end_session' parses INTO it, so the
12 keys requested and the keys read cannot drift apart. They did drift once: the
13 prompt asked for three keys while the reader expected six, leaving four report
14 fields permanently zero on every real session while the tests passed. See
15 docs/conversation-report-design.md.
16 """
17
18 import re
19 from dataclasses import dataclass
20
21 from pydantic import BaseModel, Field
22
23 from app.services import cefr_lexicon
24
25 # Lexical hesitation markers. Deliberately a short, explicit list of items that
26 # are unambiguously fillers in English rather than a cleverer heuristic – the
27 # number is reported as a rate, and a wrong word in this list quietly skews it.
28 FILLER_WORDS = frozenset({"um", "uh", "erm", "ah", "hmm", "mmm", "like", "basically", "actually"})
29
30 # Used only when a learner has no CEFR level recorded yet (placement not done).
31 DEFAULT_CEFR = "B1"
32
33 # Bumped whenever the report's shape changes. Reports written by an older
34 # version are still readable (every added field has a default) and the UI uses
35 # this to tell "not measured back then" apart from a genuine zero.
36 REPORT_VERSION = 2
37
38 _WORD_RE = re.compile(r"[a-zA-Z']+")
39
40
41 class ReportError(BaseModel):
42     bad: str
43     good: str

```

```

44     why: str
45
46
47 class LlmReportAnalysis(BaseModel):
48     """The judged half of the report – the only things the LLM is asked for.
49
50     Field names here become the JSON keys in the prompt (see
51     `report_system_prompt`), so renaming one changes both halves at once.
52     """
53
54     word_usage: dict[str, str] = Field(
55         default_factory=dict,
56         description='per target word: "spontaneous" | "prompted" | "incorrect" | "avoided"',
57     )
58     grammatical_precision: int = Field(
59         default=0, description="0-100, how grammatically accurate the learner's turns were"
60     )
61     errors: list[ReportError] = Field(
62         default_factory=list,
63         description="the 3 most instructive mistakes, each as "
64         '{ "bad": what they said, "good": the correction, "why": a short reason }',
65     )
66     self_corrections: int = Field(
67         default=0, description="how many times the learner caught and repaired their own mistake"
68     )
69     summary: str = Field(
70         default="", description="2-3 encouraging sentences ending with one concrete focus for next time"
71     )
72
73
74 def report_system_prompt(target_words: list[str]) -> str:
75     """Built from LlmReportAnalysis's own fields, so the model is asked for
76     exactly what the reader parses – never a hand-maintained second copy."""
77     shape = ", ".join(
78         f'"{name}": {field.description}' for name, field in LlmReportAnalysis.model_fields.items()
79     )
80     words_list = ", ".join(target_words) if target_words else "(none)"
81     return (
82         "You are a strict, structured language-learning analyst. Given a conversation transcript and "
83         "the target vocabulary words the learner was meant to practise, respond with ONLY a JSON "
84         "object (no prose, no markdown fences) with exactly these keys: {" + shape + "}. "
85         '"spontaneous" = used correctly and unprompted. "prompted" = used correctly only after the AI '
86         "'said or hinted the word. "incorrect" = attempted but used wrongly. "avoided" = never used at all. '
87         f"Judge only the learner's turns, never the AI's. Target words: {words_list}."
88     )
89
90
91 def report_user_prompt(transcript: list[tuple[str, str]]) -> str:
92     lines = "\n".join(f'{"learner" if speaker == "user" else "AI"}: {text}' for speaker, text in transcript)
93     return f"Transcript:\n{lines}"
94
95
96 @dataclass(frozen=True)
97 class TranscriptMetrics:
98     """The deterministic half. `None` means "not measurable for this session",
99     which is deliberately distinct from zero – a text session has no speech
100     rate at all, and reporting one would be an artifact of how fast the model
101     replied rather than anything about the learner."""
102
103     words_per_minute: int | None
104     filler_rate_per_100w: float
105     avg_response_delay_seconds: float | None
106     longest_run_words: int
107     type_token_ratio: float
108     above_level_words: list[str]
109     pronunciation_score: int | None
110     total_words: int
111
112
113 def _words(text: str) -> list[str]:
114     return _WORD_RE.findall(text.lower())
115
116
117 def says_word(text: str, word: str) -> bool:
118     """Whether `text` actually contains `word` as a word.
119

```

```

120 A plain substring test finds "cat" inside "category" and "art" inside
121 "start", which is enough to credit a learner with a target word they
122 never said – and to point the report's evidence link at the wrong turn.
123 Matched on the same normalised lemmas the other lexical metrics use, so
124 "goes" counts as "go" here exactly as it does there."""
125 wanted = cefr_lexicon.base_forms(word)
126 if not wanted:
127     return False
128 return any(wanted & cefr_lexicon.base_forms(w) for w in _words(text))
129
130
131 VALID_OUTCOMES = ("spontaneous", "prompted", "incorrect", "avoided")
132
133
134 def match_word_usage(word_usage: dict[str, str], word: str) -> str | None:
135     """The model's verdict for `word`, matched forgivingly.
136
137     The prompt lists the target words and asks for them back as JSON keys, and
138     a small model does not reliably echo them verbatim – it lowercases a
139     capitalised word, or answers about "wonders" when asked about "Wonder". An
140     exact dict lookup turns any of that into a silent miss that reads as
141     "avoided", which is indistinguishable from the learner never trying. Keys
142     are matched on shared base forms instead, the same way the transcript is
143     searched."""
144     wanted = cefr_lexicon.base_forms(word)
145     if not wanted:
146         return None
147     for key, outcome in word_usage.items():
148         if wanted & cefr_lexicon.base_forms(key) and outcome in VALID_OUTCOMES:
149             return outcome
150     return None
151
152
153 def resolve_cefr(level: str | None) -> str:
154     """A learner who hasn't been placed yet still gets a usable comparison
155     band rather than an above-level count that silently reads zero."""
156     if level and cefr_lexicon.is_valid_band(level):
157         return level.upper()
158     return DEFAULT_CEFR
159
160
161 def compute_metrics(turns: list, target_cefr: str) -> TranscriptMetrics:
162     """Pure function over turn rows. `turns` items need `speaker`, `text`,
163     `created_at`, and optionally `stt_confidence` / `speech_seconds`."""
164     user_turns = [t for t in turns if t["speaker"] == "user"]
165     all_words = [w for t in user_turns for w in _words(t["text"])]
166     total_words = len(all_words)
167
168     fillers = sum(1 for w in all_words if w in FILLER_WORDS)
169     filler_rate = round(100 * fillers / total_words, 1) if total_words else 0.0
170
171     # Real speech time, when we have it. Text sessions never do, and using
172     # wall-clock there would measure the model's latency, not the learner.
173     speech_seconds = sum(speech_seconds(t) for t in user_turns)
174     words_per_minute = round(total_words / (speech_seconds / 60.0)) if speech_seconds > 0 and total_words else None
175
176     # How long the learner took to start answering – the AI's own generation
177     # time sits outside this window, unlike the old all-turn-gaps average.
178     delays = []
179     for prev, cur in zip(turns, turns[1:]):
180         if prev["speaker"] == "ai" and cur["speaker"] == "user":
181             delays.append(_seconds_between(prev["created_at"], cur["created_at"]))
182     avg_response_delay = round(sum(delays) / len(delays), 1) if delays else None
183
184     longest_run = max((len(_words(t["text"])) for t in user_turns), default=0)
185
186     lemmas = {cefr_lexicon.normalise(w) for w in all_words}
187     type_token_ratio = round(len(lemmas) / total_words, 3) if total_words else 0.0
188
189     above_level = sorted({w for w in lemmas if cefr_lexicon.is_above_level(w, target_cefr)})
190
191     confidences = [t["stt_confidence"] for t in user_turns if t["stt_confidence"] is not None]
192     pronunciation = round(100 * sum(confidences) / len(confidences)) if confidences else None
193
194     return TranscriptMetrics(
195         words_per_minute=words_per_minute,

```

```

196         filler_rate_per_100w=filler_rate,
197         avg_response_delay_seconds=avg_response_delay,
198         longest_run_words=longest_run,
199         type_token_ratio=type_token_ratio,
200         above_level_words=above_level,
201         pronunciation_score=pronunciation,
202         total_words=total_words,
203     )
204
205
206 def _speech_seconds(turn) -> float:
207     try:
208         value = turn["speech_seconds"]
209     except (KeyError, IndexError):
210         return 0.0
211     return float(value) if value else 0.0
212
213
214 def _seconds_between(start_iso: str, end_iso: str) -> float:
215     from datetime import datetime
216
217     start = datetime.fromisoformat(start_iso.replace("Z", "+00:00"))
218     end = datetime.fromisoformat(end_iso.replace("Z", "+00:00"))
219     return (end - start).total_seconds()

```

HTTP surface

6. backend/app/routers/conversation.py

Nine endpoints: engine status, list/start/fetch/delete a session, submit a turn, end a session, fetch or regenerate a report, and stream a turn's audio chunk. Regeneration is a POST, not part of the GET, because it spends a real model call and rewrites review history — a GET must do neither.

backend/app/routers/conversation.py

261 lines

```

1  import json
2  import sqlite3
3  from pathlib import Path
4
5  from fastapi import APIRouter, Depends, File, Form, HTTPException, UploadFile, status
6  from fastapi.concurrency import run_in_threadpool
7  from fastapi.responses import FileResponse
8
9  from app.db import get_db
10 from app.models.conversation import (
11     ConversationReportOut,
12     ConversationSessionCreate,
13     ConversationSessionDetailOut,
14     ConversationSessionOut,
15     ConversationTurnOut,
16     EngineStatusOut,
17     TargetWordOut,
18     TurnSubmitOut,
19 )
20 from app.security import require_token
21 from app.services import conversation
22 from app.services.voice import tts
23 from app.services.voice.errors import EngineUnavailable
24
25 router = APIRouter(prefix="/conversation", dependencies=[Depends(require_token)])
26
27
28 def _turn_out(
29     row: sqlite3.Row, channel: str = "text", engine: str = tts.DEFAULT_ENGINE
30 ) -> ConversationTurnOut:
31     # How a reply is cut up depends on the engine, and the client both asks
32     # for exactly this many chunks and highlights words against their text -

```

```

33 # so this has to be the engine the audio endpoint will actually use.
34 chunk_texts = conversation.audio_chunk_texts(row, channel, engine)
35 chunks = len(chunk_texts)
36 # Legacy turns still carry a single pre-rendered wav in audio_path; newer
37 # ones are synthesized per sentence on request.
38 has_audio = chunks > 0 or bool(row["audio_path"])
39 return ConversationTurnOut(
40     id=row["id"],
41     turn_index=row["turn_index"],
42     speaker=row["speaker"],
43     text=row["text"],
44     audio_url=f"/conversation/turns/{row['id']}/audio" if has_audio else None,
45     audio_chunk_count=chunks,
46     audio_chunks=chunk_texts,
47     stt_confidence=row["stt_confidence"],
48     created_at=row["created_at"],
49 )
50
51
52 def _target_words_out(conn: sqlite3.Connection, session: sqlite3.Row) -> list[TargetWordOut]:
53     word_ids = json.loads(session["target_word_ids"])
54     if not word_ids:
55         return []
56     rows = conn.execute(
57         f"SELECT id, word FROM vocab_words WHERE id IN ({','.join('?' for _ in word_ids)}", word_ids
58     ).fetchall()
59
60     outcomes: dict[str, str] = {}
61     if session["report_json"]:
62         report = json.loads(session["report_json"])
63         outcomes = {r["word"]: r["outcome"] for r in report.get("routing", [])}
64
65     return [TargetWordOut(id=r["id"], word=r["word"], used_outcome=outcomes.get(r["word"])) for r in rows]
66
67
68 def _session_out(conn: sqlite3.Connection, row: sqlite3.Row) -> ConversationSessionOut:
69     engine = conversation.session_engine(conn, row)
70     return ConversationSessionOut(
71         id=row["id"],
72         user_id=row["user_id"],
73         scenario=row["scenario"],
74         channel=row["channel"],
75         target_words=_target_words_out(conn, row),
76         started_at=row["started_at"],
77         ended_at=row["ended_at"],
78         has_report=row["report_json"] is not None,
79         engine_provider=engine["provider"],
80         engine_label=engine["label"],
81     )
82
83
84 def _get_owned_session(conn: sqlite3.Connection, session_id: str, user_id: str) -> sqlite3.Row:
85     row = conversation.get_session_row(conn, session_id, user_id)
86     if row is None:
87         raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Conversation session not found")
88     return row
89
90
91 @router.get("/engine-status", response_model=EngineStatusOut)
92 def get_engine_status(user_id: str, conn: sqlite3.Connection = Depends(get_db)) -> EngineStatusOut:
93     # "llm" is pinned to whichever model/provider this user currently has
94     # selected - see conversation.full_engine_status()'s docstring.
95     return EngineStatusOut(**conversation.full_engine_status(conn, user_id))
96
97
98 @router.get("/sessions", response_model=list[ConversationSessionOut])
99 def list_sessions(user_id: str, conn: sqlite3.Connection = Depends(get_db)) -> list[ConversationSessionOut]:
100     return [_session_out(conn, row) for row in conversation.list_sessions(conn, user_id)]
101
102
103 @router.post("/sessions", response_model=ConversationSessionOut)
104 def start_session(
105     payload: ConversationSessionCreate, conn: sqlite3.Connection = Depends(get_db)
106 ) -> ConversationSessionOut:
107     try:
108         session_id = conversation.start_session(

```

```

109         conn,
110         user_id=payload.user_id,
111         scenario=payload.scenario,
112         channel=payload.channel,
113         seed_word_ids=payload.seed_word_ids,
114     )
115 except EngineUnavailable as err:
116     raise HTTPException(status_code=status.HTTP_503_SERVICE_UNAVAILABLE, detail=str(err)) from err
117 row = conversation.get_session_row(conn, session_id, payload.user_id)
118 return _session_out(conn, row)
119
120
121 @router.get("/sessions/{session_id}", response_model=ConversationSessionDetailOut)
122 def get_session_detail(
123     session_id: str, user_id: str, conn: sqlite3.Connection = Depends(get_db)
124 ) -> ConversationSessionDetailOut:
125     row = _get_owned_session(conn, session_id, user_id)
126     base = _session_out(conn, row)
127     return ConversationSessionDetailOut(
128         **base.model_dump(),
129         turns=[
130             _turn_out(t, row["channel"], tts.selected_name(conn, user_id))
131             for t in conversation.get_turns(conn, session_id)
132         ],
133     )
134
135
136 @router.post("/sessions/{session_id}/turns", response_model=TurnSubmitOut)
137 async def submit_turn(
138     session_id: str,
139     user_id: str,
140     text: str | None = Form(default=None),
141     audio: UploadFile | None = File(default=None),
142     conn: sqlite3.Connection = Depends(get_db),
143 ) -> TurnSubmitOut:
144     session = _get_owned_session(conn, session_id, user_id)
145     engine = tts.selected_name(conn, user_id)
146     # A session with a report already isn't "closed" – ending just produces a
147     # checkpoint report; the learner can keep talking afterward and re-end
148     # later for an updated one (see end_session below).
149
150     audio_bytes = await audio.read() if audio is not None else None
151     if not text and not audio_bytes:
152         raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail="Provide text or audio")
153
154     try:
155         # This does real, slow, CPU-bound work (STT + LLM generation, maybe
156         # TTS) – run it off the event loop so it doesn't block every other
157         # concurrent request (e.g. engine-status polling) for the whole
158         # duration, which otherwise looks like a dropped connection ("Failed
159         # to fetch") to the frontend.
160         user_turn, ai_turn = await run_in_threadpool(
161             conversation.submit_user_turn, conn, session, text=text, audio_bytes=audio_bytes
162         )
163     except EngineUnavailable as err:
164         raise HTTPException(status_code=status.HTTP_503_SERVICE_UNAVAILABLE, detail=str(err)) from err
165     except ValueError as err:
166         raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail=str(err)) from err
167
168     return TurnSubmitOut(
169         user_turn=_turn_out(user_turn, session["channel"], engine),
170         ai_turn=_turn_out(ai_turn, session["channel"], engine),
171     )
172
173
174 @router.post("/sessions/{session_id}/end", response_model=ConversationReportOut)
175 def end_session(session_id: str, user_id: str, conn: sqlite3.Connection = Depends(get_db)) -> ConversationReportOut:
176     """Always recomputes – a session can be ended, continued, and ended again,
177     each time producing a fresh checkpoint report over the full transcript so
178     far. The per-target-word usage logs are replaced, not appended, so one
179     conversation only ever counts once no matter how often it's re-ended."""
180     session = _get_owned_session(conn, session_id, user_id)
181     try:
182         report = conversation.end_session(conn, session)
183     except EngineUnavailable as err:
184         raise HTTPException(status_code=status.HTTP_503_SERVICE_UNAVAILABLE, detail=str(err)) from err

```

```

185     return ConversationReportOut(**report)
186
187
188 @router.delete("/sessions/{session_id}", status_code=status.HTTP_204_NO_CONTENT)
189 def delete_session(session_id: str, user_id: str, conn: sqlite3.Connection = Depends(get_db)) -> None:
190     if not conversation.delete_session(conn, user_id, session_id):
191         raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Conversation session not found")
192
193
194 @router.get("/sessions/{session_id}/report", response_model=ConversationReportOut)
195 def get_report(session_id: str, user_id: str, conn: sqlite3.Connection = Depends(get_db)) -> ConversationReportOut:
196     """A pure read. Reports written by an older version are missing the fields
197     added since, which every default on ConversationReportOut covers – so an
198     old report opens and honestly reports what it never measured, rather than
199     500-ing on validation. Use the regenerate endpoint to bring one up to date."""
200     session = _get_owned_session(conn, session_id, user_id)
201     if session["report_json"] is None:
202         raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Session hasn't ended yet")
203     return ConversationReportOut(**json.loads(session["report_json"]))
204
205
206 @router.post("/sessions/{session_id}/report/regenerate", response_model=ConversationReportOut)
207 async def regenerate_report(
208     session_id: str, user_id: str, conn: sqlite3.Connection = Depends(get_db)
209 ) -> ConversationReportOut:
210     """Re-runs the analysis over the stored transcript – used to bring a report
211     written by an older version up to the current shape.
212
213     Deliberately a POST and not folded into the GET above: it spends a real LLM
214     call (cloud quota included) and it rewrites this session's review_logs.
215     Neither belongs in a read."""
216     session = _get_owned_session(conn, session_id, user_id)
217     try:
218         report = await run_in_threadpool(conversation.end_session, conn, session)
219     except EngineUnavailable as err:
220         raise HTTPException(status_code=status.HTTP_503_SERVICE_UNAVAILABLE, detail=str(err)) from err
221     return ConversationReportOut(**report)
222
223
224 @router.get("/turns/{turn_id}/audio")
225 async def get_turn_audio(
226     turn_id: str, user_id: str, chunk: int = 0, conn: sqlite3.Connection = Depends(get_db)
227 ) -> FileResponse:
228     """One sentence of a reply, synthesized on first request and cached.
229
230     Splitting it this way is what makes a spoken reply start within seconds
231     instead of tens of seconds: synthesis is slower than real time here, so the
232     whole reply is never made up front."""
233     row = conn.execute(
234         """
235         SELECT ct.id, ct.text, ct.speaker, ct.audio_path, cs.channel FROM conversation_turns ct
236         JOIN conversation_sessions cs ON cs.id = ct.session_id
237         WHERE ct.id = ? AND cs.user_id = ?
238         """,
239         (turn_id, user_id),
240     ).fetchone()
241     if row is None:
242         raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="No audio for this turn")
243
244     # Turns recorded before streaming existed have the whole reply in one file.
245     if row["audio_path"] and chunk == 0 and Path(row["audio_path"]).exists():
246         return FileResponse(row["audio_path"], media_type="audio/wav")
247
248     engine = tts.selected_name(conn, user_id)
249     if conversation.audio_chunk_count(row, row["channel"], engine) == 0:
250         raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="No audio for this turn")
251
252     try:
253         path = await run_in_threadpool(
254             conversation.ensure_audio_chunk, turn_id, row["text"], chunk, engine
255         )
256     except IndexError as err:
257         raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="No such audio chunk") from err
258     except EngineUnavailable as err:
259         raise HTTPException(status_code=status.HTTP_503_SERVICE_UNAVAILABLE, detail=str(err)) from err
260     return FileResponse(str(path), media_type="audio/wav")

```

Voice engines

7. backend/app/services/voice/stt_engine.py

faster-whisper.transcribe() handles a microphone clip and returns text, a confidence proxy and the summed duration of recognised speech. transcribe_file() is the windowed path added for subtitle generation.

backend/app/services/voice/stt_engine.py

261 lines

```
1 """Local speech-to-text via faster-whisper. Model size is pinned to
2 tiny.en – the smallest real option – given this app's stated hardware
3 tier (see hardware_capability.py). faster-whisper *can* auto-download it
4 from Hugging Face on construction, but that's exactly the implicit-download
5 behavior this app moved away from (see download_manager.py) – so this
6 module checks the cache itself first and refuses to construct the model
7 (and therefore trigger a download) until Settings has fetched it.
8 """
9
10 import io
11 import threading
12 from pathlib import Path
13
14 from app.services.voice import model_manager
15 from app.services.voice.errors import EngineUnavailable
16
17 MODEL_SIZE = "tiny.en"
18 _SAMPLE_RATE = 16000
19
20 # A clip this quiet never contains speech – it's what a muted mic or an input
21 # device that isn't actually capturing produces. Worth telling apart from a
22 # real recording nothing could be made out of, because the fix is completely
23 # different (check your mic vs. speak more clearly).
24 _SILENCE_PEAK = 0.01
25 # Chromium emits a valid WebM with container headers but essentially no audio
26 # if the mic is stopped right after starting, so byte count alone can't catch
27 # an accidental double-tap.
28 _MIN_SECONDS = 0.4
29
30 _lock = threading.Lock()
31 _model = None
32
33
34 def _load_model_locked():
35     """Must only be called while holding `_lock` – see the matching note in
36     llm_chat_engine._load_llm_locked for why the whole load-and-transcribe
37     operation, not just loading, needs to be serialized."""
38     global _model
39     if _model is not None:
40         return _model
41     try:
42         from faster_whisper import WhisperModel
43     except ImportError as err:
44         raise EngineUnavailable("faster-whisper isn't installed") from err
45     cache = model_manager.whisper_cache_dir()
46     if not any(cache.glob(f"models--Systran--faster-whisper-{MODEL_SIZE}/**/*.bin")):
47         raise EngineUnavailable("The local STT model isn't downloaded yet – download it in Settings first.")
48     try:
49         _model = WhisperModel(
50             MODEL_SIZE,
51             device="cpu",
52             compute_type="int8",
53             download_root=str(model_manager.whisper_cache_dir()),
54         )
55     except Exception as err: # noqa: BLE001
56         raise EngineUnavailable(f"Couldn't load the local STT model: {err}") from err
57     return _model
58
59
60 def is_ready() -> bool:
```

```

61     return _model is not None
62
63
64 def warm_up() -> None:
65     """See llm_chat_engine.warm_up - same explicit-load-ahead-of-use idea."""
66     with _lock:
67         _load_model_locked()
68
69
70 def unload() -> None:
71     """See llm_chat_engine.unload - called when the model is deleted from
72     disk via Settings. Only one STT option exists, so any delete means
73     unconditionally unload."""
74     global _model
75     with _lock:
76         _model = None
77
78
79 def transcribe(audio_bytes: bytes) -> tuple[str, float, float]:
80     """Returns (text, confidence, speech_seconds).
81
82     `confidence` is a 0-1 proxy derived from the average log-probability
83     faster-whisper reports per segment - a real signal, not a fabricated
84     score, but explicitly a proxy for true word-level confidence (matches the
85     report's existing `stt_proxy` naming).
86
87     `speech_seconds` is the summed span of the recognised segments, not the
88     length of the clip: recording runs continuously in hands-free mode, so the
89     raw clip carries silence on both sides that would drag a words-per-minute
90     figure down for no reason the learner could act on.
91
92     Raises ValueError (a bad recording, surfaced as a 400) when the clip is
93     too short or silent, so those two cases get their own actionable message
94     instead of the caller's generic "nothing was recognised".
95     """
96     from faster_whisper.audio import decode_audio
97
98     with _lock:
99         model = _load_model_locked()
100         try:
101             audio = decode_audio(io.BytesIO(audio_bytes), sampling_rate=_SAMPLE_RATE)
102             except Exception as err: # noqa: BLE001
103                 raise EngineUnavailable(f"Couldn't read that recording: {err}") from err
104
105             seconds = len(audio) / _SAMPLE_RATE
106             peak = float(max(abs(audio.min(initial=0.0)), abs(audio.max(initial=0.0)))) if len(audio) else 0.0
107             if seconds < _MIN_SECONDS:
108                 raise ValueError(
109                     f"That recording was only {seconds:.1f}s long - tap the mic, speak, then tap again to stop."
110                 )
111             if peak < _SILENCE_PEAK:
112                 raise ValueError(
113                     "That recording came through silent - check your microphone isn't muted and that the right "
114                     "input device is selected, then try again."
115                 )
116
117             try:
118                 # Already decoded above, so hand the samples over directly rather
119                 # than making faster-whisper decode the same bytes a second time.
120                 segments, _info = model.transcribe(audio, vad_filter=True, language="en")
121                 segments = list(segments)
122             except Exception as err: # noqa: BLE001
123                 raise EngineUnavailable(f"Local transcription failed: {err}") from err
124
125             if not segments:
126                 return "", 0.0, 0.0
127
128             text = " ".join(s.text.strip() for s in segments).strip()
129             avg_logprob = sum(s.avg_logprob for s in segments) / len(segments)
130             # avg_logprob is typically in roughly [-1, 0]; clamp/rescale to a 0-1 proxy.
131             confidence = max(0.0, min(1.0, 1.0 + avg_logprob))
132             speech_seconds = round(sum(max(0.0, s.end - s.start) for s in segments), 2)
133             return text, confidence, speech_seconds
134
135
136 # How much audio the model sees at once. Ten minutes costs ~38 MB as float32

```

```

137 # and is long enough that window boundaries are rare; the figure trades memory
138 # against how often a sentence has to be stitched across a seam.
139 WINDOW_SECONDS = 600.0
140 # How far back from a window's end we are willing to cut in order to land on a
141 # sentence boundary. Bounds the worst case: every window advances at least
142 # WINDOW_SECONDS - TAIL_GUARD_SECONDS, so the loop always terminates.
143 TAIL_GUARD_SECONDS = 30.0
144
145
146 def transcribe_file(
147     path: str,
148     *,
149     on_segment,
150     on_progress=None,
151     should_cancel=None,
152     language: str = "en",
153 ) -> int:
154     """Transcribe a whole media file into timed segments (spec §4.1.2,
155     "automatic subtitle generation"). Returns the number of segments emitted.
156
157     `on_segment(start_ms, end_ms, text)` is called as each segment is decoded
158     rather than at the end, so a two-hour film produces a usable, growing
159     track instead of nothing for twenty minutes. `should_cancel()` is polled
160     between windows so the user can stop a job they started by mistake.
161
162     **Read in windows, never whole.** Handing the path to faster-whisper makes
163     it decode the entire file into RAM first – measured at 1,162 MB peak for a
164     two-hour film, which is what made transcribing a movie kill the app on the
165     hardware tier this targets. Memory here is flat in the length of the film:
166     one window of audio at a time, ~38 MB, whether the file is ten minutes or
167     four hours.
168
169     Boundaries are stitched on sentence ends rather than on the clock. Whisper
170     cannot know that a window stopped mid-sentence, so the last segment of each
171     window is discarded and the next window starts where the last *kept*
172     segment ended – the discarded audio is re-read and transcribed with its
173     full context instead of being cut in half.
174
175     Holds `_lock` for the entire run, for the reason given on
176     `_load_model_locked`. The consequence is real and deliberate: while a file
177     is transcribing, Conversation's own speech recognition waits. One STT
178     model is loaded at a time on the hardware tier this app targets, and
179     running a second would be an out-of-memory kill rather than a slowdown –
180     so the caller (media/tracks.py) allows only one generation job at a time
181     and says plainly in the UI that it is running.
182     """
183     from app.services.voice import audio_windows
184
185     source = Path(path)
186     total_s = audio_windows.duration_seconds(source)
187     emitted = 0
188     position_s = 0.0
189
190     def report(seconds: float) -> None:
191         if on_progress is not None and total_s > 0:
192             on_progress(min(1.0, seconds / total_s))
193
194     with _lock:
195         model = _load_model_locked()
196         try:
197             while True:
198                 if should_cancel is not None and should_cancel():
199                     break
200
201                 window = audio_windows.read_window(source, position_s, WINDOW_SECONDS)
202                 if window.size == 0:
203                     break
204                 window_s = window.size / audio_windows.SAMPLE_RATE
205                 # Short read means end of file, so nothing after this window
206                 # can complete a sentence and the final segment is kept.
207                 is_last = window_s < WINDOW_SECONDS - 1.0
208
209                 segments, _info = model.transcribe(window, vad_filter=True, language=language)
210                 segments = [s for s in segments if (s.text or "").strip()]
211                 # Free the samples before the next window is read, so peak
212                 # memory is one window rather than two.

```

```

213         del window
214
215     if not segments:
216         position_s += window_s
217         report(position_s)
218         if is_last:
219             break
220         continue
221
222     # Decide whether the trailing segment can be trusted. Dropping
223     # it is only safe if we then resume at ITS start, so the audio
224     # is transcribed again with full context rather than skipped -
225     # resuming anywhere later loses the sentence outright.
226     tail = segments[-1]
227     resume_at = tail.start
228     if is_last or len(segments) == 1 or resume_at < WINDOW_SECONDS - TAIL_GUARD_SECONDS:
229         # Either there is nothing after this window to complete the
230         # sentence, or backing up to it would mean re-reading most
231         # of the window. Keep it: a sentence that may be clipped is
232         # better than one that is lost, and better than a crawl.
233         keep = segments
234         resume_at = window_s
235     else:
236         keep = segments[:-1]
237
238     for segment in keep:
239         on_segment(
240             int((position_s + segment.start) * 1000),
241             int((position_s + segment.end) * 1000),
242             segment.text.strip(),
243         )
244         emitted += 1
245
246     if is_last:
247         report(total_s or position_s + window_s)
248         break
249
250     # resume_at is either the dropped sentence's start or a whole
251     # window; both are at least WINDOW - TAIL_GUARD, so the loop
252     # always moves forward and always terminates.
253     position_s += resume_at
254     report(position_s)
255 except Exception as err: # noqa: BLE001
256     raise EngineUnavailable(f"Local transcription failed: {err}") from err
257
258 if on_progress is not None:
259     on_progress(1.0)
260 return emitted

```

8. backend/app/services/voice/llm_chat_engine.py

The local model, through llama.cpp. A lazy singleton behind a lock, because loading the same model twice on this hardware tier is an out-of-memory kill rather than a slowdown.

backend/app/services/voice/llm_chat_engine.py

154 lines

```

1  """Local LLM chat generation via llama-cpp-python, in-process inside this
2  backend (not a second spawned server) – matching hardware_capability.py's
3  own memory-budget framing, which only makes sense as a single process.
4
5  Lazy-imports llama_cpp so the rest of the backend keeps working even if it
6  fails to import/build on a given machine; the model itself is loaded once
7  and cached at module level (loading a ~1GB GGUF per request would be
8  unusable), reloaded only if the pinned model path changes.
9  """
10
11 import os
12 import threading
13
14 from app.services.voice import model_manager
15 from app.services.voice.errors import EngineUnavailable

```

```

16 from app.services.voice.json_utils import parse_json_object
17
18 _lock = threading.Lock()
19 _llm = None
20 _loaded_path: str | None = None
21
22
23 def _load_llm_locked(repo_id: str, filename: str):
24     """Must only be called while holding `_lock`. A llama.cpp context isn't
25     safe for concurrent calls from multiple threads – running two
26     generations on the same instance at once (e.g. a turn from a session the
27     user already navigated away from, still finishing server-side, overlapping
28     with a freshly-started session's opening line) corrupts internal state
29     and can hang both requests indefinitely rather than just being slow. So
30     every caller holds `_lock` for the full load-and-generate operation,
31     which serializes engine use instead of racing it."""
32     global _llm, _loaded_path
33     model_path = model_manager.llm_model_path(repo_id, filename)
34     if not model_path.exists():
35         raise EngineUnavailable(
36             f"{filename} isn't downloaded yet – download it in Settings before starting a conversation."
37         )
38     path = str(model_path)
39     if _llm is not None and _loaded_path == path:
40         return _llm
41     try:
42         from llama_cpp import Llama
43     except ImportError as err:
44         raise EngineUnavailable("llama-cpp-python isn't installed") from err
45     # Release the outgoing model *before* allocating the replacement. Holding
46     # both at once doubles peak memory for the length of the load, which on a
47     # light machine is precisely when switching models runs out of RAM.
48     _llm = None
49     _loaded_path = None
50     try:
51         _llm = Llama(
52             model_path=path,
53             n_ctx=4096,
54             n_threads=max(1, (os.cpu_count() or 2) - 1),
55             verbose=False,
56         )
57         _loaded_path = path
58     except Exception as err: # noqa: BLE001 – any load failure is "unavailable"
59         _llm = None
60         _loaded_path = None
61         raise EngineUnavailable(f"Couldn't load the local LLM: {err}") from err
62     return _llm
63
64
65 def is_ready() -> bool:
66     return _llm is not None
67
68
69 def is_ready_for(model_path: str) -> bool:
70     """Like is_ready(), but specific to one model file – a *different*
71     model being loaded (e.g. right after switching selection in Settings)
72     must not read as "AI is launched", or a turn would silently pay the
73     reload cost mid-conversation instead of the learner choosing when via
74     Launch AI. See loaded_path()/_loaded_path for what's actually resident."""
75     return _llm is not None and _loaded_path == model_path
76
77
78 def loaded_path() -> str | None:
79     return _loaded_path
80
81
82 def unload(model_path: str | None = None) -> None:
83     """Called when a model file is deleted from disk (Settings' delete
84     button) – an in-memory Llama instance for a now-deleted file is no
85     longer meaningfully "ready". Acquires the same lock generation uses, so
86     this waits for any in-flight reply rather than racing it. If
87     `model_path` is given, only unloads when it matches what's actually
88     loaded (deleting a *different*, non-selected option shouldn't drop the
89     one currently in use); omit it to unconditionally unload (STT/TTS have
90     only one option each, so any delete means "unload it")."""
91     global _llm, _loaded_path

```

```

92     with _lock:
93         if model_path is None or _loaded_path == model_path:
94             _llm = None
95             _loaded_path = None
96
97
98 def warm_up(repo_id: str, filename: str) -> None:
99     """Explicitly loads the model into memory ahead of any real use – the
100     "Launch AI" action calls this so a deliberate click pays the cold-load
101     cost (real minutes on this hardware tier) instead of it happening
102     silently inside whatever the first real request turns out to be."""
103     with _lock:
104         _load_llm_locked(repo_id, filename)
105
106
107 def generate_reply(
108     system_prompt: str, history: list[tuple[str, str]], *, repo_id: str, filename: str, max_tokens: int = 220
109 ) -> str:
110     """history: list of (role, text) pairs, role in {'user', 'assistant'}."""
111     messages = [{"role": "system", "content": system_prompt}]
112     messages.extend({"role": role, "content": text} for role, text in history)
113
114     with _lock:
115         llm = _load_llm_locked(repo_id, filename)
116         try:
117             result = llm.create_chat_completion(messages=messages, max_tokens=max_tokens, temperature=0.7)
118         except Exception as err: # noqa: BLE001
119             raise EngineUnavailable(f"Local LLM generation failed: {err}") from err
120
121     text = result["choices"][0]["message"]["content"]
122     return text.strip()
123
124
125 def generate_json(
126     system_prompt: str,
127     user_prompt: str,
128     *,
129     repo_id: str,
130     filename: str,
131     max_tokens: int = 300,
132     temperature: float = 0.4,
133 ) -> dict:
134     """Shared by every feature that wants a structured (JSON) answer from the
135     local model rather than free-form chat text – the post-chat report analysis
136     and Vocabulary's AI explain/examples/mnemonic/practice features. The
137     prompts themselves live in the services, never here."""
138     with _lock:
139         llm = _load_llm_locked(repo_id, filename)
140         try:
141             result = llm.create_chat_completion(
142                 messages=[{"role": "system", "content": system_prompt}, {"role": "user", "content": user_prompt}],
143                 max_tokens=max_tokens,
144                 temperature=temperature,
145             )
146         except Exception as err: # noqa: BLE001
147             raise EngineUnavailable(f"Local LLM generation failed: {err}") from err
148
149     raw = result["choices"][0]["message"]["content"].strip()
150     parsed = parse_json_object(raw)
151     if parsed is None:
152         raise EngineUnavailable("The local LLM's response wasn't valid JSON")
153     return parsed

```

9. backend/app/services/voice/cloud_llm_engine.py

OpenRouter. Same two functions as the local engine.

backend/app/services/voice/cloud_llm_engine.py

121 lines

```

1 """Cloud LLM chat generation via OpenRouter (https://openrouter.ai) – the
2 opt-in alternative to the local llama.cpp path in llm_chat_engine.py.
3

```

```

4 Deliberately the plain stdlib (urllib), same choice download_manager.py
5 already made over adding an HTTP client dependency for a handful of
6 requests. There's no "load into memory" concept here (no warm_up/is_ready
7 singleton) – every call is a real, live network request, so readiness is
8 just "is an API key configured," checked by the caller (conversation.py)
9 before generation is attempted.
10 """
11
12 import json
13 import urllib.error
14 import urllib.request
15
16 from app.services.voice import engine_health
17 from app.services.voice.errors import EngineUnavailable
18 from app.services.voice.json_utils import parse_json_object
19
20 _API_URL = "https://openrouter.ai/api/v1/chat/completions"
21 DEFAULT_MODEL = "openai/gpt-4o-mini"
22
23
24 def _chat(messages: list[dict], *, api_key: str | None, model: str, max_tokens: int, temperature: float) -> str:
25     if not api_key:
26         raise EngineUnavailable(
27             "No OpenRouter API key configured – add one in Settings (AI → Cloud) to use the cloud AI."
28         )
29     body = json.dumps(
30         {"model": model, "messages": messages, "max_tokens": max_tokens, "temperature": temperature}
31     ).encode("utf-8")
32     request = urllib.request.Request(
33         _API_URL,
34         data=body,
35         method="POST",
36         headers={
37             "Authorization": f"Bearer {api_key}",
38             "Content-Type": "application/json",
39             # OpenRouter's recommended (optional) attribution headers.
40             "HTTP-Referer": "https://github.com/Team-CSQuanta/fluency-os",
41             "X-Title": "FluencyOS",
42         },
43     )
44     try:
45         with urllib.request.urlopen(request, timeout=60) as res:
46             data = json.loads(res.read().decode("utf-8"))
47     except urllib.error.HTTPError as err:
48         detail = err.read().decode("utf-8", errors="ignore")[:300]
49         message = f"OpenRouter request failed ({err.code}): {detail}"
50         engine_health.record_failure("openrouter", model, message)
51         raise EngineUnavailable(message) from err
52     except urllib.error.URLError as err:
53         message = f"Couldn't reach OpenRouter: {err.reason}"
54         engine_health.record_failure("openrouter", model, message)
55         raise EngineUnavailable(message) from err
56     # The service answered – that, and only that, is what a verified key means.
57     engine_health.record_success("openrouter", model)
58
59     try:
60         choice = data["choices"][0]
61         text = str(choice["message"]["content"]).strip()
62     except (KeyError, IndexError, TypeError) as err:
63         raise EngineUnavailable("OpenRouter returned an unexpected response shape") from err
64
65     if choice.get("finish_reason") == "length":
66         # See gemini_llm_engine for why a truncated reply is refused rather
67         # than stored: it would become part of the conversation history.
68         raise EngineUnavailable(
69             f"{model} ran out of output budget before finishing its reply – if it reasons before "
70             "answering, that reasoning spends the same budget. Try a non-reasoning model in Settings."
71         )
72     return text
73
74
75 def generate_reply(
76     system_prompt: str,
77     history: list[tuple[str, str]],
78     *,
79     api_key: str | None,

```

```

80     model: str,
81     max_tokens: int = 220,
82 ) -> str:
83     messages = [{"role": "system", "content": system_prompt}]
84     messages.extend({"role": role, "content": text} for role, text in history)
85     return _chat(messages, api_key=api_key, model=model, max_tokens=max_tokens, temperature=0.7)
86
87
88 def verify(*, api_key: str | None, model: str) -> None:
89     """One real, minimal request – the only thing that can actually establish
90     that a key works. Raises EngineUnavailable with the real reason if not."""
91     _chat(
92         [{"role": "user", "content": "ping"}],
93         api_key=api_key,
94         model=model,
95         max_tokens=8,
96         temperature=0.0,
97     )
98
99
100 def generate_json(
101     system_prompt: str,
102     user_prompt: str,
103     *,
104     api_key: str | None,
105     model: str,
106     max_tokens: int = 300,
107     temperature: float = 0.4,
108 ) -> dict:
109     raw = _chat(
110         [{"role": "system", "content": system_prompt}, {"role": "user", "content": user_prompt}],
111         api_key=api_key,
112         model=model,
113         max_tokens=max_tokens,
114         temperature=temperature,
115     )
116     parsed = parse_json_object(raw)
117     if parsed is None:
118         raise EngineUnavailable("OpenRouter's response wasn't valid JSON")
119     return parsed

```

10. backend/app/services/voice/gemini_llm_engine.py

Google Gemini. Same two functions again.

backend/app/services/voice/gemini_llm_engine.py

164 lines

```

1  """Cloud LLM chat generation via Google AI Studio's Gemini API
2  (https://ai.google.dev) – a second opt-in cloud provider alongside
3  cloud_llm_engine.py's OpenRouter path. Same stdlib-only (urllib) choice, same
4  "no load step, every call is live" readiness model – see cloud_llm_engine.py
5  for the shared reasoning.
6
7  Gemini's REST shape differs from OpenRouter's OpenAI-compatible one: chat
8  history uses role "model" instead of "assistant", the system prompt is a
9  separate `system_instruction` field rather than a message, and JSON output
10 is requested via `generationConfig.responseMimeType` rather than prompt
11 instructions alone (kept as a belt-and-suspenders backup via
12 parse_json_object, in case a model ignores it).
13 """
14
15 import json
16 import urllib.error
17 import urllib.request
18
19 from app.services.voice import engine_health
20 from app.services.voice.errors import EngineUnavailable
21 from app.services.voice.json_utils import parse_json_object
22
23 _API_BASE = "https://generativelanguage.googleapis.com/v1beta/models"
24 DEFAULT_MODEL = "gemini-2.0-flash"
25

```

```

26 # Gemini's thinking models (2.5 onward) charge internal reasoning tokens
27 # against maxOutputTokens *before* emitting any visible text, so a budget
28 # sized for the answer alone gets spent on reasoning and the reply comes back
29 # chopped mid-sentence – or empty. Thinking gets its own headroom on top of
30 # whatever the caller asked for; how long the answer actually runs is steered
31 # by the prompt, not by this ceiling.
32 _THINKING_HEADROOM_TOKENS = 2048
33
34
35 def _history_to_contents(history: list[tuple[str, str]]) -> list[dict]:
36     return [{"role": "model" if role == "assistant" else "user", "parts": [{"text": text}]} for role, text in
    history]
37
38
39 def _request(
40     system_prompt: str,
41     contents: list[dict],
42     *,
43     api_key: str | None,
44     model: str,
45     max_tokens: int,
46     temperature: float,
47     json_mode: bool = False,
48 ) -> str:
49     if not api_key:
50         raise EngineUnavailable(
51             "No Gemini API key configured – add one in Settings (AI → Cloud) to use the cloud AI."
52         )
53     generation_config: dict = {
54         "maxOutputTokens": max_tokens + _THINKING_HEADROOM_TOKENS,
55         "temperature": temperature,
56     }
57     if json_mode:
58         generation_config["responseMimeType"] = "application/json"
59     body = json.dumps(
60         {
61             "system_instruction": {"parts": [{"text": system_prompt}]},
62             "contents": contents,
63             "generationConfig": generation_config,
64         }
65     ).encode("utf-8")
66     request = urllib.request.Request(
67         f"{_API_BASE}/{model}:generateContent",
68         data=body,
69         method="POST",
70         headers={"x-goog-api-key": api_key, "Content-Type": "application/json"},
71     )
72     try:
73         with urllib.request.urlopen(request, timeout=60) as res:
74             data = json.loads(res.read().decode("utf-8"))
75     except urllib.error.HTTPError as err:
76         detail = err.read().decode("utf-8", errors="ignore")[:300]
77         message = f"Gemini request failed ({err.code}): {detail}"
78         engine_health.record_failure("gemini", model, message)
79         raise EngineUnavailable(message) from err
80     except urllib.error.URLError as err:
81         message = f"Couldn't reach Gemini: {err.reason}"
82         engine_health.record_failure("gemini", model, message)
83         raise EngineUnavailable(message) from err
84     # The service answered – that, and only that, is what a verified key means.
85     engine_health.record_success("gemini", model)
86
87     try:
88         candidate = data["candidates"][0]
89         parts = (candidate.get("content") or {}).get("parts") or []
90         # A thinking model can return its reasoning as parts flagged
91         # "thought" – those are not the reply and must never be shown as it.
92         text = "".join(str(p.get("text", "")) for p in parts if not p.get("thought")).strip()
93     except (KeyError, IndexError, TypeError) as err:
94         raise EngineUnavailable("Gemini returned an unexpected response shape") from err
95
96     finish_reason = candidate.get("finishReason")
97     if finish_reason == "MAX_TOKENS":
98         # Storing the fragment would be worse than failing: it becomes part of
99         # the conversation history and every later turn is built on it.
100         raise EngineUnavailable(

```

```

101         f"{model} ran out of output budget before finishing its reply. Thinking models spend that "
102         f"budget on reasoning first – switch to a non-thinking model such as {DEFAULT_MODEL} in "
103         "Settings (AI → Cloud) for short conversational replies."
104     )
105     if not text:
106         raise EngineUnavailable(f"Gemini returned an empty reply (finishReason={finish_reason or 'unknown'})")
107     return text
108
109
110 def generate_reply(
111     system_prompt: str,
112     history: list[tuple[str, str]],
113     *,
114     api_key: str | None,
115     model: str,
116     max_tokens: int = 220,
117 ) -> str:
118     _request(
119         system_prompt,
120         _history_to_contents(history),
121         api_key=api_key,
122         model=model,
123         max_tokens=max_tokens,
124         temperature=0.7,
125     )
126
127
128 def verify(*, api_key: str | None, model: str) -> None:
129     """One real, minimal request – the only thing that can actually establish
130     that a key works. Raises EngineUnavailable with the real reason if not."""
131     _request(
132         "Reply with the single word OK.",
133         [{"role": "user", "parts": [{"text": "ping"}]}],
134         api_key=api_key,
135         model=model,
136         max_tokens=8,
137         temperature=0.0,
138     )
139
140
141 def generate_json(
142     system_prompt: str,
143     user_prompt: str,
144     *,
145     api_key: str | None,
146     model: str,
147     max_tokens: int = 300,
148     temperature: float = 0.4,
149 ) -> dict:
150     raw = _request(
151         system_prompt,
152         [{"role": "user", "parts": [{"text": user_prompt}]}],
153         api_key=api_key,
154         model=model,
155         max_tokens=max_tokens,
156         temperature=temperature,
157         json_mode=True,
158     )
159     parsed = parse_json_object(raw)
160     if parsed is None:
161         raise EngineUnavailable("Gemini's response wasn't valid JSON")
162     return parsed

```

11. backend/app/services/voice/tts.py

Dispatcher over the text-to-speech engines, with Pocket TTS as the default.

backend/app/services/voice/tts.py

91 lines

```

1  """Which text-to-speech engine speaks, and the one interface both satisfy.
2
3  tts_engine (Kokoro) and pocket_tts_engine (Pocket TTS) expose the same five

```

```

4 functions – is_ready / warm_up / unload / split_for_streaming / synthesize –
5 so everything upstream of here can be engine-agnostic. This module is the
6 only place that knows which one a given user has chosen.
7
8 Pocket TTS is the default and the engine this app is tuned for. Kokoro is
9 kept as a fallback rather than removed: it needs no PyTorch and far less
10 memory, so on a machine where Pocket TTS will not load it is the difference
11 between a slower voice and no voice at all. It also remains the only option
12 if the optional runtime is somehow missing from an install.
13 """
14
15 import sqlite3
16 from typing import Literal, Protocol
17
18 from app.services.voice import model_catalog, pocket_tts_engine, tts_engine
19
20 TtsEngineName = Literal["kokoro", "pocket"]
21 # Pocket TTS is what the app speaks with: measured at 1.03s to first audio
22 # against Kokoro's 2.22s, and crucially at ~0.53x real time where Kokoro
23 # drifts above 1.0x and starts leaving gaps in long replies.
24 DEFAULT_ENGINE: TtsEngineName = "pocket"
25 ENGINE_NAMES: tuple[TtsEngineName, ...] = ("kokoro", "pocket")
26
27 ENGINE_LABELS: dict[str, str] = {
28     "kokoro": "Kokoro af_heart",
29     "pocket": "Pocket TTS (Kyutai)",
30 }
31
32
33 class TtsEngine(Protocol):
34     def is_ready(self) -> bool: ...
35     def warm_up(self) -> None: ...
36     def unload(self) -> None: ...
37     def split_for_streaming(self, text: str) -> list[str]: ...
38     def synthesize(self, text: str) -> bytes: ...
39
40
41 _ENGINES: dict[str, object] = {"kokoro": tts_engine, "pocket": pocket_tts_engine}
42
43
44 def normalise(name: str | None) -> TtsEngineName:
45     return name if name in ENGINE_NAMES else DEFAULT_ENGINE # type: ignore[return-value]
46
47
48 def engine_for(name: str | None):
49     """The module implementing the named engine, falling back to the default
50     rather than raising – an unrecognised value in the column (a downgrade, a
51     hand-edited DB) should still produce speech, not a 500."""
52     return _ENGINES[normalise(name)]
53
54
55 def selected_name(conn: sqlite3.Connection, user_id: str) -> TtsEngineName:
56     row = conn.execute("SELECT tts_engine FROM user_settings WHERE user_id = ?", (user_id,)).fetchone()
57     return normalise(row["tts_engine"]) if row else None
58
59
60 def selected(conn: sqlite3.Connection, user_id: str):
61     return engine_for(selected_name(conn, user_id))
62
63
64 def is_downloaded(name: str | None) -> bool:
65     return (
66         model_catalog.pocket_tts_is_downloaded()
67         if normalise(name) == "pocket"
68         else model_catalog.tts_is_downloaded()
69     )
70
71
72 def is_installed(name: str | None) -> bool:
73     """Kokoro's runtime is a required dependency; Pocket TTS's is an extra."""
74     return pocket_tts_engine.is_installed() if normalise(name) == "pocket" else True
75
76
77 def any_ready() -> bool:
78     """Whether some voice is resident. The global AI indicator asks about
79     memory in general, not about the engine this user happens to have picked,

```

```

80     so switching engines mustn't make an already-loaded voice read as unloaded
81     while its RAM is still held."""
82     return tts_engine.is_ready() or pocket_tts_engine.is_ready()
83
84
85 def unload_all() -> None:
86     """Frees whichever voice is resident. Used by the global unload and when
87     switching engines – the outgoing engine's memory is exactly what the
88     incoming one needs."""
89     tts_engine.unload()
90     pocket_tts_engine.unload()

```

12. backend/app/services/voice/pocket_tts_engine.py

Pocket TTS (Kyutai). Holds its lock across generation — unlike the Kokoro engine — because `generate_audio_stream` is not thread-safe.

backend/app/services/voice/pocket_tts_engine.py

225 lines

```

1  """Local text-to-speech via Pocket TTS (Kyutai). The alternative to
2  tts_engine.py's Kokoro, same lazy-singleton shape as the other engines, and
3  like them it never downloads implicitly – Settings/download_manager.py does.
4
5  Why it exists alongside Kokoro: Kokoro is non-streaming, so nothing is
6  audible until an entire chunk has been synthesized. That is the floor under
7  first-audio latency and the only reason conversation.py splits replies into
8  two chunks at all. Pocket TTS is autoregressive and yields audio while it is
9  still generating, so the first sound does not wait on the rest of the
10 sentence.
11
12 It is not a free win: it needs PyTorch and roughly half a gigabyte more
13 resident memory, and its lead narrows on CPUs without AVX-512/VNNI. Which
14 engine is better is therefore a per-machine question, which is why this is a
15 user-visible setting rather than a replacement.
16 """
17
18 import io
19 import threading
20 import wave
21 from pathlib import Path
22
23 import numpy as np
24
25 from app.services.voice import model_manager, reply_chunking
26 from app.services.voice.errors import EngineUnavailable
27
28 VOICE = model_manager.POCKET_VOICE
29
30 # Pocket TTS resolves weights through a YAML config. Upstream's own configs
31 # point at hf:// URLs, which would re-download (from the gated repo) on every
32 # load; this is the same config with the three paths rewritten to the local
33 # files download_manager.py has already fetched. Copied from
34 # pocket_tts/config/english_2026-04.yaml – keep in step with that file if the
35 # pinned pocket-tts version changes.
36 _CONFIG_TEMPLATE = """\
37 weights_path: {weights}
38 default_temperature: 0.3
39
40 flow_lm:
41   insert_bos_before_voice: true
42   dtype: float32
43   flow:
44     depth: 6
45     dim: 512
46   transformer:
47     d_model: 1024
48     hidden_scale: 4
49     max_period: 10000
50     num_heads: 16
51     num_layers: 6
52   lookup_table:

```

```

53     dim: 1024
54     n_bins: 4000
55     tokenizer: sentencepiece
56     tokenizer_path: {tokenizer}
57
58 mimi:
59     dtype: float32
60     sample_rate: 24000
61     inner_dim: 32
62     outer_dim: 512
63     channels: 1
64     frame_rate: 12.5
65     seagnet:
66         dimension: 512
67         channels: 1
68         n_filters: 64
69         n_residual_layers: 1
70         ratios:
71             - 6
72             - 5
73             - 4
74         kernel_size: 7
75         residual_kernel_size: 3
76         last_kernel_size: 3
77         dilation_base: 2
78         pad_mode: constant
79         compress: 2
80     transformer:
81         d_model: 512
82         num_heads: 8
83         num_layers: 2
84         layer_scale: 0.01
85         context: 250
86         dim_feedforward: 2048
87         input_dimension: 512
88         output_dimensions:
89             - 512
90     quantizer:
91         dimension: 32
92         output_dimension: 512
93 """
94
95 _lock = threading.Lock()
96 _model = None
97 _voice_state = None
98 _sample_rate = 24000
99
100
101 def _config_path(weights: Path, tokenizer: Path) -> Path:
102     """Written next to the weights on first load rather than shipped, because
103     it has to name absolute local paths that only exist on this machine."""
104     path = weights.parent / "config.yaml"
105     wanted = _CONFIG_TEMPLATE.format(weights=weights, tokenizer=tokenizer)
106     if not path.exists() or path.read_text() != wanted:
107         path.write_text(wanted)
108     return path
109
110
111 def _load_locked():
112     """Must only be called while holding `_lock` - see tts_engine for why."""
113     global _model, _voice_state, _sample_rate
114     if _model is not None:
115         return _model, _voice_state
116     # Files before imports: "not downloaded yet" is the state a learner
117     # actually lands in and can act on, and checking it needs nothing loaded.
118     # A missing package is a broken install, not a user error.
119     weights, tokenizer, voice = model_manager.pocket_tts_paths()
120     if not (weights.exists() and tokenizer.exists() and voice.exists()):
121         raise EngineUnavailable("Pocket TTS isn't downloaded yet - download it in Settings first.")
122
123     try:
124         from pocket_tts import TTSMoel
125     except ImportError as err:
126         raise EngineUnavailable(
127             "pocket-tts isn't installed - reinstall the backend dependencies to use this voice."
128         ) from err

```

```

129
130     try:
131         import torch
132
133         model = TTSModel.load_model(config=_config_path(weights, tokenizer))
134         # pocket_tts pins torch to a single thread at import time. This
135         # machine class has four cores and nothing else runs during
136         # synthesis – the LLM has already finished by the time we synthesize –
137         # so capping at one core leaves most of the CPU idle. Two, not all
138         # four: the upstream benchmarks are quoted at two, and leaving
139         # headroom keeps the event loop responsive.
140         torch.set_num_threads(min(2, torch.get_num_threads() or 2))
141         # The voice embedding is a precomputed KV state, so this is a file
142         # read rather than inference – but it is ~6MB of tensors and every
143         # generation needs it, so it is loaded once here with the model.
144         state = model.get_state_for_audio_prompt(str(voice))
145     except EngineUnavailable:
146         raise
147     except Exception as err: # noqa: BLE001
148         raise EngineUnavailable(f"Couldn't load Pocket TTS: {err}") from err
149
150     _model, _voice_state, _sample_rate = model, state, model.sample_rate
151     return _model, _voice_state
152
153
154 def is_installed() -> bool:
155     """Whether the pocket-tts package is importable. It ships behind an
156     optional extra (it needs PyTorch), so unlike Kokoro this engine can be
157     listed and downloadable on a machine that cannot actually run it – which
158     Settings has to be able to say out loud rather than only discovering at
159     the first attempt to speak."""
160     from importlib.util import find_spec
161
162     return find_spec("pocket_tts") is not None
163
164
165 def is_ready() -> bool:
166     return _model is not None
167
168
169 def warm_up() -> None:
170     with _lock:
171         _load_locked()
172
173
174 def unload() -> None:
175     global _model, _voice_state
176     with _lock:
177         _model = None
178         _voice_state = None
179
180
181 def split_for_streaming(text: str) -> list[str]:
182     """The same two-chunk rule as Kokoro, despite this engine streaming.
183
184     Streaming would make splitting pointless if the audio reached the client
185     as it was produced – the first frame lands in ~0.25s. It doesn't:
186     ensure_audio_chunk writes a complete WAV and then serves the file, so the
187     learner waits for the whole of chunk 0 either way. Measured on the
188     reference machine, one chunk per reply cost 2.05s to first audio against
189     1.09s when split – so "it streams" is not a reason to skip the split, it
190     is a reason to stream end to end some day."""
191     return reply_chunking.split_for_streaming(text)
192
193
194 def synthesize(text: str) -> bytes:
195     """Returns WAV bytes for the given text."""
196     clean = text.strip()
197     if not clean:
198         raise EngineUnavailable("Nothing to synthesize")
199
200     # Held across generation, unlike Kokoro's lock which covers loading only:
201     # generate_audio_stream is documented as NOT thread-safe (it mutates the
202     # model state and runs its own decoder thread), so two concurrent turns
203     # would corrupt each other's audio rather than merely contend.
204     with _lock:

```

```

205     model, state = _load_locked()
206     try:
207         chunks = [c for c in model.generate_audio_stream(state, clean, copy_state=True)]
208     except Exception as err: # noqa: BLE001
209         raise EngineUnavailable(f"Pocket TTS synthesis failed: {err}") from err
210
211     if not chunks:
212         raise EngineUnavailable("Pocket TTS produced no audio")
213
214     import torch
215
216     samples = torch.cat([c.reshape(-1) for c in chunks]).numpy()
217     pcm16 = (np.clip(samples, -1.0, 1.0) * 32767).astype(np.int16)
218     buffer = io.BytesIO()
219     with wave.open(buffer, "wb") as wav_file:
220         wav_file.setnchannels(1)
221         wav_file.setsampwidth(2)
222         wav_file.setframerate(_sample_rate)
223         wav_file.writeframes(pcm16.tobytes())
224     return buffer.getvalue()

```

13. backend/app/services/voice/reply_chunking.py

Splits a reply into two pieces at a sentence boundary so the first can be synthesised and returned while the second is still being made.

backend/app/services/voice/reply_chunking.py

49 lines

```

1  """How a spoken reply is cut up before synthesis.
2
3  Shared by both voice engines, because the reason for splitting is not a
4  property of either of them – it is a property of how this app serves audio.
5  'ensure_audio_chunk' synthesizes a chunk to a complete WAV file and only then
6  serves it, so what a learner waits for is the full synthesis of chunk 0, never
7  an engine's internal time-to-first-frame. Cutting the opening sentence off is
8  what makes that first wait short.
9
10 Measured on the reference machine (Ryzen 3 3200G, 4 cores, no AVX-512/VNNI),
11 synthesizing the first sentence alone rather than the whole reply:
12
13     Kokoro      2.44s -> (already split)
14     Pocket TTS  2.05s -> 1.09s
15
16 Pocket TTS streams internally and so needs no split to produce sound early –
17 but that sound cannot reach the client through a file endpoint, so it needs
18 the split just as much as Kokoro does. Were audio ever streamed end to end,
19 its first frame lands in ~0.25s and this module stops mattering for it.
20 """
21
22 import re
23
24 _SENTENCE_END = re.compile(r"(?<=[.!?])\s+")
25
26 # An opener shorter than this (roughly two words) is not worth its own
27 # synthesis call: the per-call fixed cost would exceed what splitting saves,
28 # and it delays the remainder without meaningfully advancing the first sound.
29 _MIN_CHUNK_CHARS = 12
30
31
32 def split_for_streaming(text: str) -> list[str]:
33     """At most two pieces: the opening sentence, then everything else.
34
35     Exactly one split, not more. Each additional chunk buys nothing but its
36     own fixed cost – on Kokoro, splitting a reply four ways measured ~2.7s
37     slower overall. One split is worth paying for: it gets the first sentence
38     audible far sooner, and the remainder is synthesized while it plays."""
39     clean = (text or "").strip()
40     if not clean:
41         return []
42     sentences = [p.strip() for p in _SENTENCE_END.split(clean) if p.strip()]
43     if len(sentences) <= 1:

```

```

44     return sentences
45     head, rest = sentences[0], " ".join(sentences[1:])
46     if len(head) < _MIN_CHUNK_CHARS:
47         return [f"{head} {rest}"]
48     return [head, rest]

```

14. backend/app/services/voice/json_utils.py

Salvages JSON from a model that wrapped it in prose, fenced it, or left a bracket open.

backend/app/services/voice/json_utils.py

125 lines

```

1  """Getting a JSON object out of what a small language model actually returns.
2
3  Every AI feature in the app asks for JSON and a 1B model obliges most of the
4  time. The rest of the time it is *nearly* JSON, and the difference between
5  salvaging that and refusing it is the difference between a working feature
6  and a 503 the learner can do nothing about.
7
8  These are the failures observed in practice, not hypotheticals – the repair
9  below exists because of output like this, from gemma-3-1b:
10
11     ```json
12     {"examples": ["one.", "two.", "three."]}
13     ```
14
15  which never closes the array and then closes the object twice.
16  """
17
18  import json
19  import re
20
21  _FENCE = re.compile(r"^\s*````(?:json)?\s*|\s*```\s*$", re.IGNORECASE)
22  # A trailing comma before a closing bracket – legal in JavaScript, not JSON,
23  # and something models emit constantly.
24  _TRAILING_COMMA = re.compile(r",\s*([}\]])")
25
26
27  def parse_json_object(raw: str) -> dict | None:
28      """The model's JSON object, or None if nothing usable can be recovered.
29
30      Only ever returns a dict: every caller expects keyed fields, and quietly
31      handing back a list would push the failure into their .get() calls."""
32      value = parse_json_value(raw)
33      return value if isinstance(value, dict) else None
34
35
36  def parse_json_value(raw: str) -> dict | list | None:
37      """As above but also accepts a bare array, which models return when the
38      requested object has exactly one list-valued key."""
39      if not raw:
40          return None
41      text = _FENCE.sub("", raw.strip()).strip()
42
43      for candidate in _candidates(text):
44          try:
45              return json.loads(candidate)
46          except json.JSONDecodeError:
47              continue
48      return None
49
50
51  def _candidates(text: str):
52      """Progressively more aggressive readings of the same text, cheapest
53      first. Each is handed to the real parser – nothing here decides that a
54      string is valid, it only proposes what to try."""
55      yield text
56      yield _TRAILING_COMMA.sub(r"\1", text)
57
58      # The model often wraps its JSON in a sentence or two. Start at the first
59      # opening bracket and let the balancer decide where it ends.
60      start = min(

```

```

61         (i for i in (text.find("{"), text.find("(")) if i != -1),
62         default=-1,
63     )
64     if start == -1:
65         return
66     body = text[start:]
67     repaired = _balance(body)
68     if repaired is not None:
69         yield repaired
70         yield _TRAILING_COMMA.sub(r"\1", repaired)
71
72
73 def _balance(text: str) -> str | None:
74     """Closes brackets the model left open, and cuts off anything after the
75     structure it actually finished.
76
77     Walks the text tracking whether we are inside a string (and whether the
78     previous character escaped the next), so a brace inside a quoted sentence
79     is never mistaken for structure. Two repairs come out of it:
80
81     * extra closers – stop at the point the stack would underflow, which is
82     the `}}` case above;
83     * missing closers – append what is still open, in reverse, which also
84     rescues a response cut short by hitting max_tokens.
85     """
86     stack: list[str] = []
87     in_string = False
88     escaped = False
89     end = None
90
91     for i, ch in enumerate(text):
92         if in_string:
93             if escaped:
94                 escaped = False
95             elif ch == "\\":
96                 escaped = True
97             elif ch == "'":
98                 in_string = False
99             continue
100
101         if ch == "'":
102             in_string = True
103         elif ch in "{[":
104             stack.append("{}" if ch == "{" else "]")
105         elif ch in "}]" :
106             if not stack or stack[-1] != ch:
107                 # A closer that does not match what is open. Everything from
108                 # here on is noise; keep what was well-formed.
109                 end = i
110                 break
111             stack.pop()
112         if not stack:
113             end = i + 1
114             break
115
116     body = text if end is None else text[:end]
117     if not body.strip():
118         return None
119
120     if in_string:
121         body += "'"
122     # A value left dangling after its comma ("a", ) parses once the comma
123     # goes, which _TRAILING_COMMA handles on the next candidate.
124     return body + "".join(reversed(stack))

```

Renderer — state

15. `renderer/src/store/conversationStore.ts`

The Zustand store. Owns the session, its turns, engine status and the report; every method is an HTTP request. It never touches audio hardware.

`renderer/src/store/conversationStore.ts`

239 lines

```
1 import { create } from 'zustand';
2 import { api, fetchBlobUrl } from '@lib/apiClient';
3 import { useAppStore } from '@store/appStore';
4 import type {
5   ConversationChannel,
6   ConversationReportOut,
7   ConversationSessionDetailOut,
8   ConversationSessionOut,
9   EngineStatusOut,
10  ScenarioKey,
11  TurnSubmitOut,
12 } from '@types/api';
13
14 interface ConversationState {
15   sessions: ConversationSessionOut[];
16   sessionsStatus: 'idle' | 'loading' | 'error';
17
18   activeSession: ConversationSessionDetailOut | null;
19   activeStatus: 'idle' | 'loading' | 'error';
20   activeError: string | null;
21
22   report: ConversationReportOut | null;
23   reportStatus: 'idle' | 'loading' | 'error';
24   reportError: string | null;
25
26   engineStatus: EngineStatusOut | null;
27
28   // True only right after startSession created a brand-new session – lets
29   // ConversationLive auto-play the AI's opening line for a genuinely fresh
30   // conversation without ever auto-playing on a plain resume, where the
31   // same fetchSessionDetail call is used to load an existing transcript.
32   justStarted: boolean;
33
34   fetchSessions: () => Promise<void>;
35   startSession: (scenario: ScenarioKey, channel: ConversationChannel, seedWordIds?: string[]) => Promise<string>;
36   regenerateReport: (sessionId: string) => Promise<void>;
37   fetchSessionDetail: (sessionId: string) => Promise<void>;
38   consumeJustStarted: () => boolean;
39   submitTextTurn: (sessionId: string, text: string) => Promise<TurnSubmitOut>;
40   submitAudioTurn: (sessionId: string, blob: Blob) => Promise<TurnSubmitOut>;
41   endSession: (sessionId: string) => Promise<void>;
42   fetchReport: (sessionId: string) => Promise<void>;
43   fetchEngineStatus: () => Promise<void>;
44   turnAudioUrl: (turnId: string, chunk?: number) => Promise<string>;
45   deleteSession: (sessionId: string) => Promise<void>;
46 }
47
48 function requireUserId(): string {
49   const id = useAppStore.getState().currentUserId;
50   if (!id) throw new Error('No signed-in user – cannot use Conversation yet');
51   return id;
52 }
53
54 // Tracks the most recently requested session id so an out-of-order
55 // fetchSessionDetail response (an earlier request that resolves after a
56 // later one) can be dropped instead of clobbering fresher data.
57 let lastRequestedSessionId: string | null = null;
58
59 export const useConversationStore = create<ConversationState>((set, get) => ({
60   sessions: [],
```

```

61 sessionsStatus: 'idle',
62
63 activeSession: null,
64 activeStatus: 'idle',
65 activeError: null,
66
67 report: null,
68 reportStatus: 'idle',
69 reportError: null,
70
71 engineStatus: null,
72 justStarted: false,
73
74 fetchSessions: async () => {
75   set({ sessionsStatus: 'loading' });
76   try {
77     const userId = requireUserId();
78     const sessions = await api.get<ConversationSessionOut[]>(
79       `/conversation/sessions?user_id=${encodeURIComponent(userId)}`,
80     );
81     set({ sessions, sessionsStatus: 'idle' });
82   } catch {
83     set({ sessionsStatus: 'error' });
84   }
85 },
86
87 startSession: async (scenario, channel, seedWordIds) => {
88   const userId = requireUserId();
89   const session = await api.post<ConversationSessionOut>('/conversation/sessions', {
90     user_id: userId,
91     scenario,
92     channel,
93     seed_word_ids: seedWordIds,
94   });
95   set({ activeSession: null, activeStatus: 'idle', report: null });
96   await get().fetchSessionDetail(session.id);
97   // fetchSessionDetail always clears justStarted at the top of its own
98   // call (so a resume never sets it) – set it true here, after, so only
99   // this genuinely-fresh creation flow leaves it set.
100   set({ justStarted: true });
101   return session.id;
102 },
103
104 consumeJustStarted: () => {
105   const value = get().justStarted;
106   set({ justStarted: false });
107   return value;
108 },
109
110 fetchSessionDetail: async (sessionId) => {
111   lastRequestedSessionId = sessionId;
112   set({ activeStatus: 'loading', activeError: null, justStarted: false });
113   try {
114     const userId = requireUserId();
115     const detail = await api.get<ConversationSessionDetailOut>(
116       `/conversation/sessions/${sessionId}?user_id=${encodeURIComponent(userId)}`,
117     );
118     // If a newer fetchSessionDetail call has started since (e.g. the
119     // learner clicked resume on a different session before this one
120     // returned), drop this response instead of overwriting the newer one.
121     if (lastRequestedSessionId !== sessionId) return;
122     set({ activeSession: detail, activeStatus: 'idle' });
123   } catch (err) {
124     if (lastRequestedSessionId !== sessionId) return;
125     set({ activeStatus: 'error', activeError: err instanceof Error ? err.message : String(err) });
126   }
127 },
128
129 submitTextTurn: async (sessionId, text) => {
130   const userId = requireUserId();
131   const form = new FormData();
132   form.set('text', text);
133   const result = await api.postForm<TurnSubmitOut>(
134     `/conversation/sessions/${sessionId}/turns?user_id=${encodeURIComponent(userId)}`,
135     form,
136   );

```

```

137 // Guard against a stale response landing after the learner has since
138 // navigated away and resumed a *different* session – without the id
139 // check this would silently append the old session's turns onto
140 // whatever session happens to be active by the time this resolves.
141 set((s) =>
142   s.activeSession?.id === sessionId
143   ? { activeSession: { ...s.activeSession, turns: [...s.activeSession.turns, result.user_turn, result.ai_turn]
144 } }
145   : {},
146 );
147 return result;
148 },
149 submitAudioTurn: async (sessionId, blob) => {
150   const userId = requireUserId();
151   const form = new FormData();
152   form.set('audio', blob, 'turn.webm');
153   const result = await api.postForm<TurnSubmitOut>(
154     `/conversation/sessions/${sessionId}/turns?user_id=${encodeURIComponent(userId)}`,
155     form,
156   );
157   set((s) =>
158     s.activeSession?.id === sessionId
159     ? { activeSession: { ...s.activeSession, turns: [...s.activeSession.turns, result.user_turn, result.ai_turn]
160 } }
161     : {},
162   );
163   return result;
164 },
165 endSession: async (sessionId) => {
166   const userId = requireUserId();
167   const report = await api.post<ConversationReportOut>(
168     `/conversation/sessions/${sessionId}/end?user_id=${encodeURIComponent(userId)}`,
169   );
170   set({ report, reportStatus: 'idle', reportError: null });
171   // Ending is what sets ended_at and has_report server-side, so without
172   // this the history list keeps showing the conversation as unfinished and
173   // offers no way into the report that was just generated.
174   await get().fetchSessions();
175 },
176 fetchReport: async (sessionId) => {
177   // Clearing 'report' up front matters: without it a failed fetch left the
178   // previously-viewed session's report on screen, so the Report screen
179   // showed another conversation's numbers as if they were this one's.
180   set({ reportStatus: 'loading', report: null, reportError: null });
181   try {
182     const userId = requireUserId();
183     const report = await api.get<ConversationReportOut>(
184       `/conversation/sessions/${sessionId}/report?user_id=${encodeURIComponent(userId)}`,
185     );
186     set({ report, reportStatus: 'idle' });
187   } catch (err) {
188     set({ reportStatus: 'error', reportError: err instanceof Error ? err.message : String(err) });
189   }
190 },
191 regenerateReport: async (sessionId) => {
192   // A POST, not a refetch: this re-runs the analysis and rewrites the
193   // session's usage logs, so it costs a real LLM call.
194   const userId = requireUserId();
195   set({ reportStatus: 'loading', reportError: null });
196   try {
197     const report = await api.post<ConversationReportOut>(
198       `/conversation/sessions/${sessionId}/report/regenerate?user_id=${encodeURIComponent(userId)}`,
199     );
200     set({ report, reportStatus: 'idle' });
201   } catch (err) {
202     // Keep the old report on screen – it's still the real one, just older.
203     set({ reportStatus: 'idle', reportError: err instanceof Error ? err.message : String(err) });
204   }
205 },
206 fetchEngineStatus: async () => {
207   const userId = requireUserId();
208 }

```

```

211     const engineStatus = await api.get<EngineStatusOut>(
212       `/conversation/engine-status?user_id=${encodeURIComponent(userId)}`,
213     );
214     set({ engineStatus });
215   },
216
217   turnAudioUrl: async (turnId, chunk = 0) => {
218     // This request is also what triggers synthesis of that sentence, so it can
219     // take a few seconds on the first fetch and is instant afterwards.
220     const userId = requireUserId();
221     return fetchBlobUrl(
222       `/conversation/turns/${turnId}/audio?user_id=${encodeURIComponent(userId)}&chunk=${chunk}`,
223     );
224   },
225
226   deleteSession: async (sessionId) => {
227     const userId = requireUserId();
228     await api.delete(`/conversation/sessions/${sessionId}?user_id=${encodeURIComponent(userId)}`);
229     set((s) => ({
230       sessions: s.sessions.filter((sess) => sess.id !== sessionId),
231       activeSession: s.activeSession?.id === sessionId ? null : s.activeSession,
232       // The report belongs to the session that just stopped existing –
233       // leaving it loaded would show a deleted conversation's numbers on the
234       // next visit to the Report screen.
235       report: s.activeSession?.id === sessionId ? null : s.report,
236     }));
237   },
238 });

```

Renderer — screens

16. `renderer/src/features/conversation/ConversationList.tsx`

Scenario picker and session history — the screen the learner starts from.

`renderer/src/features/conversation/ConversationList.tsx`

278 lines

```

1  import { useEffect, useState } from 'react';
2  import { CONV_SCENARIOS } from '@features/conversation/conversationMockData';
3  import { useConversationStore } from '@store/conversationStore';
4  import { useEngineStore } from '@store/engineStore';
5  import { useShellStore } from '@store/shellStore';
6  import type { ConversationSessionOut } from '@types/api';
7
8  const SCENARIO_LABEL: Record<string, string> = Object.fromEntries(CONV_SCENARIOS.map((s) => [s.key, s.n]));
9
10 function formatDuration(startedAt: string, endedAt: string | null): string {
11   if (!endedAt) return 'in progress';
12   const ms = new Date(endedAt).getTime() - new Date(startedAt).getTime();
13   const totalSeconds = Math.max(0, Math.round(ms / 1000));
14   const m = Math.floor(totalSeconds / 60);
15   const s = totalSeconds % 60;
16   return `${m}m ${s}s`;
17 }
18
19 export function ConversationList() {
20   const goConvLive = useShellStore((s) => s.goConvLive);
21   const goReport = useShellStore((s) => s.goReport);
22   const sessions = useConversationStore((s) => s.sessions);
23   const sessionsStatus = useConversationStore((s) => s.sessionsStatus);
24   const fetchSessions = useConversationStore((s) => s.fetchSessions);
25   const startSession = useConversationStore((s) => s.startSession);
26   const fetchSessionDetail = useConversationStore((s) => s.fetchSessionDetail);
27   const fetchReport = useConversationStore((s) => s.fetchReport);
28   const deleteSession = useConversationStore((s) => s.deleteSession);
29   const fetchEngineStatus = useConversationStore((s) => s.fetchEngineStatus);
30   const engineStatus = useConversationStore((s) => s.engineStatus);
31   // Every scenario below starts a voice session, so this screen must gate on
32   // voice readiness specifically – text readiness always reports STT/TTS as

```

```

33 // ready and would wave through a session that can't actually run.
34 const readiness = useEngineStore((s) => s.readiness.voice);
35 const fetchReadiness = useEngineStore((s) => s.fetchReadiness);
36 const engineGlobalStatus = useEngineStore((s) => s.status);
37 const fetchEngineGlobalStatus = useEngineStore((s) => s.fetchStatus);
38 const llmProvider = useEngineStore((s) => s.llmProvider);
39 const fetchLlmProvider = useEngineStore((s) => s.fetchLlmProvider);
40 const aiLaunching = useEngineStore((s) => s.launching);
41 const aiLaunchError = useEngineStore((s) => s.launchError);
42 const launchAi = useEngineStore((s) => s.launchAi);
43 const goSettings = useShellStore((s) => s.goSettings);
44 const [starting, setStarting] = useState<string | null>(null);
45 const [startError, setStartError] = useState<string | null>(null);
46 const [deletingId, setDeletingId] = useState<string | null>(null);
47
48 useEffect(() => {
49   void fetchSessions();
50   void fetchReadiness('voice');
51   void fetchEngineGlobalStatus();
52   void fetchLlmProvider();
53 }, [fetchSessions, fetchReadiness, fetchEngineGlobalStatus, fetchLlmProvider]);
54
55 const notReady = readiness !== null && !readiness.ready;
56 // A cloud provider has nothing to download for the LLM – what it's missing
57 // is an API key, so "go download a model" would be the wrong instruction.
58 const usingCloud = llmProvider !== null && llmProvider.provider !== 'local';
59 // Every scenario here runs on the voice channel, so all three engines
60 // (LLM/STT/TTS) need to be actually loaded, not just downloaded, before a
61 // session can start – see backend's _ensure_launched.
62 const aiLaunched =
63   engineGlobalStatus !== null &&
64   engineGlobalStatus.llm === 'ready' &&
65   engineGlobalStatus.stt === 'ready' &&
66   engineGlobalStatus.tts === 'ready';
67 const needsLaunch = !notReady && !aiLaunched;
68
69 // The local LLM/STT/TTS models load lazily on first real use and can take
70 // a real while (a ~1GB GGUF, cold CPU inference) – poll while a session is
71 // starting so "starting..." can become an honest "warming up the local
72 // model..." instead of looking stuck.
73 useEffect(() => {
74   if (starting === null) return;
75   void fetchEngineStatus();
76   const id = setInterval(() => void fetchEngineStatus(), 1500);
77   return () => clearInterval(id);
78 }, [starting, fetchEngineStatus]);
79
80 // Keeps this screen in sync if AI gets launched from elsewhere (the
81 // AppNav button) while it's open, without the learner needing to reload.
82 useEffect(() => {
83   if (!needsLaunch) return;
84   const id = setInterval(() => void fetchEngineGlobalStatus(), 2000);
85   return () => clearInterval(id);
86 }, [needsLaunch, fetchEngineGlobalStatus]);
87
88 const handleStart = async (scenarioKey: (typeof CONV_SCENARIOS)[number]['key']) => {
89   if (notReady || needsLaunch) return;
90   setStartError(null);
91   setStarting(scenarioKey);
92   try {
93     await startSession(scenarioKey, 'voice');
94     goConvLive(SCENARIO_LABEL[scenarioKey]);
95   } catch (err) {
96     setStartError(err instanceof Error ? err.message : 'Could not start a conversation');
97   } finally {
98     setStarting(null);
99   }
100 };
101
102 const warmingUp = starting !== null && engineStatus && engineStatus.llm !== 'ready';
103
104 const handleResume = async (session: ConversationSessionOut) => {
105   await fetchSessionDetail(session.id);
106   goConvLive(SCENARIO_LABEL[session.scenario] ?? session.scenario);
107 };
108

```

```

109 const handleReport = async (session: ConversationSessionOut) => {
110     await fetchReport(session.id);
111     goReport();
112 };
113
114 const handleDelete = async (session: ConversationSessionOut) => {
115     setDeletingId(session.id);
116     try {
117         await deleteSession(session.id);
118     } finally {
119         setDeletingId(null);
120     }
121 };
122
123 return (
124     <div className="h-full overflow-y-auto">
125         <div className="mx-auto max-w-[1000px] px-[var(--pad)] pb-11 pt-[18px]">
126             <div className="flex flex-wrap items-end gap-[18px] border-b border-line2 pb-[18px]">
127                 <div className="min-w-[260px] flex-1">
128                     <div className="font-sans text-[22px] font-semibold tracking-[-0.02em] text-tx">Conversations</div>
129                     <div className="mt-[6px] font-sans text-[11.5px] leading-[1.6] text-tx2">
130                         Practice with a real local AI partner – your saved vocabulary is injected as target words.
131                     </div>
132                 </div>
133                 <div className="min-w-[104px] rounded-field border border-line2 bg-panel px-3 py-[10px]">
134                     <div className="font-mono text-[8.5px] font-semibold uppercase tracking-[0.12em] text-
tx3">sessions</div>
135                     <div className="mt-[5px] font-sans text-[15px] font-semibold text-tx">{sessions.length}</div>
136                 </div>
137             </div>
138
139             <div className="mt-5 rounded-panel border border-acclLine bg-accSoft p-4">
140                 <div className="flex flex-wrap items-center gap-[14px]">
141                     <div className="min-w-[220px] flex-1">
142                         <div className="font-sans text-[13.5px] font-semibold text-tx">Start a new conversation</div>
143                         <div className="mt-1 font-sans text-[11px] leading-[1.6] text-tx2">
144                             {notReady
145                                 ? usingCloud && !readiness!.llm
146                                 ? 'Your cloud provider needs an API key before you can start.'
147                                 : "You'll need to download a model before you can start."
148                             : needsLaunch
149                                 ? 'Models are downloaded, but the AI needs to be launched before a conversation can use it.'
150                                 : 'Runs a real local model – the first message may take a while to warm up.'}
151                         </div>
152                     </div>
153                 </div>
154
155                 {notReady ? (
156                     <div className="mt-[14px] flex flex-wrap items-center gap-[10px] border-t border-acclLine pt-[13px]">
157                         <span className="font-mono text-[10.5px] leading-[1.6] text-tx2">
158                             Missing: {[
159                                 !readiness!.llm && (usingCloud ? 'a cloud API key' : 'AI model'),
160                                 !readiness!.stt && 'speech-to-text',
161                                 !readiness!.tts && 'text-to-speech',
162                             ]
163                                 .filter(Boolean)
164                                 .join(', ')}
165                         </span>
166                         <button
167                             onClick={() => goSettings('AI')}
168                             className="rounded-field bg-acc px-[13px] py-[7px] font-sans text-[11px] font-semibold text-white
169                             hover:brightness-110"
170                             >
171                             Go to Settings
172                         </button>
173                     </div>
174                 ) : needsLaunch ? (
175                     <div className="mt-[14px] flex flex-wrap items-center gap-[10px] border-t border-acclLine pt-[13px]">
176                         <span className="font-mono text-[10.5px] leading-[1.6] text-tx2">
177                             AI isn't launched yet – it needs to be loaded into memory first.
178                         </span>
179                         <button
180                             onClick={() => void launchAi().catch(() => {})}
181                             disabled={aiLaunching}
182                             className="rounded-field bg-acc px-[13px] py-[7px] font-sans text-[11px] font-semibold text-white
183                             hover:brightness-110 disabled:opacity-50"

```

```

182     >
183     {aiLaunching ? 'launching...' : 'Launch AI'}
184   </button>
185   {aiLaunchError && <span className="font-mono text-[10px] text-[#c0563f]">{aiLaunchError}</span>}
186 </div>
187 ) : (
188   <div className="mt-[14px] flex flex-wrap gap-[6px] border-t border-accLine pt-[13px]">
189     {CONV_SCENARIOS.map((c) => (
190       <button
191         key={c.key}
192         onClick={() => void handleStart(c.key)}
193         disabled={starting !== null}
194         className="flex items-center gap-[7px] rounded-full border border-line2 bg-panel px-3 py-[7px]
font-sans text-[11px] font-medium text-tx2 hover:border-acc disabled:opacity-50"
195       >
196         {starting === c.key ? (warmingUp ? 'warming up the local model...' : 'starting...') : c.n}
197         <span className="font-mono text-[9.5px] text-tx3">{c.k}</span>
198       </button>
199     )]}
200   </div>
201 )}
202 {startError && <div className="mt-3 font-mono text-[10.5px] text-[#c0563f]">{startError}</div>}
203 </div>
204
205 <div className="mb-[10px] mt-6 flex items-center justify-between gap-[10px]">
206   <span className="font-mono text-[9px] font-semibold uppercase tracking-[0.12em] text-tx3">
207     Previous sessions · {sessions.length}
208   </span>
209   <span className="font-mono text-[9.5px] text-tx3">transcripts kept locally</span>
210 </div>
211
212 {sessionsStatus === 'loading' && <div className="font-mono text-[11px] text-tx3">loading...</div>}
213 {sessionsStatus === 'idle' && sessions.length === 0 && (
214   <div className="font-sans text-[12px] leading-[1.6] text-tx2">
215     No conversations yet – start one above.
216   </div>
217 )}
218 <div className="flex flex-col gap-[9px]">
219   {sessions.map((c) => (
220     <div
221       key={c.id}
222       className="flex flex-wrap items-center gap-[14px] rounded-field border border-line2 bg-panel px-[15px]
py-[13px]"
223     >
224       <span className="grid h-[34px] w-[34px] flex-none place-items-center rounded-field bg-tile font-mono
text-[9px] text-tx3">
225         {c.scenario}
226       </span>
227       <span className="min-w-[180px] flex-1">
228         <span className="block font-sans text-[13px] font-semibold text-tx">
229           {SCENARIO_LABEL[c.scenario] ?? c.scenario}
230         </span>
231         <span className="mt-1 block font-mono text-[10px] text-tx3">
232           {new Date(c.started_at).toLocaleDateString()} · {formatDuration(c.started_at, c.ended_at)}
233         </span>
234         {/* Resuming keeps the engine it started on, so name it here
235           rather than letting the global indicator imply otherwise. */}
236         <span className="mt-[3px] block truncate font-mono text-[9.5px] text-tx3" title={c.engine_label}>
237           {c.engine_provider === 'local' ? 'local' : 'cloud'} · {c.engine_label}
238         </span>
239       </span>
240       <span className="min-w-[96px] flex-none">
241         <span className="block font-mono text-[9px] uppercase tracking-[0.1em] text-tx3">target words</span>
242         <span className="mt-[5px] block font-mono text-[10px] font-medium text-tx2">
243           {c.target_words.filter((w) => w.used_outcome === 'spontaneous' || w.used_outcome ===
'prompted').length}/
244           {c.target_words.length} used
245         </span>
246       </span>
247       <span className="flex flex-none gap-[6px]">
248         {c.has_report && (
249           <button
250             onClick={() => void handleReport(c)}
251             className="rounded-field border border-line px-[11px] py-[7px] font-mono text-[10.5px] font-
medium text-tx2 hover:border-acc hover:text-acc"
252           >

```

```

253         report
254     </button>
255 }
256 <button
257     onClick={() => void handleResume(c)}
258     className="rounded-field border border-acLine bg-accSoft px-3 py-[7px] font-mono text-[10.5px]
font-medium text-acc"
259     >
260         resume
261     </button>
262     <button
263         onClick={() => void handleDelete(c)}
264         disabled={deletingId === c.id}
265         title="Delete this conversation"
266         className="rounded-field border border-line px-[9px] py-[7px] font-mono text-[10.5px] font-medium
text-tx3 hover:border-[#c0563f] hover:text-[#c0563f] disabled:opacity-50"
267         >
268             {deletingId === c.id ? '...' : 'x'}
269         </button>
270     </span>
271 </div>
272 )}
273 </div>
274 </div>
275 </div>
276 );
277 }

```

17. `renderer/src/features/conversation/ConversationLive.tsx`

The live conversation. Drives the recorder, submits turns, plays reply audio chunk by chunk, and coordinates the word highlighting.

`renderer/src/features/conversation/ConversationLive.tsx`

637 lines

```

1  import { useEffect, useRef, useState } from 'react';
2  import { SpokenText, type SpeakingState } from './SpokenText';
3  import { spokenCount, timeWords } from './spokenTiming';
4  import { useMicRecorder } from '@features/conversation/useMicRecorder';
5  import { useVadRecorder } from '@features/conversation/useVadRecorder';
6  import { useConversationStore } from '@store/conversationStore';
7  import { useEngineStore } from '@store/engineStore';
8  import { useShellStore } from '@store/shellStore';
9
10 type MicState = 'idle' | 'recording' | 'thinking' | 'speaking' | 'listening' | 'hearing';
11
12 const MIC_META: Record<MicState, { label: string; sub: string; bg: string; bd: string; fg: string; icon: string }> =
13 {
14     idle: { label: 'Ready', sub: 'tap to talk', bg: 'var(--panel)', bd: 'var(--line2)', fg: 'var(--tx2)', icon: '●' },
15     recording: { label: 'Recording...', sub: 'tap to stop', bg: 'var(--accSoft)', bd: 'var(--acc)', fg: 'var(--acc)',
16 icon: '■' },
17     listening: { label: 'Listening', sub: 'just start talking', bg: 'var(--panel)', bd: 'var(--acc)', fg: 'var(--
18 acc)', icon: '●' },
19     hearing: { label: 'I hear you...', sub: 'pause when you're done', bg: 'var(--accSoft)', bd: 'var(--acc)', fg: 'var(-
20 -acc)', icon: '■' },
21     thinking: { label: 'Thinking...', sub: 'transcribing + generating a reply', bg: 'var(--panel)', bd: 'var(--line2)',
22 fg: 'var(--tx2)', icon: '...' },
23     speaking: { label: 'Juno is speaking', sub: 'playing the reply', bg: 'var(--tile)', bd: 'var(--line2)', fg: 'var(-
24 -tx)', icon: '■■' },
25 };
26
27 export function ConversationLive() {
28     const scenario = useShellStore((s) => s.convScenario);
29     const goScreen = useShellStore((s) => s.goScreen);
30     const goReport = useShellStore((s) => s.goReport);
31     const setConvBusy = useShellStore((s) => s.setConvBusy);
32
33     const activeSession = useConversationStore((s) => s.activeSession);
34     const activeStatus = useConversationStore((s) => s.activeStatus);
35     const submitTextTurn = useConversationStore((s) => s.submitTextTurn);
36     const submitAudioTurn = useConversationStore((s) => s.submitAudioTurn);

```

```

31  const endSession = useConversationStore((s) => s.endSession);
32  const turnAudioUrl = useConversationStore((s) => s.turnAudioUrl);
33
34  const engineGlobalStatus = useEngineStore((s) => s.status);
35  const fetchEngineGlobalStatus = useEngineStore((s) => s.fetchStatus);
36  const fetchLlmProvider = useEngineStore((s) => s.fetchLlmProvider);
37  const aiLaunching = useEngineStore((s) => s.launching);
38  const aiLaunchError = useEngineStore((s) => s.launchError);
39  const launchAi = useEngineStore((s) => s.launchAi);
40
41  const [micState, setMicState] = useState<MicState>('idle');
42  const [textInput, setTextInput] = useState('');
43  const [error, setError] = useState<string | null>(null);
44  const [ending, setEnding] = useState(false);
45  const [handsFree, setHandsFree] = useState(true);
46  // Shown immediately on submit, before the round trip (STT + LLM reply)
47  // resolves – otherwise a typed message just sits invisible in the input's
48  // cleared state until the AI's reply arrives alongside it, which reads as
49  // the app ignoring the message rather than thinking about it.
50  const [pendingUserText, setPendingUserText] = useState<string | null>(null);
51  const recorder = useMicRecorder();
52  // Lets the unmount cleanup reach the current recorder without re-running.
53  const recorderRef = useRef(recorder);
54  recorderRef.current = recorder;
55  const lastAutoPlayedTurnId = useRef<string | null>(null);
56  // Only one reply should ever be audible at once – the direct-trigger call
57  // in the submit handlers below and the mount/resume fallback effect can
58  // otherwise both end up starting playback for close-together turns (and a
59  // manual "► play" tap can land while an autoplay is still going), each
60  // producing its own independent Audio object with nothing stopping the
61  // others. Tracking the single currently-playing element and always
62  // stopping it first makes overlap impossible regardless of which path
63  // triggered it.
64  const currentAudioRef = useRef<HTMLAudioElement | null>(null);
65  // Bumped to cancel an in-progress sentence-by-sentence playback run.
66  const playbackIdRef = useRef(0);
67  // Which turn is being spoken and how far into it, so the bubble can light
68  // up in time with the voice. Null whenever nothing is playing, which is
69  // what makes finished replies render as ordinary text.
70  const [speaking, setSpeaking] = useState<SpeakingState & { turnId: string } | null>(null);
71  const transcriptEndRef = useRef<HTMLDivElement | null>(null);
72
73  const turns = activeSession?.turns ?? [];
74  const isVoice = activeSession?.channel === 'voice';
75  // This session's own pinned engine – the one its turns actually call. The
76  // global engine state reflects what Settings points at *now*, which can be
77  // something else entirely, so gating on it claimed the AI was ready while
78  // the turns were going somewhere else.
79  const sessionUsesCloud = activeSession !== null && activeSession.engine_provider !== 'local';
80  // A turn already being submitted. Two overlapping submissions would each
81  // compute the next turn_index from the same pre-insert snapshot server-side,
82  // producing two turns with the same index and an interleaved transcript –
83  // so mic and send are both held until the in-flight one lands.
84  const turnInFlight = pendingUserText !== null;
85  // Text-only turns don't need STT/TTS loaded, only the LLM – voice needs
86  // all three, matching backend's _ensure_launched requirements. A
87  // cloud-pinned session has no LLM to load at all, so requiring one would
88  // block a conversation that is perfectly able to run.
89  const aiLaunched =
90    engineGlobalStatus !== null &&
91    (sessionUsesCloud || engineGlobalStatus.llm === 'ready') &&
92    (!isVoice || (engineGlobalStatus.stt === 'ready' && engineGlobalStatus.tts === 'ready'));
93
94  useEffect(() => {
95    transcriptEndRef.current?.scrollIntoView({ block: 'end' });
96  }, [turns.length, pendingUserText]);
97
98  useEffect(() => {
99    void fetchEngineGlobalStatus();
100    void fetchLlmProvider();
101  }, [fetchEngineGlobalStatus, fetchLlmProvider]);
102
103  // Keeps this in sync if AI gets launched from elsewhere (AppNav, or the
104  // sessions list) while this screen is already open.
105  useEffect(() => {
106    if (aiLaunched) return;

```

```

107     const id = setInterval(() => void fetchEngineGlobalStatus(), 2000);
108     return () => clearInterval(id);
109 }, [aiLaunched, fetchEngineGlobalStatus]);
110
111 // Tells the shell whether a real operation is in flight, so navigating
112 // away (sidebar, "< sessions") gets gated behind a confirmation instead of
113 // silently abandoning it – see LeaveConversationDialog. Deliberately
114 // excludes 'ending': clicking "End & analyse" is itself the leave action,
115 // so its own goReport() call shouldn't immediately re-prompt.
116 useEffect(() => {
117     setConvBusy(micState !== 'idle' || pendingUserText !== null);
118 }, [micState, pendingUserText, setConvBusy]);
119
120 // Leaving this screen (confirmed or otherwise) shouldn't leave the mic hot
121 // or a reply still audibly playing in the background.
122 useEffect(() => {
123     return () => {
124         setConvBusy(false);
125         // stop() is a no-op on an inactive recorder, so this is called
126         // unconditionally – reading recorder.status here would see the *first*
127         // render's value (this cleanup has no deps) and so never fire.
128         void recorderRef.current.stop();
129         if (currentAudioRef.current) {
130             currentAudioRef.current.pause();
131             currentAudioRef.current = null;
132         }
133     };
134     // eslint-disable-next-line react-hooks/exhaustive-deps
135 }, []);
136
137 /** Stops whatever the AI is currently saying, and reports whether anything
138  * was actually playing. */
139 const stopPlayback = () => {
140     // Invalidate any running sequence first, so its next sentence never starts.
141     playbackIdRef.current += 1;
142     setSpeaking(null);
143     const audio = currentAudioRef.current;
144     if (!audio) return false;
145     audio.onended = null;
146     audio.onerror = null;
147     audio.pause();
148     currentAudioRef.current = null;
149     return true;
150 };
151
152 /** Plays a reply one sentence at a time, fetching the next while the current
153  * one is audible.
154  *
155  * Each fetch is also what synthesizes that sentence, and synthesis is slower
156  * than real time on this hardware – so waiting for a whole reply before any
157  * of it was audible cost tens of seconds. Starting on sentence one cuts the
158  * wait to roughly its own synthesis time. 'playbackId' makes every sequence
159  * cancellable, so barge-in or a newer reply stops the whole run rather than
160  * just the clip currently playing. */
161 const playTurnAudio = async (turnId: string, chunkTexts: string[] = []) => {
162     lastAutoPlayedTurnId.current = turnId; // dedupes against the effect below firing for the same turn
163     stopPlayback();
164     const runId = ++playbackIdRef.current;
165     // A turn recorded before the API returned chunk texts still has exactly
166     // one clip to play – it just gets no word highlighting, which is the
167     // right fallback rather than refusing to speak.
168     const total = Math.max(1, chunkTexts.length);
169     setMicState('speaking');
170
171     const fetchChunk = (index: number) =>
172         index < total ? turnAudioUrl(turnId, index).catch(() => null) : Promise.resolve(null);
173
174     // Each chunk arrives as an object URL holding a whole WAV in memory, and
175     // the browser frees one only when it is explicitly revoked – closing the
176     // page is otherwise the only thing that reclaims it. A voice conversation
177     // fetches two per reply, so leaving them was a steady leak of a few
178     // hundred KB per turn for as long as the app stayed open. Every URL this
179     // run creates is tracked and released when the run ends, including the
180     // prefetched one that barge-in means we never play.
181     const created: string[] = [];
182     const trackedFetch = async (index: number) => {

```

```

183     const url = await fetchChunk(index);
184     if (url) created.push(url);
185     return url;
186 };
187
188 let pending = trackedFetch(0);
189 try {
190     for (let i = 0; i < total; i += 1) {
191         const url = await pending;
192         if (runId !== playbackIdRef.current) return; // superseded
193         if (!url) break;
194         // Kick off the next sentence's synthesis before playing this one, so
195         // the gap between sentences is as small as the hardware allows.
196         pending = trackedFetch(i + 1);
197         const words = timeWords(chunkTexts[i] ?? '');
198         await new Promise<void>((resolve) => {
199             const audio = new Audio(url);
200             currentAudioRef.current = audio;
201
202             // Read from the element every frame rather than running a timer
203             // alongside it. currentTime is the sound's own position, so the
204             // highlight cannot drift away from what is audible – through a
205             // stall, a slow decode, or a pause.
206             let frame = 0;
207             let lastCount = -1;
208             const follow = () => {
209                 if (runId !== playbackIdRef.current) return;
210                 const total = audio.duration;
211                 if (Number.isFinite(total) && total > 0) {
212                     const count = spokenCount(words, audio.currentTime / total);
213                     // Only when the word actually changes: re-rendering the
214                     // transcript 60 times a second to move a highlight nobody can
215                     // see move is not worth it on this hardware.
216                     if (count !== lastCount) {
217                         lastCount = count;
218                         setSpeaking({ turnId, chunkIndex: i, spokenInChunk: count });
219                     }
220                 }
221                 frame = requestAnimationFrame(follow);
222             };
223             frame = requestAnimationFrame(follow);
224
225             const finish = () => {
226                 cancelAnimationFrame(frame);
227                 resolve();
228             };
229             audio.onended = finish;
230             audio.onerror = finish;
231             void audio.play().catch(finish);
232         });
233         if (runId !== playbackIdRef.current) return;
234         // The clip finished: mark every one of its words said, so the last
235         // word doesn't sit un-highlighted while the next clip is fetched.
236         setSpeaking({ turnId, chunkIndex: i, spokenInChunk: words.length });
237     }
238 } finally {
239     // Awaited, not fire-and-forget: the in-flight prefetch has to land
240     // before its URL can be revoked, or it leaks exactly in the barge-in
241     // case this is here to cover.
242     void pending.finally(() => created.forEach((url) => URL.revokeObjectURL(url)));
243     if (runId === playbackIdRef.current) {
244         currentAudioRef.current = null;
245         setSpeaking(null);
246         setMicState('idle');
247     }
248 }
249 };
250
251 // Fallback path: covers turns that arrive without going through this
252 // component's own submit handlers below (specifically the opening AI line
253 // of a session that was JUST freshly started). The handlers call
254 // playTurnAudio directly as the primary path for every other turn – that
255 // keeps play() tied as closely as possible to the actual user gesture
256 // (mic tap / send click), since routing it only through a state effect
257 // several async hops away from the click risks Chromium's autoplay policy
258 // silently dropping it.

```

```

259 //
260 // Resuming an existing session also runs through fetchSessionDetail and
261 // lands here with turns already populated – that must NOT trigger
262 // playback (it used to, and would loudly replay whatever the AI last said
263 // the moment you opened an old conversation). 'justStarted' distinguishes
264 // the two: only startSession sets it, and consuming it here resets it
265 // immediately so it can't leak into a later resume in the same app session.
266 useEffect(() => {
267   if (!isVoice) return;
268   const lastAiTurn = [...turns].reverse().find((t) => t.speaker === 'ai');
269   if (!lastAiTurn?.audio_url || lastAiTurn.id === lastAutoPlayedTurnId.current) return;
270
271   const isFreshOpeningLine = turns.length === 1 && useConversationStore.getState().consumeJustStarted();
272   if (isFreshOpeningLine) {
273     void playTurnAudio(lastAiTurn.id, lastAiTurn.audio_chunks);
274   } else {
275     // Not something to auto-play (a resumed session's existing history) –
276     // still mark it "seen" so nothing tries to play it later either.
277     lastAutoPlayedTurnId.current = lastAiTurn.id;
278   }
279   // eslint-disable-next-line react-hooks/exhaustive-deps
280 }, [turns.length, isVoice]);
281
282 /** The one path both tap-to-talk and hands-free capture funnel through. */
283 const submitAudio = async (blob: Blob) => {
284   if (!activeSession) return;
285   setPendingUserText('🎤 ...'); // real text isn't known until the backend transcribes it
286   setError(null);
287   try {
288     const { ai_turn } = await submitAudioTurn(activeSession.id, blob);
289     setPendingUserText(null);
290     if (isVoice && ai_turn.audio_url) {
291       void playTurnAudio(ai_turn.id, ai_turn.audio_chunks);
292     } else {
293       setMicState('idle');
294     }
295   } catch (err) {
296     setPendingUserText(null);
297     setError(err instanceof Error ? err.message : 'Something went wrong');
298     setMicState('idle');
299     // A failed turn can mean the engine itself is no longer usable (key
300     // revoked, quota gone) – refresh so the header indicator says so now
301     // rather than at the next screen change.
302     void fetchEngineGlobalStatus();
303   }
304 };
305
306 const vad = useVadRecorder({
307   enabled: handsFree && isVoice && aiLaunched && activeSession !== null,
308   paused: turnInFlight,
309   // While the reply is audible the bar is raised, so the AI's own voice
310   // leaking through the speakers doesn't read as the learner interrupting.
311   thresholdScale: micState === 'speaking' ? 2.4 : 1,
312   onSpeechStart: () => {
313     if (stopPlayback()) setMicState('idle');
314   },
315   onUtterance: (blob) => void submitAudio(blob),
316 });
317
318 const handleMicClick = async () => {
319   setError(null);
320   if (!aiLaunched) {
321     setError('AI isn't launched yet – use the Launch AI button below before talking.');
```

```

335     setMicState('idle');
336     return;
337   }
338   await submitAudio(blob);
339 }
340 };
341
342 const handleTextSubmit = async (e: React.FormEvent) => {
343   e.preventDefault();
344   const text = textInput.trim();
345   // Enter submits the form even while the send button is disabled, so the
346   // in-flight guard has to live here too, not only on the button.
347   if (!text || !activeSession || turnInFlight) return;
348   if (!aiLaunched) {
349     setError('AI isn't launched yet – use the Launch AI button below before sending.');
```

350 return;

351 }

352 setTextInput('');

353 setError(null);

354 setPendingUserText(text);

355 try {

356 const { ai_turn } = await submitTextTurn(activeSession.id, text);

357 setPendingUserText(null);

358 if (isVoice && ai_turn.audio_url) {

359 void playTurnAudio(ai_turn.id, ai_turn.audio_chunks);

360 }

361 } catch (err) {

362 setPendingUserText(null);

363 // Put the message back in the box. The input is cleared optimistically

364 // so the conversation feels responsive, but a turn can genuinely fail

365 // (AI not launched, quota gone, model still loading) and losing what

366 // someone just typed makes a recoverable error feel like data loss –

367 // they have to retype it to retry.

368 setTextInput((current) => (current ? current : text));

369 setError(err instanceof Error ? err.message : 'Something went wrong');

370 void fetchEngineGlobalStatus();

371 }

372 };
373
374 const handleEnd = async () => {
375 if (!activeSession) return;
376 setEnding(true);
377 try {
378 await endSession(activeSession.id);
379 goReport();
380 } catch (err) {
381 setError(err instanceof Error ? err.message : 'Could not end the session');

382 } finally {

383 setEnding(false);

384 }

385 };
386
387 if (activeStatus === 'loading' || !activeSession) {
388 return (
389 <div className="flex h-full items-center justify-center font-mono text-[11px] text-tx3">
390 {activeStatus === 'loading' ? 'loading...' : 'no active session'}
391 </div>
392);
393 }
394
395 // In hands-free mode the circle is a status light, not a button: what it
396 // shows comes from the detector, except while a turn is in flight or the
397 // reply is playing, which the component itself still owns.
398 const displayState: MicState = handsFree
399 ? turnInFlight
400 ? 'thinking'
401 : micState === 'speaking'
402 ? 'speaking'
403 : vad.phase === 'capturing'
404 ? 'hearing'
405 : vad.phase === 'listening'
406 ? 'listening'
407 : 'idle'
408 : micState;
409 const meta = MIC_META[displayState];
410 const handsFreeUnavailable = handsFree && !isVoice;

```

411     return (
412       <div className="flex h-full w-full min-h-0">
413         <div className="flex min-w-0 flex-1 flex-col">
414           <div className="flex flex-none flex-wrap items-center gap-[10px] border-b border-line2 px-[var(--pad)] py-
3">
415             <button
416               onClick={() => goScreen('conv')}
417               className="flex items-center gap-[6px] rounded-field border border-line2 px-[9px] py-[5px] font-mono
418               text-[10.5px] font-medium text-tx2 hover:border-acc hover:text-acc"
419             >
420               < sessions
421             </button>
422             <span className="rounded-field bg-panel2 px-[9px] py-1 font-mono text-[10.5px] font-medium text-tx2">
{scenario}</span>
423             <span className="rounded-field bg-panel2 px-[9px] py-1 font-mono text-[10.5px] font-medium text-tx2">
424               {activeSession.channel}
425             </span>
426             <div className="flex-1" />
427             <button
428               onClick={() => void handleEnd()}
429               disabled={ending}
430               className="rounded-field border border-line px-[11px] py-[6px] font-mono text-[11px] font-medium text-
431               tx2 hover:border-acc hover:text-acc disabled:opacity-50"
432             >
433               {ending ? 'analysing...' : 'End & analyse'}
434             </button>
435           </div>
436           <div className="flex flex-1 flex-col gap-[14px] overflow-y-auto p-[var(--pad)]">
437             {turns.map((t) => {
438               const isAi = t.speaker === 'ai';
439               return (
440                 <div
441                   key={t.id}
442                   className="flex max-w-[78%] gap-[11px]"
443                   style={{ flexDirection: isAi ? 'row' : 'row-reverse', alignSelf: isAi ? 'flex-start' : 'flex-end' }}
444                 >
445                   <div
446                     className="grid h-7 w-7 flex-none place-items-center rounded-full font-mono text-[8px] font-
447                     semibold"
448                     style={{
449                       background: isAi ? 'var(--accSoft)' : 'var(--tile)',
450                       color: isAi ? 'var(--acc)' : 'var(--tx3)',
451                     }}
452                   >
453                     {isAi ? 'fox' : 'you'}
454                   </div>
455                   <div>
456                     <div
457                       className="rounded-field border px-[14px] py-[11px] font-sans text-[13.5px] leading-[1.65]"
458                       style={{
459                         background: isAi ? 'var(--panel)' : 'var(--accSoft)',
460                         borderColor: isAi ? 'var(--line2)' : 'var(--accLine)',
461                         color: 'var(--tx)',
462                       }}
463                     >
464                       {t.text ? (
465                         <SpokenText
466                           text={t.text}
467                           chunks={t.audio_chunks}
468                           speaking={speaking?.turnId === t.id ? speaking : null}
469                         />
470                       ) : (
471                         <span className="italic text-tx3">(no speech detected)</span>
472                       )}
473                     </div>
474                     <div
475                       className="mt-[5px] flex items-center gap-[8px] font-mono text-[9.5px] text-tx3"
476                       style={{ justifyContent: isAi ? 'flex-start' : 'flex-end' }}
477                     >
478                       {isAi ? 'Juno' : t.stt_confidence !== null ? `confidence ${Math.round(t.stt_confidence * 100)}%`
479                     : ''}
480                       {t.audio_url && (
481                         <button onClick={() => void playTurnAudio(t.id, t.audio_chunks)} className="hover:text-acc">
482                           ▶ play

```

```

481         </button>
482     )}
483 </div>
484 </div>
485 </div>
486 );
487 }}}
488 {pendingUserText && (
489     <div className="flex max-w-[78%] gap-[11px]" style={{ flexDirection: 'row-reverse', alignSelf: 'flex-
end' }}>
490         <div className="grid h-7 w-7 flex-none place-items-center rounded-full bg-tile font-mono text-[8px]
font-semibold text-tx3">
491             you
492         </div>
493         <div>
494             <div
495                 className="rounded-field border px-[14px] py-[11px] font-sans text-[13.5px] leading-[1.65] text-tx
opacity-60"
496                 style={{ background: 'var(--accSoft)', borderColor: 'var(--accLine)' }}
497             >
498                 {pendingUserText}
499             </div>
500             <div className="mt-[5px] text-right font-mono text-[9.5px] text-tx3">sending...</div>
501         </div>
502     </div>
503 )}
504 <div ref={transcriptEndRef} />
505 </div>
506
507 <div className="flex flex-none flex-col items-center gap-3 border-t border-line2 px-[var(--pad)] pb-[18px]
pt-[14px]">
508     {!aiLaunched && (
509         <div className="flex w-full max-w-[500px] flex-wrap items-center gap-[10px] rounded-field border border-
accLine bg-accSoft px-3 py-[10px]">
510             <span className="flex-1 font-mono text-[10.5px] leading-[1.6] text-tx2">
511                 AI isn't launched yet – nothing can be sent until it's loaded into memory.
512             </span>
513             <button
514                 onClick={() => void launchAi().catch(() => {})}
515                 disabled={aiLaunching}
516                 className="rounded-field bg-acc px-3 py-[6px] font-mono text-[10.5px] font-semibold text-white
hover:brightness-110 disabled:opacity-50"
517             >
518                 {aiLaunching ? 'launching...' : 'Launch AI'}
519             </button>
520             {aiLaunchError && <span className="w-full font-mono text-[10px] text-[#c0563f]">{aiLaunchError}
</span>}
521         </div>
522     )}
523     {error && <div className="font-mono text-[10.5px] text-[#c0563f]">{error}</div>}
524     {vad.error && handsFree && (
525         <div className="font-mono text-[10.5px] text-[#c0563f]">hands-free needs the mic: {vad.error}</div>
526     )}
527 <div className="flex items-center gap-[14px]">
528     <button
529         onClick={() => void handleMicClick()}
530         disabled={
531             handsFree || micState === 'thinking' || micState === 'speaking' || turnInFlight || !aiLaunched
532         }
533         title={handsFree ? 'hands-free is on – just talk' : undefined}
534         className="relative grid h-14 w-14 place-items-center rounded-full border-2 font-mono text-[9px] font-
semibold disabled:opacity-60"
535         style={{ background: meta.bg, borderColor: meta.bd, color: meta.fg }}
536     >
537         {/* Live input level, so it's obvious the mic is actually hearing
538            something before anything is sent anywhere. */}
539         {handsFree && (displayState === 'listening' || displayState === 'hearing') && (
540             <span
541                 className="pointer-events-none absolute inset-[-4px] rounded-full border transition-opacity"
542                 style={{ borderColor: 'var(--acc)', opacity: Math.min(1, vad.level * 14) }}
543             />
544         )}
545         {meta.icon}
546     </button>
547     <div className="w-[160px]">
548         <div className="font-sans text-[12px] font-semibold" style={{ color: meta.fg }}>

```

```

549         {meta.label}
550     </div>
551     <div className="font-mono text-[10px] text-tx3">
552         {handsFreeUnavailable ? 'text session - type below' : meta.sub}
553     </div>
554 </div>
555 <button
556     onClick={() => {
557         setHandsFree((on) => !on);
558         setError(null);
559         setMicState('idle');
560     }}
561     disabled={!isVoice}
562     title={
563         isVoice
564         ? 'Hands-free listens continuously and lets you interrupt the reply by talking over it'
565         : 'Hands-free only applies to voice sessions'
566     }
567     className="rounded-field border px-[11px] py-[7px] font-mono text-[10px] font-medium disabled:opacity-
40"
568     style={{
569         borderColor: handsFree && isVoice ? 'var(--accLine)' : 'var(--line2)',
570         background: handsFree && isVoice ? 'var(--accSoft)' : 'transparent',
571         color: handsFree && isVoice ? 'var(--acc)' : 'var(--tx3)',
572     }}
573 >
574     hands-free {handsFree ? 'on' : 'off'}
575 </button>
576 </div>
577 <form onSubmit={handleTextSubmit} className="flex w-full max-w-[500px] items-center gap-2">
578     <input
579         value={textInput}
580         onChange={(e) => setTextInput(e.target.value)}
581         placeholder="or type instead..."
582         className="min-w-0 flex-1 rounded-field border border-line2 bg-panel2 px-3 py-2 font-sans text-
[11.5px] text-tx placeholder:text-tx3 focus:border-acc focus:outline-none"
583     />
584     <button
585         type="submit"
586         disabled={!textInput.trim() || turnInFlight || !aiLaunched}
587         className="rounded-field border border-accLine bg-accSoft px-[13px] py-2 font-mono text-[11px] font-
medium text-acc disabled:opacity-50"
588     >
589         send
590     </button>
591 </form>
592 </div>
593 </div>
594
595 <aside className="flex w-[270px] flex-none flex-col overflow-y-auto border-l border-line2 bg-panel">
596     <div className="flex items-center gap-[11px] border-b border-line2 p-4">
597         <div className="grid h-11 w-11 flex-none place-items-center rounded-full border border-accLine bg-accSoft
font-mono text-[7.5px] text-acc">
598             fox
599         </div>
600         <div className="min-w-0">
601             <div className="font-sans text-[12.5px] font-semibold text-tx">Juno</div>
602             <div className="truncate font-mono text-[10px] text-tx3" title={activeSession.engine_label}>
603                 {sessionUsesCloud ? 'cloud' : 'local'} · {activeSession.engine_label}
604             </div>
605         </div>
606     </div>
607
608     <div className="p-4">
609         <div className="mb-[11px] font-mono text-[9px] font-semibold uppercase tracking-[0.12em] text-tx3">
610             Target words · {activeSession.target_words.length}
611         </div>
612         {activeSession.target_words.length === 0 ? (
613             <div className="font-mono text-[10.5px] text-tx3">
614                 save some words in Vocabulary and they'll show up here next session
615             </div>
616         ) : (
617             <div className="flex flex-col gap-[5px]">
618                 {activeSession.target_words.map((w) => (
619                     <div
620                         key={w.id}

```

```

621         className="flex items-center justify-between gap-2 rounded-field border border-line2 px-[10px] py-
2 text-left"
622         >
623         <span className="font-sans text-[12px] font-medium text-tx">{w.word}</span>
624         <span className="font-mono text-[9px] font-medium text-tx3">{w.used_outcome ?? 'due'}</span>
625         </div>
626     )}
627 </div>
628 )}
629 <div className="mt-[14px] border-t border-line2 pt-3 font-mono text-[10px] leading-[1.7] text-tx3">
630     the model is asked to never say a target word first
631 </div>
632 </div>
633 </aside>
634 </div>
635 );
636 }

```

18. `renderer/src/features/conversation/Report.tsx`

The post-session report. Shows computed and judged figures side by side, labelled, and renders 'not measured in this report' for fields absent from an older report version.

`renderer/src/features/conversation/Report.tsx`

324 lines

```

1  import { useEffect, useRef, useState } from 'react';
2  import { CONV_SCENARIOS } from '@features/conversation/conversationMockData';
3  import { useConversationStore } from '@store/conversationStore';
4  import { useShellStore } from '@store/shellStore';
5
6  const SCENARIO_LABEL: Record<string, string> = Object.fromEntries(CONV_SCENARIOS.map((s) => [s.key, s.n]));
7
8  // Must track REPORT_VERSION in backend/app/services/conversation_report.py.
9  const CURRENT_REPORT_VERSION = 2;
10
11  interface Dial {
12    n: string;
13    v: number | null;
14    sub: string;
15  }
16
17  /** Null means the metric was never measured – for a legacy report, or for a
18   * session that gave nothing to measure. Showing a zero would read as a real,
19   * bad score for something we simply don't know. */
20  function Metric({ value, suffix = '' }: { value: number | null; suffix?: string }) {
21    if (value === null) return <span className="text-tx3"></span>;
22    return (
23      <>
24        {value}
25        {suffix}
26      </>
27    );
28  }
29
30  export function Report() {
31    const goScreen = useShellStore((s) => s.goScreen);
32    const reportOrigin = useShellStore((s) => s.reportOrigin);
33    const report = useConversationStore((s) => s.report);
34    const reportStatus = useConversationStore((s) => s.reportStatus);
35    const reportError = useConversationStore((s) => s.reportError);
36    const regenerateReport = useConversationStore((s) => s.regenerateReport);
37    const startSession = useConversationStore((s) => s.startSession);
38    const goConvLive = useShellStore((s) => s.goConvLive);
39    const sessions = useConversationStore((s) => s.sessions);
40    const activeSession = useConversationStore((s) => s.activeSession);
41    const [regenerating, setRegenerating] = useState(false);
42    const [practising, setPractising] = useState(false);
43    const attemptedRef = useRef<Set<string>>(new Set());
44
45    const backLabel = reportOrigin === 'convlive' ? 'back to session' : 'back to sessions';
46

```

```

47  const handleRegenerate = async () => {
48    if (!report) return;
49    setRegenerating(true);
50    try {
51      await regenerateReport(report.session_id);
52    } finally {
53      setRegenerating(false);
54    }
55  };
56
57  // Opening a report written by an older version brings it up to date by
58  // itself. Attempted once per session: a failure (no AI configured, quota
59  // gone) leaves the old report on screen with a retry rather than looping.
60  const legacySessionId =
61    report && report.report_version < CURRENT_REPORT_VERSION ? report.session_id : null;
62  useEffect(() => {
63    if (!legacySessionId || attemptedRef.current.has(legacySessionId)) return;
64    attemptedRef.current.add(legacySessionId);
65    void handleRegenerate();
66    // eslint-disable-next-line react-hooks/exhaustive-deps
67  }, [legacySessionId]);
68
69  // Repeat the same scenario with the same words, so whatever went badly is
70  // what comes back rather than a fresh unrelated set. The session just ended
71  // isn't in the fetched list yet, so prefer the active one – otherwise
72  // arriving here straight from "End & analyse" would silently reseed.
73  const source =
74    (activeSession?.id === report?.session_id ? activeSession : null) ??
75    sessions.find((s) => s.id === report?.session_id) ??
76    null;
77
78  const handlePractiseAgain = async () => {
79    if (!report) return;
80    setPractising(true);
81    try {
82      const seedIds = source?.target_words.map((w) => w.id);
83      await startSession(source?.scenario ?? 'free', source?.channel ?? 'voice', seedIds?.length ? seedIds :
undefined);
84      goConvLive(SCENARIO_LABEL[source?.scenario ?? 'free'] ?? 'Free talk');
85    } catch {
86      setPractising(false);
87    }
88  };
89
90  if (reportStatus === 'error') {
91    return (
92      <div className="flex h-full flex-col items-center justify-center gap-3 px-[var(--pad)] text-center">
93        <div className="font-sans text-[13px] font-medium text-tx">Couldn't load this report.</div>
94        {reportError && <div className="max-w-[420px] font-mono text-[10.5px] leading-[1.6] text-tx3">{reportError}
</div>}
95        <button
96          onClick={() => goScreen(reportOrigin)}
97          className="mt-1 rounded-field border border-line2 px-4 py-[9px] font-mono text-[11px] font-medium text-tx2
hover:border-acc hover:text-acc"
98        >
99          < {backLabel}
100        </button>
101      </div>
102    );
103  }
104
105  if (reportStatus === 'loading' || !report) {
106    return (
107      <div className="flex h-full items-center justify-center font-mono text-[11px] text-tx3">
108        {reportStatus === 'loading' ? 'loading...' : 'no report to show yet'}
109      </div>
110    );
111  }
112
113  const isLegacy = report.report_version < CURRENT_REPORT_VERSION;
114
115  const dials: Dial[] = [
116    {
117      n: 'Contextual accuracy',
118      v: report.contextual_accuracy_pct,
119      sub: report.contextual_accuracy_pct === null ? 'no target words this session' : 'target words used correctly',

```

```

120   },
121   { n: 'Grammatical precision', v: report.grammatical_precision, sub: 'error density, judged by the model' },
122   { n: 'Lexical range', v: report.lexical_range, sub: 'type-token ratio of your own words' },
123   {
124     n: 'Pronunciation',
125     v: report.pronunciation_score,
126     sub: report.pronunciation_score === null ? 'no audio in this session' : 'stt_proxy · not a phoneme score',
127   },
128 ];
129
130 return (
131   <div className="flex h-full flex-col gap-[14px] overflow-y-auto p-[var(--pad)]">
132     <div className="mx-auto w-full max-w-[1120px]">
133       <button
134         onClick={() => goScreen(reportOrigin)}
135         className="flex items-center gap-[6px] rounded-field border border-line2 px-[9px] py-[5px] font-mono text-
136 [10.5px] font-medium text-tx2 hover:border-acc hover:text-acc"
137       >
138         <svg viewBox="0 0 16 16" className="h-[10px] w-[10px]" fill="none" stroke="currentColor" strokeWidth={1.5}
139 strokeLinecap="round" strokeLinejoin="round">
140           <path d="M9.5 3.5L5 8 4.5 4.5" />
141         </svg>
142         {backLabel}
143       </button>
144     </div>
145     {isLegacy && (
146       <div className="mx-auto flex w-full max-w-[1120px] flex-wrap items-center gap-[10px] rounded-panel border
147 border-accLine bg-accSoft px-4 py-3">
148       <span className="flex-1 font-mono text-[10.5px] leading-[1.6] text-tx2">
149         {regenerating
150           ? 'Bringing this report up to date – re-running the analysis over the saved transcript...'
151           : 'This report predates the current analysis, so several measurements are missing.'}
152       </span>
153       {!regenerating && (
154         <button
155           onClick={() => void handleRegenerate()}
156           className="rounded-field bg-acc px-[13px] py-[7px] font-sans text-[11px] font-semibold text-white
157 hover:brightness-110"
158         >
159           Try again
160         </button>
161       )}
162       {reportError && <span className="w-full font-mono text-[10px] text-[#c0563f]">{reportError}</span>}
163     </div>
164   )}
165
166   <div className="mx-auto grid w-full max-w-[1120px] grid-cols-4 gap-[14px]">
167     {dials.map((d) => (
168       <div
169         key={d.n}
170         className="flex items-center gap-[14px] rounded-panel border border-line2 bg-panel p-4 shadow-panel"
171       >
172         <div
173           className="grid h-[62px] w-[62px] flex-none place-items-center rounded-full"
174           style={{ background: `conic-gradient(var(--acc) ${((d.v ?? 0) / 100) * 360}deg, var(--line2) 0)` }}
175         >
176           <div className="grid h-12 w-12 place-items-center rounded-full bg-panel font-mono text-[14px] font-
177 semibold">
178             {d.v === null ? <span className="text-tx3">—</span> : d.v}
179           </div>
180         </div>
181         <div>
182           <div className="font-sans text-[11.5px] font-semibold text-tx">{d.n}</div>
183           <div className="mt-[3px] font-mono text-[10px] leading-[1.5] text-tx3">
184             {d.v === null ? 'not measured in this report' : d.sub}
185           </div>
186         </div>
187       </div>
188     )})
189   </div>
190
191   <div className="mx-auto w-full max-w-[1120px] rounded-panel border border-line2 bg-panel p-[18px] shadow-
192 panel">
193     <div className="mb-3 font-mono text-[9px] font-semibold uppercase tracking-[0.12em] text-tx3">
194       Real usage this conversation logged for your target words
195     </div>

```

```

190     <div className="grid grid-cols-[1.4fr_1fr_1.6fr] gap-[10px] border-b border-line2 pb-2 font-mono text-[9px]
font-semibold uppercase tracking-[0.1em] text-tx3">
191         <span>Word</span>
192         <span>Usage</span>
193         <span>Evidence · turn</span>
194     </div>
195     {report.routing.length === 0 && (
196         <div className="py-3 font-mono text-[10.5px] text-tx3">no target words in this session</div>
197     )}
198     {report.routing.map((r) => (
199         <div key={r.word} className="grid grid-cols-[1.4fr_1fr_1.6fr] items-center gap-[10px] border-b border-
line2 py-[10px]">
200             <span className="font-sans text-[12.5px] font-semibold text-tx">{r.word}</span>
201             <span
202                 className="justify-self-start rounded-[4px] px-[7px] py-[3px] font-mono text-[9.5px] font-medium"
203                 style={{
204                     background: r.outcome === 'spontaneous' || r.outcome === 'prompted' ? 'var(--accSoft)' : 'var(--
tile)',
205                     color: r.outcome === 'spontaneous' || r.outcome === 'prompted' ? 'var(--acc)' : 'var(--tx3)',
206                 }}
207             >
208                 {r.outcome}
209             </span>
210             <span className="font-sans text-[11px] text-tx2">
211                 {r.evidence_turn !== null ? `turn ${r.evidence_turn}` : '-'}
212             </span>
213         </div>
214     )]}
215     <div className="mt-[11px] font-mono text-[10px] text-tx3">
216         {report.routing.length} row{report.routing.length === 1 ? '' : 's'} written to review_logs with source =
'conversation'
217     </div>
218 </div>
219
220 <div className="mx-auto grid w-full max-w-[1120px] grid-cols-[1.2fr_1fr] gap-[14px]">
221     <div className="rounded-panel border border-line2 bg-panel p-[18px] shadow-panel">
222         <div className="mb-3 font-mono text-[9px] font-semibold uppercase tracking-[0.12em] text-tx3">
223             Instructive errors
224         </div>
225         {report.errors.length === 0 ? (
226             <div className="font-mono text-[10.5px] text-tx3">no notable errors flagged</div>
227         ) : (
228             <div className="flex flex-col gap-3">
229                 {report.errors.map((e, i) => (
230                     <div key={i} className="border-l-2 border-line pl-3">
231                         <div className="font-sans text-[12.5px] leading-[1.6] text-tx3 line-through">{e.bad}</div>
232                         <div className="mt-[2px] font-sans text-[12.5px] leading-[1.6] text-tx">{e.good}</div>
233                         <div className="mt-1 font-mono text-[10px] text-tx3">{e.why}</div>
234                     </div>
235                 )]}
236             </div>
237         )}
238     </div>
239     <div className="flex flex-col gap-[14px]">
240         <div className="rounded-panel border border-line2 bg-panel p-[18px] shadow-panel">
241             <div className="mb-3 font-mono text-[9px] font-semibold uppercase tracking-[0.12em] text-tx3">
242                 Fluency proxies
243             </div>
244             <div className="grid grid-cols-2 gap-3">
245                 <div>
246                     <div className="font-sans text-[22px] font-light tracking-[-0.02em] text-tx">
247                         <Metric value={report.words_per_minute} />
248                     </div>
249                     <div className="font-mono text-[10px] text-tx3">
250                         words / min{report.words_per_minute === null ? ' · no speech timed' : ''}
251                     </div>
252                 </div>
253                 <div>
254                     <div className="font-sans text-[22px] font-light tracking-[-0.02em] text-tx">
255                         <Metric value={report.filler_rate_per_100w} />
256                     </div>
257                     <div className="font-mono text-[10px] text-tx3">fillers / 100 words</div>
258                 </div>
259                 <div>
260                     <div className="font-sans text-[22px] font-light tracking-[-0.02em] text-tx">
261                         <Metric value={report.avg_response_delay_seconds} suffix="s" />

```

```

262         </div>
263         <div className="font-mono text-[10px] text-tx3">avg reply delay</div>
264     </div>
265     <div>
266         <div className="font-sans text-[22px] font-light tracking-[-0.02em] text-tx">
267             <Metric value={report.longest_run_words} />
268         </div>
269         <div className="font-mono text-[10px] text-tx3">longest run (words)</div>
270     </div>
271     <div>
272         <div className="font-sans text-[22px] font-light tracking-[-0.02em] text-tx">
273             <Metric value={report.self_corrections} />
274         </div>
275         <div className="font-mono text-[10px] text-tx3">self-corrections</div>
276     </div>
277     <div>
278         <div className="font-sans text-[22px] font-light tracking-[-0.02em] text-tx">{report.turn_count}</div>
279     </div>
280     <div>
281     </div>
282     {report.above_level_words.length > 0 && (
283         <div className="mt-3 border-t border-line2 pt-[10px]">
284             <div className="mb-[6px] font-mono text-[9px] font-semibold uppercase tracking-[0.1em] text-tx3">
285                 Above your level · {report.above_level_words.length}
286             </div>
287             <div className="flex flex-wrap gap-[5px]">
288                 {report.above_level_words.map((w) => (
289                     <span
290                         key={w}
291                         className="rounded-full border border-accLine px-[8px] py-[2px] font-mono text-[10px] text-
acc"
292                     >
293                         {w}
294                     </span>
295                 ))}
296             </div>
297         </div>
298     )}
299     <div className="mt-3 border-t border-line2 pt-[10px] font-mono text-[9.5px] leading-[1.6] text-tx3">
300         words/min is measured from your speech only · pronunciation is an stt_proxy, not a phoneme score
301     </div>
302 </div>
303 <div className="rounded-panel border border-accLine bg-accSoft p-[18px]">
304     <div className="mb-[9px] font-mono text-[9px] font-semibold uppercase tracking-[0.12em] text-acc">
305         Summary
306     </div>
307     <div className="font-sans text-[13px] leading-[1.7] text-tx">{report.summary}</div>
308     <div className="mt-[14px] flex gap-2">
309         <button
310             onClick={() => void handlePractiseAgain()}
311             disabled={practising}
312             title="Starts a new conversation in the same scenario with these same target words"
313             className="rounded-field bg-acc px-[13px] py-[9px] font-sans text-[11.5px] font-semibold text-white
hover:brightness-110 disabled:opacity-50"
314         >
315             {practising ? 'starting...' : 'Practise this again'}
316         </button>
317     </div>
318 </div>
319 </div>
320 </div>
321 </div>
322 );
323 }

```

19. `renderer/src/features/conversation/LeaveConversationDialog.tsx`

Guards navigation away from a session with work in flight.

`renderer/src/features/conversation/LeaveConversationDialog.tsx`

43 lines

```

1 import { useShellStore } from '@store/shellStore';
2
3 /** Rendered at the app root – appears whenever navigation away from a live
4  * conversation was deferred because a real operation (recording,
5  * transcribing, waiting on the local LLM, or playing back a reply) was in
6  * flight, so it isn't silently abandoned mid-turn. */
7 export function LeaveConversationDialog() {
8   const pendingNav = useShellStore((s) => s.pendingNav);
9   const confirmLeaveConv = useShellStore((s) => s.confirmLeaveConv);
10  const cancelLeaveConv = useShellStore((s) => s.cancelLeaveConv);
11
12  if (!pendingNav) return null;
13
14  return (
15    <div className="fixed inset-0 z-[90] grid place-items-center bg-black/45 p-6" onClick={cancelLeaveConv}>
16      <div
17        className="w-full max-w-[400px] rounded-panel border border-line bg-panel p-5 shadow-
18        [0_24px_60px_rgba(0,0,0,.35)]"
19        onClick={e} => e.stopPropagation()>
20        <div className="font-sans text-[14px] font-semibold text-tx">Leave this conversation?</div>
21        <div className="mt-2 font-sans text-[12px] leading-[1.6] text-tx2">
22          Juno is still recording, transcribing, thinking, or speaking. Leaving now interrupts that – your messages
23          so far stay saved, and you can resume this session later.
24        </div>
25        <div className="mt-4 flex justify-end gap-[8px]">
26          <button
27            onClick={cancelLeaveConv}
28            className="rounded-field border border-line2 px-3 py-[7px] font-mono text-[11px] font-medium text-tx2
29            hover:border-acc hover:text-acc"
30          >
31            stay
32          </button>
33          <button
34            onClick={confirmLeaveConv}
35            className="rounded-field border border-[#c0563f] bg-[#c0563f]/10 px-3 py-[7px] font-mono text-[11px]
36            font-medium text-[#c0563f] hover:bg-[#c0563f]/20"
37          >
38            leave anyway
39          </button>
40        </div>
41      </div>
42    );
43  }

```

Renderer — audio capture and timing

20. `renderer/src/features/conversation/useVadRecorder.ts`

Voice-activity detection. Holds the microphone open, decides when an utterance starts and ends, keeps a pre-roll buffer so the first syllable survives, and raises its threshold while the AI speaks so the reply does not interrupt itself.

`renderer/src/features/conversation/useVadRecorder.ts`

224 lines

```

1 import { useEffect, useRef, useState } from 'react';
2
3 export type VadPhase = 'off' | 'listening' | 'capturing' | 'error';
4
5 interface VadOptions {
6   /** Holds the mic open and runs detection. Turning this off releases the device. */
7   enabled: boolean;
8   /** A turn is already being submitted – keep the stream open but capture nothing. */
9   paused: boolean;
10  /** Raised while the AI's reply is audible so its own voice through the
11   * speakers is far less likely to trip detection and interrupt itself. */

```

```

12   thresholdScale: number;
13   /** Fires the moment speech is detected – the barge-in signal. */
14   onSpeechStart: () => void;
15   /** A complete utterance, captured from before it began through to the pause
16       * that ended it. */
17   onUtterance: (blob: Blob) => void;
18 }
19
20 const FRAME_MS = 50;
21 // Two consecutive loud frames before believing it's speech – one is too easy
22 // to trip with a keyboard clack or a chair creak.
23 const SPEECH_FRAMES = 2;
24 // How long a pause has to last before the utterance counts as finished. Every
25 // millisecond here is dead air the learner feels, so it is kept just long
26 // enough that a normal mid-sentence breath doesn't cut them off.
27 const SILENCE_MS = 400;
28 // Matches the backend's own too-short guard, so a blip never round-trips.
29 const MIN_UTTERANCE_MS = 400;
30 const MAX_UTTERANCE_MS = 30000;
31 // Recording runs continuously so an utterance is never clipped at its onset,
32 // which means each blob carries a little audio from before speech began. That
33 // pre-roll is recycled aggressively: anything longer and a barge-in would ship
34 // seconds of the AI's own reply (however well echo cancellation suppressed it)
35 // to the transcriber as if the learner had said it.
36 const IDLE_RESTART_MS = 1500;
37 const NOISE_FLOOR_FRAMES = 10;
38 // Ambient noise is measured per session, but a room this quiet still needs a
39 // floor or the threshold collapses to near zero and everything reads as speech.
40 const ABS_MIN_THRESHOLD = 0.012;
41 const NOISE_FLOOR_MULTIPLE = 3;
42
43 /** Continuous hands-free capture: holds the microphone open, watches the live
44     * signal level, and emits one blob per utterance without anything being
45     * clicked. Detection is energy-based against a per-session ambient noise
46     * floor – deliberately simple and fully local, with the real transcription
47     * still done server-side by whisper (whose own VAD trims the padding this
48     * leaves around each utterance). */
49 export function useVadRecorder({ enabled, paused, thresholdScale, onSpeechStart, onUtterance }: VadOptions) {
50   const [phase, setPhase] = useState<VadPhase>('off');
51   const [level, setLevel] = useState(0);
52   const [error, setError] = useState<string | null>(null);
53
54   // Every value the polling loop reads lives in a ref: the loop is started
55   // once per enable and would otherwise capture the first render's props.
56   const callbacksRef = useRef({ onSpeechStart, onUtterance });
57   callbacksRef.current = { onSpeechStart, onUtterance };
58   const pausedRef = useRef(paused);
59   pausedRef.current = paused;
60   const scaleRef = useRef(thresholdScale);
61   scaleRef.current = thresholdScale;
62
63   useEffect(() => {
64     if (!enabled) {
65       setPhase('off');
66       return;
67     }
68
69     let stream: MediaStream | null = null;
70     let audioContext: AudioContext | null = null;
71     let recorder: MediaRecorder | null = null;
72     let timer: ReturnType<typeof setInterval> | null = null;
73     let chunks: Blob[] = [];
74     let disposed = false;
75
76     let noiseFloor = 0;
77     let floorFrames = 0;
78     let loudFrames = 0;
79     let speaking = false;
80     let speechStartedAt = 0;
81     let lastLoudAt = 0;
82     let recorderStartedAt = 0;
83
84     const startRecorder = () => {
85       if (!stream || disposed) return;
86       chunks = [];
87       recorder = new MediaRecorder(stream);

```

```

88     recorder.ondataavailable = (e) => {
89         if (e.data.size > 0) chunks.push(e.data);
90     };
91     recorder.start();
92     recorderStartedAt = Date.now();
93 };
94
95 /** Closes the current recording and hands back its audio, then immediately
96  * arms a fresh recorder so the next utterance is never missed. */
97 const cutRecording = (): Promise<Blob | null> =>
98     new Promise((resolve) => {
99         const active = recorder;
100         if (!active || active.state === 'inactive') {
101             resolve(null);
102             return;
103         }
104         active.onstop = () => {
105             const blob = chunks.length > 0 ? new Blob(chunks, { type: active.mimeType || 'audio/webm' }) : null;
106             recorder = null;
107             if (!disposed) startRecorder();
108             resolve(blob);
109         };
110         active.stop();
111     });
112
113 const finishUtterance = () => {
114     const spokenMs = Date.now() - speechStartedAt;
115     speaking = false;
116     loudFrames = 0;
117     setPhase('listening');
118     void cutRecording().then((blob) => {
119         if (disposed || !blob || spokenMs < MIN_UTTERANCE_MS) return;
120         callbacksRef.current.onUtterance(blob);
121     });
122 };
123
124 const tick = (analyser: AnalyserNode, buffer: Float32Array<ArrayBuffer>) => {
125     analyser.getFloatTimeDomainData(buffer);
126     let sumSquares = 0;
127     for (let i = 0; i < buffer.length; i += 1) sumSquares += buffer[i] * buffer[i];
128     const rms = Math.sqrt(sumSquares / buffer.length);
129     // Quantised so the live meter doesn't re-render the transcript 20x/second.
130     setLevel((prev) => (Math.abs(prev - rms) > 0.004 ? rms : prev));
131
132     if (floorFrames < NOISE_FLOOR_FRAMES) {
133         floorFrames += 1;
134         noiseFloor += (rms - noiseFloor) / floorFrames;
135         return;
136     }
137
138     const now = Date.now();
139     const threshold = Math.max(ABS_MIN_THRESHOLD, noiseFloor * NOISE_FLOOR_MULTIPLE) * scaleRef.current;
140
141     if (pausedRef.current) {
142         // Drop anything captured while a turn was mid-flight rather than
143         // stitching it onto the next utterance – including the idle buffer,
144         // which would otherwise grow for the whole round trip.
145         if (speaking) {
146             speaking = false;
147             loudFrames = 0;
148             setPhase('listening');
149             void cutRecording();
150         } else if (now - recorderStartedAt > IDLE_RESTART_MS) {
151             void cutRecording();
152         }
153         return;
154     }
155
156     if (rms > threshold) {
157         loudFrames += 1;
158         lastLoudAt = now;
159         if (!speaking && loudFrames >= SPEECH_FRAMES) {
160             speaking = true;
161             speechStartedAt = now;
162             setPhase('capturing');
163             callbacksRef.current.onSpeechStart();

```

```

164     }
165   } else {
166     loudFrames = 0;
167     if (speaking && now - lastLoudAt >= SILENCE_MS) {
168       finishUtterance();
169       return;
170     }
171     if (!speaking && now - recorderStartedAt > IDLE_RESTART_MS) {
172       void cutRecording();
173     }
174   }
175
176   if (speaking && now - speechStartedAt > MAX_UTTERANCE_MS) finishUtterance();
177 };
178
179 void (async () => {
180   try {
181     stream = await navigator.mediaDevices.getUserMedia({
182       // Echo cancellation is what makes barge-in workable on speakers at
183       // all - without it the AI's own reply reliably interrupts itself.
184       audio: { echoCancellation: true, noiseSuppression: true, autoGainControl: true },
185     });
186     if (disposed) {
187       stream.getTracks().forEach((t) => t.stop());
188       return;
189     }
190     audioContext = new AudioContext();
191     if (audioContext.state === 'suspended') await audioContext.resume();
192     const analyser = audioContext.createAnalyser();
193     analyser.fftSize = 1024;
194     analyser.smoothingTimeConstant = 0;
195     audioContext.createMediaStreamSource(stream).connect(analyser);
196     const buffer = new Float32Array(analyser.fftSize);
197
198     startRecorder();
199     setError(null);
200     setPhase('listening');
201     timer = setInterval(() => tick(analyser, buffer), FRAME_MS);
202   } catch (err) {
203     if (disposed) return;
204     setPhase('error');
205     setError(err instanceof Error ? err.message : 'Microphone access was denied');
206   }
207 })();
208
209 return () => {
210   disposed = true;
211   if (timer) clearInterval(timer);
212   if (recorder && recorder.state !== 'inactive') {
213     recorder.onstop = null;
214     recorder.stop();
215   }
216   stream?.getTracks().forEach((t) => t.stop());
217   void audioContext?.close();
218   setLevel(0);
219 };
220 }, [enabled]);
221
222 return { phase, level, error };
223 }

```

21. `renderer/src/features/conversation/useMicRecorder.ts`

The simpler push-to-talk recorder, used where explicit control is wanted.

`renderer/src/features/conversation/useMicRecorder.ts`

57 lines

```

1 import { useCallback, useRef, useState } from 'react';
2
3 type RecorderStatus = 'idle' | 'recording' | 'error';
4
5 /** Real push-to-talk mic capture via the standard Web Audio APIs - Electron's

```

```

6  * renderer is a full Chromium context, so no native module is needed. */
7  export function useMicRecorder() {
8    const [status, setStatus] = useState<RecorderStatus>('idle');
9    const mediaRecorderRef = useRef<MediaRecorder | null>(null);
10   const chunksRef = useRef<Blob[]>([]);
11   const streamRef = useRef<MediaStream | null>(null);
12
13   /** Resolves to null on success, or the failure message – returned rather
14    * than only stored in state because a caller awaiting start() still holds
15    * its own render's closure, where a just-set error state is not yet
16    * visible (that read silently showed "Recording..." after a denied mic). */
17   const start = useCallback(async (): Promise<string | null> => {
18     try {
19       const stream = await navigator.mediaDevices.getUserMedia({ audio: true });
20       streamRef.current = stream;
21       chunksRef.current = [];
22       const recorder = new MediaRecorder(stream);
23       recorder.ondataavailable = (e) => {
24         if (e.data.size > 0) chunksRef.current.push(e.data);
25       };
26       mediaRecorderRef.current = recorder;
27       recorder.start();
28       setStatus('recording');
29       return null;
30     } catch (err) {
31       setStatus('error');
32       return err instanceof Error ? err.message : 'Microphone access was denied';
33     }
34   }, []);
35
36   const stop = useCallback(() : Promise<Blob | null> => {
37     return new Promise((resolve) => {
38       const recorder = mediaRecorderRef.current;
39       if (!recorder || recorder.state === 'inactive') {
40         resolve(null);
41         return;
42       }
43       recorder.onstop = () => {
44         const blob = new Blob(chunksRef.current, { type: recorder.mimeType || 'audio/webm' });
45         streamRef.current?.getTracks().forEach((t) => t.stop());
46         streamRef.current = null;
47         mediaRecorderRef.current = null;
48         setStatus('idle');
49         resolve(blob);
50       };
51       recorder.stop();
52     });
53   }, []);
54
55   return { status, start, stop };
56 }

```

22. `renderer/src/features/conversation/spokenTiming.ts`

Distributes the reply's words across each audio chunk's duration, weighted by word length. An estimate, and deliberately an honest one — Pocket TTS returns no word timings.

`renderer/src/features/conversation/spokenTiming.ts`

68 lines

```

1  /** Deciding which word the voice is on, given where the audio is.
2   *
3   * Neither TTS engine reports word timings, and forcing an alignment would
4   * cost more than the feature is worth. What we have instead is exact: the
5   * clip's real duration and its exact text. Distributing the words across that
6   * duration by their spoken weight is an estimate, but it is anchored at both
7   * ends and re-read from `audio.currentTime` every frame – so it cannot drift,
8   * and a stall or a slow chunk moves the highlight with the sound rather than
9   * away from it.
10  */
11
12  /** Roughly how long a word takes to say, relative to other words.

```

```

13 *
14 * Length dominates, but not from zero: even a one-letter word takes time to
15 * articulate, so there is a floor. Trailing punctuation buys extra weight
16 * because the pause after a comma or a full stop is real time during which no
17 * new word is being said – without it every sentence-final word lights up
18 * early and the highlight runs ahead of the voice for the rest of the clip. */
19 export function wordWeight(word: string): number {
20   const letters = word.replace(/[\p{L}\p{N}']/gu, '').length;
21   let weight = 1 + letters;
22   if (/[,;:]\$/ .test(word)) weight += 1.5;
23   if (/[\.!?\_]\$/ .test(word)) weight += 2.5;
24   if (/[\---]\$/ .test(word)) weight += 1;
25   return weight;
26 }
27
28 export interface TimedWord {
29   text: string;
30   /** Fraction of the clip (0-1) at which this word starts being spoken. */
31   start: number;
32   /** Fraction of the clip (0-1) at which the next word takes over. */
33   end: number;
34 }
35
36 /** Splits a chunk into words, each with the slice of the clip it occupies.
37  * Whitespace is kept on the word it follows so re-joining reproduces the
38  * original text exactly. */
39 export function timeWords(chunk: string): TimedWord[] {
40   const tokens = chunk.match(/[\S+\s*/g) ?? [];
41   const weights = tokens.map((t) => wordWeight(t.trim()));
42   const total = weights.reduce((a, b) => a + b, 0);
43   if (total <= 0) return [];
44
45   let acc = 0;
46   return tokens.map((text, i) => {
47     const start = acc / total;
48     acc += weights[i];
49     return { text, start, end: acc / total };
50   });
51 }
52
53 /** How many words of `timed` have been reached at `fraction` through the clip.
54  *
55  * A word counts as spoken once the voice has *started* it, not once it has
56  * finished – highlighting on completion always reads as lagging, because the
57  * listener has already heard the word by then. */
58 export function spokenCount(timed: TimedWord[], fraction: number): number {
59   if (fraction <= 0) return 0;
60   if (fraction >= 1) return timed.length;
61   let n = 0;
62   for (const w of timed) {
63     if (fraction >= w.start) n += 1;
64     else break;
65   }
66   return n;
67 }

```

23. `renderer/src/features/conversation/SpokenText.tsx`

Renders the reply and lights each word as it is spoken.

`renderer/src/features/conversation/SpokenText.tsx`

78 lines

```

1 import { useMemo } from 'react';
2 import { timeWords } from './spokenTiming';
3
4 export interface SpeakingState {
5   /** Index into `chunks` of the clip currently audible. */
6   chunkIndex: number;
7   /** How many words of that clip the voice has reached.
8    *
9    * A word count rather than a 0-1 position, deliberately: the player already
10    * has to work out which word is being said, and passing that through means

```

```

11  * this re-renders once per word instead of once per animation frame. On a
12  * four-core machine a 60fps re-render of the whole transcript is a real
13  * cost for an effect nobody can see between words. */
14  spokenInChunk: number;
15  }
16
17  interface Props {
18    text: string;
19    /** The exact per-clip split the synthesizer used, from the API. */
20    chunks: string[];
21    /** Absent when this turn isn't the one being spoken right now. */
22    speaking: SpeakingState | null;
23  }
24
25  /** A reply that lights up word by word as the voice reaches each word.
26  *
27  * Words not yet spoken are dimmed rather than hidden. Hiding them makes the
28  * bubble grow line by line, which reflows everything below it on every word
29  * and pushes the conversation around while the learner is trying to read –
30  * and it takes away the choice to read ahead, which is a real part of
31  * following speech in a language you are still learning.
32  *
33  * Once a turn has finished being spoken it renders as plain text, so old
34  * replies in the transcript carry no leftover highlighting. */
35  export function SpokenText({ text, chunks, speaking }: Props) {
36    const timed = useMemo(() => chunks.map((c) => timeWords(c)), [chunks]);
37
38    if (!speaking || chunks.length === 0) {
39      return <>{text}</>;
40    }
41
42    return (
43      <>
44        {timed.map((words, chunkIndex) => {
45          // Everything before the audible clip has already been said; everything
46          // after it has not been started.
47          const spoken =
48            chunkIndex < speaking.chunkIndex
49              ? words.length
50              : chunkIndex > speaking.chunkIndex
51                ? 0
52                : speaking.spokenInChunk;
53
54          return words.map((w, i) => {
55            const isSpoken = i < spoken;
56            // The word being said right now, rather than merely already said.
57            const isCurrent = i === spoken - 1;
58            return (
59              <span
60                key={` ${chunkIndex}-${i} `}
61                style={{
62                  color: isSpoken ? 'var(--tx)' : 'var(--tx3)',
63                  // Only the leading edge is emphasised. Colouring the whole
64                  // spoken half would turn the bubble into two blocks and lose
65                  // the one thing this is for: where the voice is right now.
66                  textShadow: isCurrent ? '0 0 12px var(--accLine)' : undefined,
67                  transition: 'color 140ms ease-out',
68                }}
69              >
70                {w.text}
71              </span>
72            );
73          });
74        })}
75      </>
76    );
77  }

```